



UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide

Algoritmi e complessità

APPUNTI LEZIONI

Prof. Domenico Cantone

Anno Accademico 2025 - 2026

Indice

1	Analisi Ammortizzata	4
1.1	Metodo dell'aggregazione	5
1.1.1	Stack MultiPop	5
1.1.2	Contatore binario	6
1.2	Metodo dell'accantonamento	7
1.2.1	Stack MultiPop	7
1.2.2	Contatore binario	8
1.3	Metodo del potenziale	8
1.3.1	Stack Multipop inizialmente vuoto	9
1.3.2	Stack Multipop non vuoto	10
1.3.3	Contatore binario inizialmente nullo	10
1.3.4	Contatore binario inizialmente non nullo	10
1.4	Tabelle dinamiche	11
1.4.1	Analisi mediante metodo dell'aggregazione	11
1.4.2	Analisi mediante metodo degli accantonamenti	11
1.4.3	Analisi con il metodo del potenziale	12
1.4.4	Tabelle dinamiche con cancellazione	13
1.5	Scalare i costi	15
1.6	Algoritmi Online vs Algoritmi Offline	16
1.7	Self-adjusting list	16
2	Splay Trees	20
2.1	Zig, Zig-Zag, Zig-Zig	21
2.2	Analisi ammortizzata di Splay	22
2.3	Top-Down Splay Trees	26
3	B-Trees	28
3.1	Ottimizzazione su Hard Disk	28
3.2	Definizione e altezza di un B-tree	29
3.3	Operazioni di base sui B-Tree	31
3.4	B-Tree-Search(x,k)	31
3.5	B-Tree-Create	32
3.6	Inserimento in un B-Tree	32
3.7	Cancellazione da un B-Tree	35

4	Heap Binomiali	37
4.1	Alberi binomiali	37
4.2	Heap binomiali	39
4.3	Operazioni Heap Binomiali	39
4.3.1	Make-Binomial-Heap()	40
4.3.2	Binomial-Heap-Minimum(H)	40
4.3.3	Binomial-Link(y, z)	40
4.3.4	Binomial-Heap-Union(H_1, H_2)	41
4.3.5	Binomial-Heap-Insert(H, x)	42
4.3.6	Binomial-Heap-Extract-min(H)	43
4.3.7	Binomial-Heap-Decrease-key(H, x, k)	43
4.3.8	Binomial-Heap-Delete(x)	44
4.4	Analisi ammortizzata di un inserimento in uno heap binomiale inizialmente vuoto	44
5	Heap di Fibonacci	45
5.1	Funzione potenziale	46
5.2	Alberi binomiali non ordinati	46
5.3	Operazioni su Heap di Fibonacci	47
5.3.1	Make-Fibonacci-Heap()	48
5.3.2	Fib-Heap-Insert(H, x)	48
5.3.3	Minimum(H)	48
5.3.4	Fib-Heap-Union(H_1, H_2)	49
5.3.5	Fib-Heap-Extract-Min(H)	49
5.3.6	Decrease Key	51
5.3.7	Delete	53
5.4	Proprietà di Fibonacci	53
6	Strutture Dati per Insiemi Disgiunti	57
6.1	Componenti Connesse in un Grafo	57
6.2	Generazione di Labirinti	57
6.3	Percolazione	58
6.4	Implementazione di Insiemi Disgiunti	58
6.5	Insiemi Disgiunti mediante Alberi Disgiunti	61
7	Alberi ricoprenti minimi	63
7.1	Taglio e Colorazione	63
7.2	Algoritmo di Boruvka	65
7.3	Algoritmo di Kruskal	65
7.4	Algoritmo di Prim	66
7.5	Signature di uno Spanning Tree	67
7.6	Minimalità della Signature di un MST	69
7.7	Clustering	70
7.7.1	Algoritmo di Kruskal Clustering	71
7.7.2	Minimum Spanning Tree in Grafi Sparsi	72

8	Cammini Minimi in un Grafo	75
8.1	Algoritmi per Cammini Minimi da Una Sorgente	77
8.1.1	GSSSP Ottimizzato	83
8.1.2	Algoritmo di Bellman-Ford	84
8.1.3	Algoritmo di Dijkstra	84
8.1.4	Algoritmo DAG	86
8.2	Cammini Minimi tra Tutte le Coppie	86
8.2.1	Bellman-Ford per Coppie	87
8.2.2	Algoritmo Floyd-Warshall	88
8.3	Rompere la barriera $O(V^3)$	89
8.3.1	Algoritmo di Johnson per Grafi Sparsi	90
9	Reti di Flusso	92
9.1	Problema del Massimo Flusso	93
9.2	Metodo Ford-Fulkerson	98
9.3	Tagli in Reti di flusso	101
9.4	Teorema del Massimo Flusso/Minimo Taglio	102
10	Edge Connectivity	104

Capitolo 1

Analisi Ammortizzata

Nell'analisi ammortizzata il tempo richiesto per eseguire una serie di operazioni n su una struttura dati viene calcolato come media dei tempi di tutte le operazioni eseguite. Questo tipo di analisi si può utilizzare per dimostrare che il costo medio di un'operazione è piccolo se si considera la media di una sequenza di operazioni anche se magari un'operazione all'interno della sequenza è particolarmente costosa. In caso di analisi ammortizzata non utilizziamo teoria delle probabilità, bensì valutiamo le prestazioni media di ciascuna operazione nel caso peggiore.

Per effettuare queste analisi saranno considerati due esempi in particolare:

1. **Stack Multipop**: uno stack in cui, oltre che le operazioni classiche di pop e push con costo $O(1)$, troviamo un'operazione chiamata **Multipop(S, k)** che effettua, sullo stack S , k pop. Questa operazione avrà come costo $O(\min(|S|, k))$ poiché, se lo stack è vuoto, l'operazione terminerà prima e si fermerà al numero di elementi presenti nello stack.

Multipop(S, k)

- 1: **while** $\neg \text{StackEmpty}(S)$ **and** $k > 0$ **do**
- 2: $\text{Pop}(S)$
- 3: $k \leftarrow k - 1$
- 4: **end while**

2. **Contatore binario con incremento:** questo sarà un array di k elementi che rappresenteranno, dalla cifra più significativa alla cifra meno significativa, un numero che potrà essere al più $2^k - 1$. Questo contatore avrà una sola operazione che è **increment** che non fa altro che aumentare il contatore binario di uno. Nel caso in cui tutti gli elementi dell'array siano 1, cioè abbiamo raggiunto il numero massimo disponibile, effettuando increment il contatore si resetterà con tutti i bit a 0. Questo caso è lo scenario più costoso e la sua complessità è in $O(k)$.

Increment(A)

```

1:  $i \leftarrow 0$ 
2: while  $i < A.length$  and  $A[i] = 1$  do
3:    $A[i] \leftarrow 0$ 
4:    $i \leftarrow i + 1$ 
5: end while
6: if  $i < A.length$  then
7:    $A[i] \leftarrow 1$ 
8: end if
```

Un modo naturale di calcolare la complessità di una sequenza di operazioni n è quello di:

1. Calcolare il caso pessimo di una singola operazione;
2. Moltiplicare il costo per n .

Per quanto questo approccio dia sicuramente una stima corretta, questa è fin troppo pessimistica. Se guardiamo, infatti, una sequenza di operazioni su di uno stack multipop, noteremo che la singola operazione ha complessità $O(n)$ dato che non posso fare più pop di quanti push ho effettuato e, quindi, il costo di n operazioni sarà di $O(n^2)$. Allo stesso modo, avremo che per l'incremento del contatore binario, la complessità della singola operazione è $O(k)$ e, dunque, la complessità di n operazioni sarebbe $O(nk)$. Vedremo adesso come questa stima sia sovrabbondante e possiamo renderla più precisa tramite un'ulteriore analisi.

1.1 Metodo dell'aggregazione

L'analisi basata sul metodo dell'aggregazione dimostra che, per ogni n , una sequenza di operazioni n impiega, nel caso peggiore, un tempo totale $T(n)$ e, quindi, il **costo ammortizzato** per operazione è $T(n)/n$. Avremo dunque la necessità di calcolare $T(n)$ in maniera diretta analizzando le operazioni passo dopo passo.

1.1.1 Stack MultiPop

Consideriamo ora il caso dello *stack multipop*. Supponiamo di avere una sequenza di n operazioni del tipo:

$$\langle op_1, \dots, op_n \rangle \leftarrow \{\text{pop}, \text{push}, \text{multipop}\}.$$

Possiamo espandere ogni operazione `multipop` sostituendola con una serie di `pop`, ottenendo così una nuova sequenza:

$$\langle op'_1, \dots, op'_m \rangle \leftarrow \{\text{pop}, \text{push}\}.$$

Naturalmente, queste due sequenze avranno la stessa complessità totale. Osserviamo che, in ogni prefisso della sequenza,

$$\#\text{Pop}(\langle op'_1, \dots, op'_m \rangle) \leq \#\text{Push}(\langle op'_1, \dots, op'_m \rangle) = \#\text{Push}(\langle op_1, \dots, op_n \rangle) \leq n.$$

Pertanto:

$$\#\text{Pop}(\langle op'_1, \dots, op'_m \rangle) \leq n.$$

Sia dunque $\langle op_1, \dots, op_n \rangle$ la sequenza **più costosa**. Allora:

$$\begin{aligned} T(n) &= \text{costo}(\langle op_1, \dots, op_n \rangle) \\ &= \text{costo}(\langle op'_1, \dots, op'_m \rangle) \\ &= \#\text{Pop}(\langle op'_1, \dots, op'_m \rangle) + \#\text{Push}(\langle op'_1, \dots, op'_m \rangle) \\ &\leq n + n = 2n. \end{aligned}$$

Di conseguenza, il costo ammortizzato per operazione è:

$$\frac{T(n)}{n} \leq \frac{2n}{n} = 2.$$

1.1.2 Contatore binario

Considerando invece il caso del **contatore binario**, notiamo quanto segue in figura:

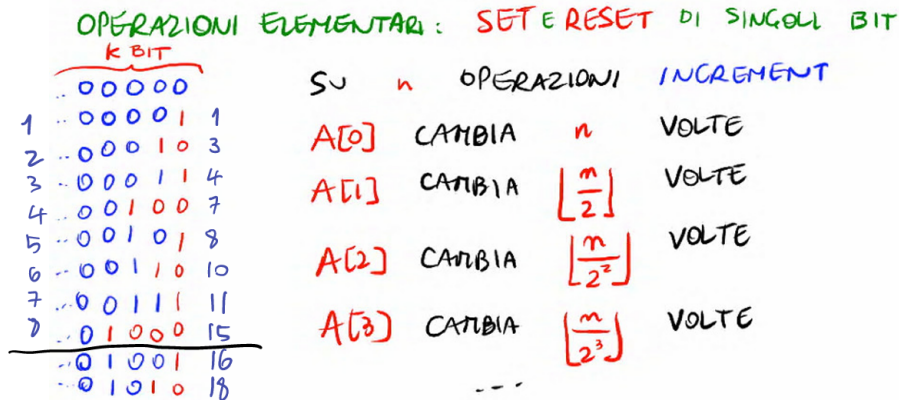


Figura 1.1: Cambi dei bit a ogni incremento

Notiamo quindi come possiamo calcolare:

$$\begin{aligned} T(n) &= n + \left\lfloor \frac{n}{2} \right\rfloor + \dots + \left\lfloor \frac{n}{2^{k-1}} \right\rfloor \\ &= \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq \sum_{i=0}^{k-1} \frac{n}{2^i} < \sum_{i=0}^{\infty} \frac{n}{2^i} \\ &= n \sum_{i=0}^{\infty} \frac{1}{2^i} = n \cdot 2 = 2n. \end{aligned}$$

1

Per cui il costo ammortizzato per operazione è: $\frac{T(n)}{n} < \frac{2n}{n} = 2$.

1.2 Metodo dell'accantonamento

Il metodo degli accantonamenti comincia adesso ad utilizzare l'idea di **ammortizzare** i costi su più operazioni e ciò viene fatto in maniera tale che valga:

$$T(n) = \sum_i^n c_i \leq \sum_i^n c'_i$$

Dove c'_i rappresenta il costo ammortizzato che stiamo definendo e c_i rappresenta il costo reale. Se vale $c'_i > c_i$ allora quello che faremo sarà consumare c_i per il reale costo dell'operazione e il restante "credito", lo terremo da parte per ammortizzare i costi di altre operazioni. Nel caso contrario in cui $c_i > c'_i$, la differenza di crediti per eseguire quella specifica operazione sarà recuperata dai crediti conservati.

Sia D_0 la configurazione iniziale di una struttura dati, siano $op_1, \dots, op_n$ una sequenza di operazioni, e sia $D_i \xrightarrow{op_i} D_{i+1}$ la configurazione della struttura dati dopo l'operazione i -esima. Supponiamo che $\forall j = 0, \dots, n-1$ su D_j sia sempre immagazzinata una quantità di costi pari almeno a $c_{j+1} - c'_{j+1}$, la quantità di costi immagazzinata in D_j è uguale a:

$$D_j = \sum_{i=1}^j (c'_i - c_i) = \sum_i^j c'_i - \sum_i^j c_i$$

E quindi da

$$c_{j+1} - c'_{j+1} \leq \sum_i^j c'_i - \sum_i^j c_i$$

Otteniamo

$$0 \leq \sum_i^j c'_i - \sum_i^j c_i$$

Per cui avremo che

$$\sum_i^{j+1} c_i \leq \sum_i^{j+1} c'_i \rightarrow \sum_i^n c_i \leq \sum_i^n c'_i$$

1.2.1 Stack MultiPop

Per utilizzare il metodo dell'accantonamento nel caso di uno stack con multipop, l'idea è quella di fornire un credito ammortizzato di 2 all'operazione di push. Per cui, un credito lo utilizziamo per il costo reale dell'operazione di push, il restante credito lo utilizzeremo per effettuare l'operazione di pop che, di conseguenza, avrà

¹Il risultato è ottenuto dal fatto che è una serie geometrica.

un costo ammortizzato di 0. Da qui segue:

$$\begin{aligned}
 c'_{push} &= 2 \quad c'_{pop} = 0 \\
 \sum_i^j c'_i - \sum_i^j c_i &= |S_j| \geq 0 \\
 \sum_i^j c_i &\leq \sum_i^j c'_i \rightarrow \sum_i^n c_i \leq \sum_i^n c'_i = 2 * num_{push} \leq 2
 \end{aligned}$$

1.2.2 Contatore binario

Come già fatto prima, anche in questo caso possiamo assegnare ad ogni operazione di set un costo ammortizzato di 2 di cui uno verrà utilizzato per effettuare l'operazione di set e l'altro per l'operazione di reset. In base a ciò, sappiamo che il costo ammortizzato di un operazione di increment sarà $\leq$ di 2 poiché al massimo può esserci una sola operazione di set.² Effettuando l'analisi:

$$\begin{aligned}
 c'_{set} &= 2 \quad c'_{reset} = 0 \\
 \sum_i^j c'_i - \sum_i^j c_i &= \# \text{ bit uguali ad 1} \\
 \sum_i^n c'_i &\geq \sum_i^n c_i \rightarrow \sum_i^n c_i \leq \sum_i^n c'_i \leq 2n
 \end{aligned}$$

1.3 Metodo del potenziale

Il metodo del potenziale risulta essere un caso generico del metodo dell'accantonamento. Questa volta, anziché assegnare un costo ammortizzato alle singole operazioni, andremo a generare una **funzione potenziale** ϕ che conserva i crediti dell'intera struttura dati assegnando ad ogni configurazione D_i un numero reale. Definiamo:

$$\begin{aligned}
 D &\xrightarrow{\phi} \phi(D) \in R \\
 c'_i &= c_i + \phi(D_i) - \phi(D_{i-1}) \\
 \Delta\phi_i &= \phi(D_i) - \phi(D_{i-1}) \quad \text{Variazione di potenziale}
 \end{aligned}$$

²Il minore c'è perché quando il contatore raggiunge il suo valore massimo e vengono fatte solo operazioni di reset si utilizza solo il credito accumulato.

Possiamo continuare:

$$\begin{aligned}
\sum_i^n c'_i &= \sum c_i + \phi(D_i) - \phi(D_{i-1}) \\
&= c_1 + (\phi(D_1) - \phi(D_0)) + c_2 + (\phi(D_2) - \phi(D_1)) + \dots + \\
&\dots + c_{n-1} + \phi(D_{n-1}) - \phi(D_{n-2}) + c_n + (\phi(D_n) - \phi(D_{n-1})) \\
&= \sum_i^n c_i + (\phi(D_n) - \phi(D_0))
\end{aligned}$$

Segue:

$$\sum_i^n c'_i = \sum_i^n c_i + \phi(D_n) - \phi(D_0)$$

Per cui avremo che:

$$\begin{aligned}
\sum_i^n c'_i &\geq \sum_i^n c_i \rightarrow \sum_i^n c_i + \phi(D_n) - \phi(D_0) \geq \sum_i^n c_i \\
\phi(D_n) &\geq \phi(D_0) \xrightarrow{\text{implica}} \sum_i^n c'_i \geq \sum_i^n c_i
\end{aligned}$$

Che sono in genere le funzioni potenziali che riteniamo più utili.

1.3.1 Stack Multipop inizialmente vuoto

La funzione potenziale ϕ che utilizzeremo sarà $\phi(S) = |S|$ se S_0 è lo stack vuoto, avremo che $\phi(S_0) = 0$, quindi avremo che $\phi(S) \geq \phi(S_0)$.³ Analizziamo adesso i costi uno ad uno:

$$\begin{aligned}
c'_{pop} &= c_{pop} + \phi(S_i) - \phi(S_{i-1}) = 1 + |S_i| - |S_{i-1}| = 0 \text{ in quanto } |S_i| = |S_{i-1}| + 1 \\
c'_{multipop} &= c_{multipop} + \phi(S_i) - \phi(S_{i-1}) = 0 \text{ per lo stesso motivo di prima} \\
c'_{push} &= c_{push} + \phi(S_i) - \phi(S_{i-1}) = 1 + 1 = 2
\end{aligned}$$

$$\text{Per tanto: } \sum_i^n c_i \leq \sum_i^n c'_i = 2 * num_{push} \leq 2n$$

³Nel caso in cui questa condizione non valga, avremo che il risultato della nostra analisi avrà, oltre che n , una costante.

1.3.2 Stack Multipop non vuoto

Cerchiamo adesso di fare i calcoli con uno stack multipop per cui però $\phi(D_0) = 0$.

$$\begin{aligned}
 \sum_i^n c'_i &= \sum_i^n c_i + \phi(D_n) - \phi(D_0) \\
 \sum_i^n c_i &= \sum_i^n c'_i - \phi(D_n) + \phi(D_0) \\
 &= \sum_i^n c'_i - |S_n| + |S_0| \\
 &\leq \sum_i^n c'_i + |S_0| \leq 2n + |S_0|
 \end{aligned}$$

Da cui avremo una complessità di $O(n + |S_0|)$

1.3.3 Contatore binario inizialmente nullo

Scegliamo in questo caso la funzione potenziale come $\phi(D_i) = \#$ bit uguali ad 1. Visto che il contatore è inizialmente nullo avremo che $\phi(D_0) = 0$ per cui $\phi(D_i) \geq \phi(D_0)$.

$$c'_i = c_i + \phi(D_i) - \phi(D_{i-1})$$

$$c_i = num_{set_i} + num_{reset_i}$$

$$\Delta\phi_i = num_{set_i} - num_{reset_i}$$

Pertanto avremo che:

$$c'_i = c_i + \Delta\phi_i = (num_{set_i} + num_{reset_i}) + (num_{set_i} - num_{reset_i}) = 2 * num_{set_i} \leq 2$$

Da cui:

$$\sum_i^n c_i \leq \sum_i^n c'_i = 2 * \sum_i^n num_{set_i} \leq 2n$$

1.3.4 Contatore binario inizialmente non nullo

Come per lo stack multipop, proviamo a fare i conti per un contatore inizialmente non nullo.

$$\sum_i^n c_i = \sum_i^n c'_i - \phi(D_n) + \phi(D_0) \leq 2n + \phi(D_0) \leq 2n + k$$

Se $k = O(n)$ allora avremo che $2n + k = O(n) \rightarrow \sum_i^n c_i = O(n + k)$

1.4 Tabelle dinamiche

Le tabelle dinamiche sono delle tabelle che aumentano e diminuiscono la loro dimensione per risolvere problemi overflow e di underflow. Sia T una tabella diremo che:

- **size[T]**: dimensione della tabella;
- **num[T]**: numero di elementi in T ;
- $\alpha(T)$: una funzione che restituisce $\frac{\text{num}[T]}{\text{size}[T]}$ se $\text{size}[T] > 0$, 1 altrimenti. Questo valore sarà il **fattore di carico**.

L'inserimento nel caso in cui il fattore di carico sia 1, cioè la tabella è piena, farà sì che la tabella aumenterà le sue dimensioni del doppio. In termini di implementazione, verrà creata una nuova tabella con dimensioni doppie rispetto alla prima e gli elementi verranno ricopiati al suo interno aggiungendo anche l'ultimo da inserire. Partendo convenzionalmente con una tabella di dimensione 0, vediamo che gli inserimenti $2^i + 1$ sono i più costosi poiché sono quelli in cui la tabella è piena e si necessita una espansione. Analizziamo la complessità in termini di inserimenti elementari. Sia $n = 2^k$ il numero di inserimenti, sappiamo che l'inserimento più costoso è il $2^{k-1} + 1$ -Esimo con costo $2^{k-1} + 1 = n/2 + 1$ ⁴

1.4.1 Analisi mediante metodo dell'aggregazione

Andando ad analizzare il costo delle operazioni mediante il metodo dell'aggregazione, partiamo tenendo conto di come ogni operazione di inserimento abbia costo 1 nel caso in cui questo la tabella abbia posizioni disponibili mentre il costo è i se $i - 1$ è una potenza di 2. Per cui, eseguendo i calcoli avremo:

$$\begin{aligned} T(n) &= \\ \sum_i^n c_i &= (2^0 + 2^1 + \dots + 2^{\lfloor \lg(n-1) \rfloor}) + n = \\ &= (2^{\lfloor \lg(n-1) \rfloor + 1} - 1) + n \leq 2^{\lg(n-1) + 1} - 1 + n \\ &= 2(n-1) - 1 + n = 3n - 3 = O(n) \end{aligned}$$

per cui, il costo ammortizzato sarà:

$$c'_i = \frac{3n - 3}{n} = O(1)$$

1.4.2 Analisi mediante metodo degli accantonamenti

Come ipotesi di costo ammortizzato possiamo utilizzare 3 per cui:

- 1 unità servirà per il costo reale dell'inserimento;

⁴Il costo 2^{k-1} è quello derivante dalla copia dell'array.

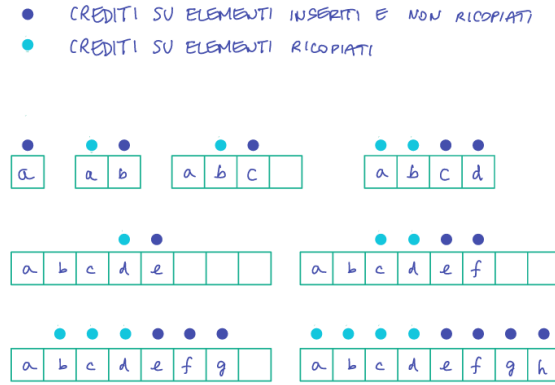


Figura 1.2: Utilizzo dei gettoni per copia

- 1 unità per pagare il costo della propria copia;
- 1 unità per pagare il costo di copia di un elemento che è già stato ricopiato.

L'idea è che ad ogni espansione, avremo nessun elemento precedentemente inserito avrà credito per la successiva copia. Il numero di elementi che necessita di crediti per la successiva copia all' i -esimo inserimento sono 2^{i-1} per cui dividiamo questo credito nei successivi inserimenti che sono $2^{i+1} - 2^i = 2^i$. Per cui ci basterà dividere $2^i / 2^{i-1}$ e avremo che ad ogni inserimento dovremo accumulare 2 crediti, uno per la copia dell'elemento appena inserito e 1 per gli elementi precedenti. Per fissare meglio il concetto di quello che succede, possiamo vedere il seguente schema. Osserviamo che $\sum_i^n c'_i \geq \sum_i^n c_i$ per cui $T(n) \leq 3n$. Analizzando il numero di crediti che abbiamo vedremo:

$$\begin{aligned} \#credits &= \#copia_precedente + \#copia_inserito \\ &= 2 * \#copia_inserito = 2 * (num[T] - 1/2size[T]) = 2num[T] - size[T] \end{aligned}$$

1.4.3 Analisi con il metodo del potenziale

Definiamo adesso la funzione potenziale sfruttando quanto fatto prima. Sappiamo che sono necessari $2num[T] - size[T]$ crediti. Per cui la nostra funzione potenza sarà:

$$\phi(T) = 2num[T] - size[T]$$

Possiamo notare anche qui come $\phi(T_0) = 0$ nel caso in cui la tabella sia ancora vuota e, poiché $1/2size[T] \leq num[T]$, avremo che $\phi(T) \geq 0 = \phi(T_0)$. Sulla base di questo

possiamo analizzare i costi reali degli inserimenti:

Caso senza espansione

$$\begin{aligned} c'_{ins} &= c_{ins} + \phi(T_i) - \phi(T_{i-1}) \\ &= 1 + (2 * n_i - s_i) - (2n_{i-1} - s_{i-1}) = 1 + 2 = 3. \end{aligned}$$

Caso con espansione

$$\begin{aligned} c'_{ins} &= c_{ins} + \phi(T_i) - \phi(T_{i-1}) \\ &= n_i + (2n_i - s_i) - (2n_{i-1} - s_{i-1}) \quad \text{Ricordando che } S_{i-1} = n_i - 1 \\ &= 3 \end{aligned}$$

1.4.4 Tabelle dinamiche con cancellazione

Consideriamo adesso le tabelle dinamiche che possono effettuare anche operazioni di cancellazione di elementi. Immaginando di non voler mai far scendere il fattore di carico sotto di $1/2$, accettando quindi di avere al massimo metà spazio inutilizzato, ad ogni cancellazione che farebbe scendere il fattore di carico sotto $1/2$ avverrebbe un'operazione di contrazione. Immaginiamo lo scenario in cui, dopo aver riempito la tabella, inseriamo un altro elemento per aumentare la dimensione della tabella. Dopodiché cancelliamo due elementi causando una contrazione e poi così in loop. Questo sarebbe naturalmente lo scenario peggiore che causerebbe un grosso problema in termini di complessità.

CONTRAZIONI

	I GRUPPO					II GRUPPO					...		
	$\frac{n}{2}$					$\frac{n}{2}$							
	I	I	...	I		I	C	C	I	I	C	C	...
$mem[T] =$	...	...	...	$\frac{n}{2}$		$\frac{n}{2}+1$	$\frac{n}{2}$	$\frac{n}{2}-1$	$\frac{n}{2}$	$\frac{n}{2}+1$	$\frac{n}{2}$	$\frac{n}{2}-1$	...
$size[T] =$	...	...	...	$\frac{n}{2}$		n	n	$\frac{n}{2}$	$\frac{n}{2}$	n	n	$\frac{n}{2}$	...
$\alpha[T] =$	...	...	...	1		$>\frac{1}{2}$	$=\frac{1}{2}$	<1	1	$>\frac{1}{2}$	$=\frac{1}{2}$	<1	...
$COSTI =$	$\leftarrow \sim \frac{3n}{2} \rightarrow$					$\frac{n}{2}+1$	1	$\frac{n}{2}$	1	$\frac{n}{2}+1$	1	$\frac{n}{2}$	...

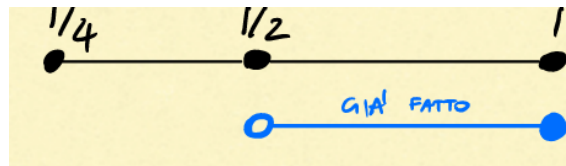
Analizziamo la complessità di questo processo:

$$\begin{aligned}
 & 3 * n/2 \quad \text{Costo dei primi } n/2 \text{ operazioni} \\
 & + 1/4 * n/2 * (n/2 + 1) \quad \text{Inserimenti con espansione II gruppo} \\
 & + 1/4 * n/2 * (n/2) \quad \text{Cancellazione con contrazione II gruppo} \\
 & + 2/4 * n/2 \quad \text{Inserimenti e cancellazioni semplici II gruppo}
 \end{aligned}$$

Sommando tutto abbiamo una complessità di circa n^2 . Il problema di questo processo è il fatto che, quando avviene una contrazione subito dopo un'espansione, non abbiamo il tempo per accumulare crediti. Per cui, ci accontenteremo di avere $\alpha(T) \geq 1/4$. Utilizziamo il metodo del potenziale per fare un'analisi di questi casi. Poniamo:

$$\phi(T) = \begin{cases} 2num[T] - size[T] & \text{se } \alpha[T] > 1/2 \\ 1/2size[T] - num[T] & \text{se } \alpha[T] \leq 1/2 \end{cases}$$

È importante notare che se $\alpha(T) = 1/2$ entrambe valgono zero, rendendo la funzione potenziale continua. Inoltre vale sempre $\phi(T) \geq \phi(T_0) = 0$. Analizziamo i vari casi di inserimento e cancellazione sulla base di questo schema:



Caso $1/4 \leq \alpha(T) < 1/2$

$$\begin{aligned}
 c'_i &= c_i + \phi(T_i) - \phi(T_{i-1}) \\
 &= 1 + (1/2s_i - n_i) - (1/2s_{i-1} - n_{i-1}) \\
 &= 1 + 1/2s_{i-1} - n_{i-1} - 1 - 1/2s_{i-1} + n_{i-1} = 0
 \end{aligned}$$

Caso $\alpha(T) = 1/2$

$$\begin{aligned}
 c'_i &= c_i + \phi(T_i) - \phi(T_{i-1}) \\
 &= 1 + (2n_i - s_i) - (2n_{i-1} - s_{i-1}) \\
 &= 1 + 2(n_{i-1} + 1 - s_{i-1} - 2n_{i-1} + s_{i-1}) = 3
 \end{aligned}$$

Gli stessi casi sono da considerarsi per le cancellazioni. **Caso** $1/2 < \alpha(T) \leq 1$

$$\begin{aligned}
 c'_c &= c_c + \phi(T_i) - \phi(T_{i-1}) \\
 &= 1 + (2n_i - s_i) - (2n_{i-1} - s_{i-1}) \\
 &= 1 + 2n_{i-1} - 2 - s_{i-1} - 2n_{i-1} + s_{i-1} = -1
 \end{aligned}$$

Caso $1/4 \leq \alpha(T) < 1/2$

$$\begin{aligned}
 c'_c &= c_c + \phi(T_i) - \phi(T_{i-1}) \\
 &= 1 + (1/2s_i - n_i) - (1/2s_{i-1} - n_{i-1}) \\
 &= 1 + 1/2s_{i-1} - n_{i-1} + 1 - 1/2s_{i-1} + n_{i-1} = 2
 \end{aligned}$$

Caso $\alpha(T) = 1/4$

$$\begin{aligned} c'_c &= c_c + \phi(T_i) - \phi(T_{i-1}) \\ &= n_{i-1} + (1/2s_i - n_i) - (1/2s_{i-1} - n_{i-1}) \\ &= \frac{s_i - 1}{4} + \frac{s_i - 1}{4} - \frac{s_i - 1}{4} + 1 - \frac{s_i - 1}{2} + \frac{s_i - 1}{4} = 1 \end{aligned}$$

$$\begin{aligned} \hat{c}_{ins} &= \begin{cases} 0 \\ 3 \end{cases} & \text{SE } \frac{1}{4} \leq \alpha(T) < \frac{1}{2} \\ & & \text{SE } \frac{1}{2} \leq \alpha(T) \leq 1 \\ \hat{c}_{canc} &= \begin{cases} -1 \\ 2 \\ 1 \end{cases} & \text{SE } \frac{1}{2} < \alpha(T) \leq 1 \\ & & \text{SE } \frac{1}{4} < \alpha(T) \leq \frac{1}{2} \\ & & \text{SE } \alpha(T) = \frac{1}{4} \end{aligned}$$

In base a ciò sappiamo che $c'_{ins} \leq 3$, $c'_{canc} \leq 2 < 3$ da cui $T(n) \leq \sum_i^n c'_i \leq 3n$

1.5 Scalare i costi

Analizzando alcune strutture dati con il metodo del potenziale, ci potremmo trovare a situazioni in cui i conti potrebbero non tornare per qualche tipo di variabile che spesso può essere aggiustata **scalando i costi**. Vediamone un esempio. Immaginiamo uno stack multipop con i costi:

$$c_{push} = 1 \quad c_{pop} = k \quad c_{multipop} = ks$$

Con k costante. Immaginiamo di analizzare questo con il metodo del potenziale e, visto che stiamo valutando la stessa struttura dati di prima, potrebbe venirci in mente di utilizzare come funzione $\phi(S) = |S|$, per cui avremo che:

$$\begin{aligned} c'_{push} &= c_{push} + \Delta\phi = 2 \\ c'_{pop} &= c_{pop} + \Delta\phi = k - 1 \\ c'_{multipop} &= c_{multipop} + \Delta\phi = ks - s = (k - 1)s = \Theta(s) \end{aligned}$$

Per cui, la complessità non risulterebbe essere quella già vista prima. Proviamo ad aggiustare i conti. Notiamo che il problema nei conti è proprio quel k in ks che non si elimina, per cui, utilizziamo come funzione potenziale $\phi(S) = k|S|$. Proviamo ora:

$$\begin{aligned} c'_{push} &= c_{push} + \Delta\phi = k + 1 \\ c'_{pop} &= c_{pop} + \Delta\phi = 0 \\ c'_{multipop} &= c_{multipop} + \Delta\phi = ks - ks = 0 \end{aligned}$$

Che dimostra di avere complessità $O(n)$.

1.6 Algoritmi Online vs Algoritmi Offline

Definiamo un algoritmo come **offline** se questo ha l'intera sequenza di input già prima che inizi l'elaborazione mentre definiamo un algoritmo **online** se gli input possono arrivare durante l'elaborazione dei dati. Se ci pensiamo, questa differenza risulta fondamentale poiché gli algoritmi offline, sapendo già tutto ciò che devono fare, possono ottimizzare al meglio il loro processo, cosa che invece gli algoritmi online non possono fare. Questo comporta che gli algoritmi online debbano necessariamente cercare di adoperare soluzioni che sono buone in quel momento nella speranza che lo siano anche per il futuro.

Per analizzare un algoritmo online si utilizza quella che viene definita come **analisi competitiva** per cui si confronta un algoritmo online con il corrispondente algoritmo offline ottimo sulla stessa σ . Si definisce **rapporto competitivo** il valore:

$$\sup_{\sigma} = \frac{C_{online}(\sigma)}{C_{OPT}(\sigma)}$$

E questo misura quanto l'algoritmo online sia vicino a quello ottimo. Tanto più è grande questo numero, tanto peggio sarà l'algoritmo online rispetto a quello offline. Definiamo un algoritmo come α -**competitivo** se esiste $b \geq 0$ tale che per ogni sequenza σ abbiamo che:

$$C_{online}(\sigma) \leq \alpha C_{opt}(\sigma) + b$$

Il problema di questa analisi è che ragiona sempre nel caso peggiore e, dunque, può spesso risultare pessimistica. Un esempio di problema che potremmo avere è quello dell'affitto degli sci. Supponiamo di poter decidere se affittare gli sci a 1 € al giorno o comprarli una volta per tutte a 10 €. Naturalmente se sapessimo a prescindere di stare più di 10 giorni, compreremmo direttamente gli sci, tuttavia non è possibile questa cosa nel caso online mentre lo è nel caso offline. Allo stesso modo vale per il problema della gestione delle pagine in memoria per i computer, vediamo come gli algoritmi FIFO e LRU online e deterministici siano k -competitivi rispetto all'algoritmo Békàdy ottimale che però è offline.

1.7 Self-adjusting list

Supponiamo di voler mantenere in memoria una lista doppiamente concatenata di oggetti che possono essere richiesti uno per volta e supponiamo anche di poter riordinare la lista di volta in volta per ottimizzare gli accessi futuri e minimizzare i costi. Per fare ciò, utilizziamo un algoritmo online che sfrutta un'euristica basata sul fatto che l'elemento cercato al tempo t , sarà probabilmente ricercato nuovamente nel tempo $t+x$ per cui, lo spostiamo all'inizio della lista. Questa euristica è detta **Move to front** e, per calcolarne il costo, consideriamo la **posizione dell'elemento ricercato + costo degli scambi di elementi contigui**. Avremo quindi una struttura L con in input una sequenza $\sigma = (x_1, x_2, \dots, x_m)$ di richieste. Ad ogni richiesta si cerca x_i scorrendo la lista da sinistra a destra. Supponiamo di avere a disposizione un algoritmo detto **foresee**, che è un algoritmo offline che "conosce il

FORESEE

Elemento cercato	L	Search cost	Swap cost	Search + Swap cost	Costo Cumulativo
5	(1,2,3,4,5)	5	0	5	5
3	(1,2,3,4,5)	3	3	6	11
4	(4,1,2,3,5)	1	0	1	12
4	(4,1,2,3,5)	1	0	1	13

MOVE-TO-FRONT

Elemento cercato	L	Search cost	Swap cost	Search + Swap cost	Costo Cumulativo
5	(1,2,3,4,5)	5	4	9	9
3	(5,1,2,3,4)	4	3	7	16
4	(3,5,1,2,4)	5	4	9	25
4	(4,3,5,1,2)	1	0	1	26

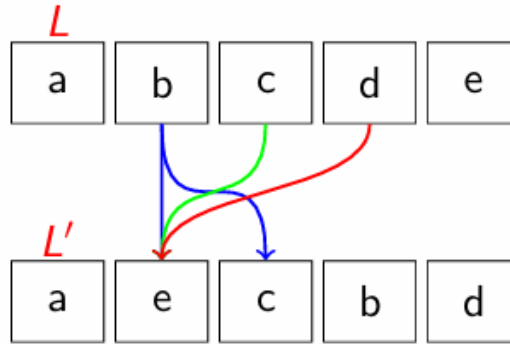


Figura 1.3: In questo esempio le coppie b,c b,e c,e e d,e sono delle inversioni

futuro” e ottimizza tutti i processi, per fungere da benchmark per la nostra analisi.

Dimostreremo che il nostro algoritmo Move-To-Front è 4-competitivo e lo faremo dimostrando che:

$$\sum_i^m c_i^M \leq 4 \sum_i^m c_i^F$$

Dove M sta per l’algoritmo Move-To-Front e F per l’algoritmo Foresee. Siano L e L' due liste contenenti gli stessi elementi, ma in ordine possibilmente diverso, definiamo una **inversione** tra L e L' una coppia non ordinata x,y tale che:

$$x \text{ viene prima di } y \text{ in } L \text{ ma } y \text{ precede } x \text{ in } L'$$

Indicheremo $Dist(L, L')$ il numero di inversioni tra le due liste che rappresenta una misura di distanza tra le due.

Diamo alcune definizioni prima di effettuare un’analisi:

- Sia L una lista di elementi distinti e siano m gli accessi $(x_1, \dots, x_m)$.
- Sia L^M la lista di MTF e sia L^F quella mantenuta da Foresee a partire dalla medesima configurazione L .
- Siano L_i^M, L_i^F le configurazioni delle rispettive liste dopo l’ i -esima operazione.

Avremo inizialmente $L_0^M = L_0^F = L$ e mostreremo quanto detto prima cioè che $\sum_i^m c_i^M \leq 4 \sum_i^m c_i^F$.

- **BB**_{i-1} = { elementi **prima** di x sia in L_{i-1}^M che in L_{i-1}^F };
- **BA**_{i-1} = { elementi **prima** di x in L_{i-1}^M e **dopo** di x in L_{i-1}^F };
- **AB**_{i-1} = { elementi **dopo** di x in L_{i-1}^M ma **prima** di x in L_{i-1}^F }.



Facciamo un esempio completo e supponiamo di voler fare la ricerca dell'elemento x :

In questo caso, analizzando cosa succede nel caso della lista L_{i-1}^M se x passa alla prima posizione, noteremo come cambieranno le cardinalità di alcuni gruppi:

- il gruppo BB cambierà poiché adesso tutti gli elementi che erano prima di x in entrambi saranno adesso dopo di x in L_{i-1}^M ma prima di x rispetto a L_{i-1}^F ;
- il gruppo BA cambia perché gli elementi che erano prima di x in L_{i-1}^M e dopo di x in L_{i-1}^F , adesso saranno dopo x in entrambi;
- Il gruppo AB resterà invece invariato.

Sappiamo anche che:

$$\begin{aligned} r_{L_{i-1}^M}(x) &= |BB| + |BA| + 1 \\ r_{L_{i-1}^F}(x) &= |BB| + |BA| + 1 \\ \text{Dist}(L_i^M, L_{i-1}^F) - \text{Dist}(L_{i-1}^M, L_{i-1}^F) &= |BB| - |BA| \end{aligned}$$

⁵ Inoltre:

$$\begin{aligned} c_i^M &= 2r_{L_{i-1}^M}(x) - 1 \quad \text{la posizione dell'elemento più il numero di scambi} \\ c_i^F &\geq r_{L_{i-1}^F}(x) + s_i^F \quad s_i^F \text{ rappresenta il numero di scambi fatti da Foresee} \\ \text{Dist}(L_i^M, L_i^F) - \text{Dist}(L_{i-1}^M, L_{i-1}^F) &\leq s_i^F \\ \text{Dist}(L_i^M, L_i^F) - \text{Dist}(L_{i-1}^M, L_{i-1}^F) &= \\ &= ((\text{Dist}(L_i^M, L_i^F) - \text{Dist}(L_i^M, L_{i-1}^F)) + ((\text{Dist}(L_i^M, L_{i-1}^F) - \text{Dist}(L_{i-1}^M, L_{i-1}^F))) \\ &\leq |BB| - |BA| + s_i^F \end{aligned}$$

Sulla base di queste considerazioni, proviamo ad utilizzare la funzione potenziale $\phi(\text{Dist}(L_i^M, L_i^F))$.

In questi conti sono stati segnati in rosso ciò che deriva dal normale conto, in blu ciò che deriva dal potenziale da noi scelto e, in verde, ciò che è stato aggiunto per far quadrare i conti. Come vediamo non arriviamo al giusto risultato. Proviamo adesso a scalare il costo del potenziale e vediamo come effettivamente riusciamo nella dimostrazione.

⁵ $r_{L_{i-1}}(x)$ rappresenta il rango/la posizione dell'elemento x

$$\begin{aligned}
\hat{c}_i^M &= c_i^M + \Psi_i - \Psi_{i-1} \\
&= 2r_{L_{i-1}^M}(x) - 1 + \mathcal{I}(L_i^M, L_i^F) - \mathcal{I}(L_{i-1}^M, L_{i-1}^F) \\
&\leq 2r_{L_{i-1}^M}(x) - 1 + |BB| - |BA| + s_i^F \\
&= 2r_{L_{i-1}^M}(x) - 1 + |BB| - (r_{L_{i-1}^M}(x) - |BB| - 1) + s_i^F \\
&= (2r_{L_{i-1}^M}(x) - r_{L_{i-1}^M}(x)) + 2|BB| + 1 + s_i^F - 1 \\
&\leq (2r_{L_{i-1}^M}(x) - r_{L_{i-1}^M}(x)) + 2|BB| + 1 + s_i^F - 1 + 2|AB| + 1 + s_i^F \\
&= (2r_{L_{i-1}^M}(x) - r_{L_{i-1}^M}(x)) + [2(|BB| + |AB|) + 1 + 1 + s_i^F + s_i^F] - 1 \\
&= (2r_{L_{i-1}^M}(x) - r_{L_{i-1}^M}(x)) + 2(r_{L_{i-1}^F}(x) + s_i^F) - 1 \\
&\leq (2r_{L_{i-1}^M}(x) - r_{L_{i-1}^M}(x)) + 2c_i^F \quad ???
\end{aligned}$$

Scaliamo adesso il potenziale con un fattore 2: $\Phi_i = 2 \cdot \mathcal{I}(L_i^M, L_i^F)$.

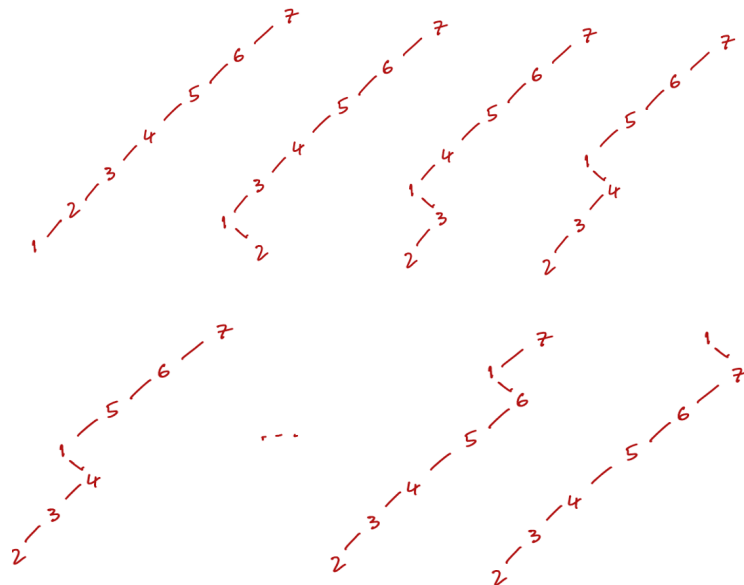
$$\begin{aligned}
\hat{c}_i^M &= c_i^M + \Phi_i - \Phi_{i-1} \\
&= 2r_{L_{i-1}^M}(x) - 1 + 2(\mathcal{I}(L_i^M, L_i^F) - \mathcal{I}(L_{i-1}^M, L_{i-1}^F)) \\
&\leq 2r_{L_{i-1}^M}(x) - 1 + 2(|BB| - |BA| + s_i^F) \\
&= 2r_{L_{i-1}^M}(x) - 1 + 2|BB| - 2(r_{L_{i-1}^M}(x) - |BB| - 1) + 2s_i^F \\
&= 4|BB| + 1 + 2s_i^F \\
&\leq 4|BB| + 4|AB| + 4 + 4s_i^F \\
&= 4(|BB| + |AB| + 1 + s_i^F) \\
&= 4(r_{L_{i-1}^F}(x) + s_i^F) \\
&\leq 4c_i^F.
\end{aligned}$$

Capitolo 2

Splay Trees

Gli splay trees sono delle strutture **auto bilancianti**, questi implementano M operazioni consecutive di **ricerca, inserimento e cancellazione** in un tempo $O(M \log N)$ dove N rappresenta il numero di inserimenti in un albero inizialmente vuoto. Alla base di questa struttura dati, vi è l'idea che i nodi che sono stati ricercati una volta, saranno ricercati ancora in un breve tempo. Di conseguenza, si cerca di avvicinare i nodi che si incontrano alla radice per velocizzare questo accesso. Per fare ciò, il modo più semplice è quello di continuare a fare rotazioni da basso verso l'alto lungo tutto il cammino. Tuttavia, questo metodo necessita di qualche piccola modifica per funzionare correttamente, infatti, possiamo analizzare la complessità senza modifiche e vedere cosa succede.

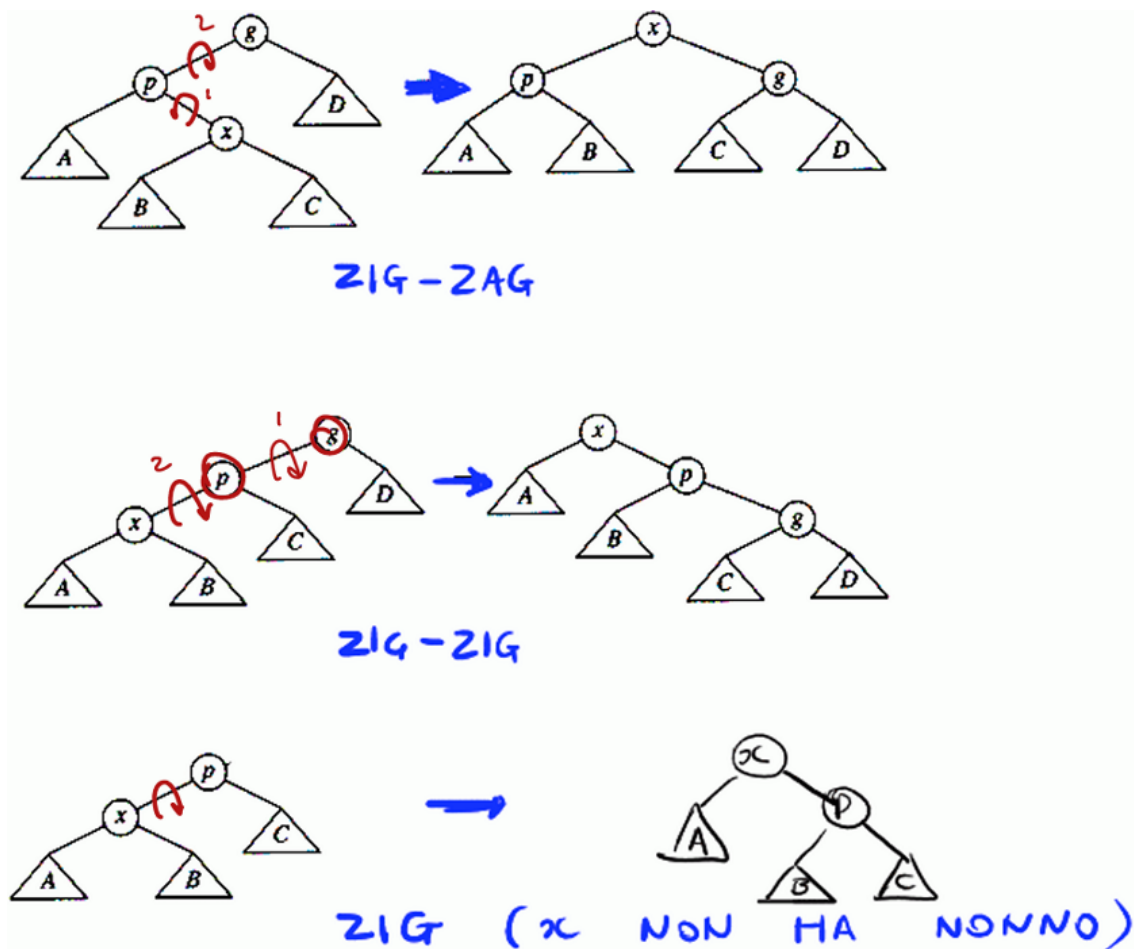
Immaginiamo di avere N inserimenti e di contare il costo di un'operazione in base al numero di rotazioni che è necessario fare per portare un nodo alla radice. Prendiamo in considerazione adesso la sequenza di operazioni $\{ \text{insert}(1), \text{insert}(2), \dots, \text{insert}(n), \text{search}(1), \text{search}(2), \dots, \text{search}(n) \}$. Per quando riguarda gli inserimenti, avremo che la complessità totale sarà $\theta(N)$ poiché, ad ogni inserimento faremo almeno una rotazione. Le cose si complicano quando cominciamo ad effettuare le ricerche:



Come possiamo vedere, la ricerca di 1 ha un costo di $N-1$ rotazioni e sarà lo stesso per la ricerca di 2. Vedremo però, che passando alla ricerca di tre il numero di rotazioni sarà $N-2$, poi per 4 sarà $N-3$ ecc. In totale, avremo la complessità per la seconda parte delle operazioni di $\sum_i^n i - 1$, da cui avremo all'incirca una complessità totale di $\theta(N^2)$.

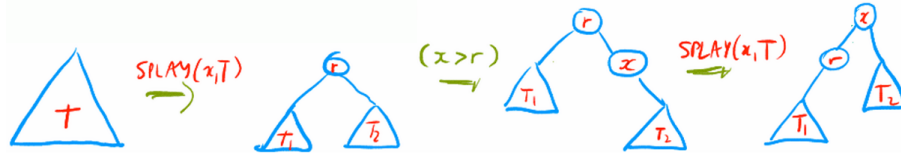
2.1 Zig, Zig-Zag, Zig-Zig

Per risolvere questo problema, introduciamo alcune casistiche utili per l'analisi della complessità. Queste sono **Zig**, **Zig-Zag** e **Zig-Zig**, che possiamo tranquillamente vedere in figura.



In pratica, l'unica cosa che cambia è che nel caso simmetrico, anziché partire dal basso verso l'alto, la prima rotazione sarà quella in alto e poi faremo quella in basso. Questo cambiamento, seppure molto piccolo, cambia tutto e fa sì che l'albero riesca a bilanciarsi nella maggior parte dei casi. Introduciamo quindi le operazioni che possono essere eseguite:

- **Splay(x, T):** Si cerca sull'albero T l'elemento x mediante ricerca binaria. Dopodiché, a partire dal nodo ove la ricerca si è fermata, si applicano le operazioni di zig, zig-zag e zig-zig necessarie.
- **Insert(x, T):** Si esegue una Splay(x, T) e, una volta che l'elemento non viene trovato, si inserisce x e si esegue Splay a partire dal nodo appena inserito.

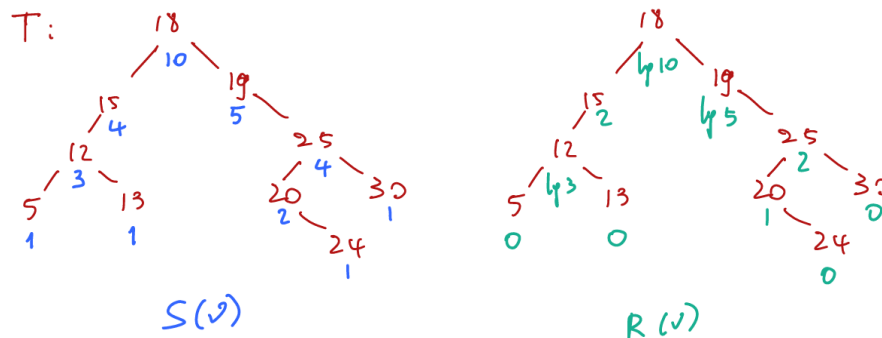


- **Delete(x, T)** Si effettua una Splay(x, T) e si cancella il nodo¹ ottenendo quindi due alberi T_1, T_2 . Si effettua nuovamente un splay(x, T) e, sia r la nuova radice, si ponga la radice di T_2 come figlio destro del nodo r.

2.2 Analisi ammortizzata di Splay

Consideriamo i costi delle operazioni elementari che per zig è 1 mentre per zig-zag e zig-zig è 2. Poniamo

- $S(\nu) = \#$ nodi nel sotto albero con radice ν ;
- $R(\nu) = \log(S(\nu))$ che chiameremo **rango di ν** ;
- $\phi(T) = \sum_{\nu \in T} R(\nu)$;



Sia T_0 un albero vuoto, avremo che $\phi(T_0) = 0$ e, sia T un albero qualsiasi, avremo che:

$$\phi(T) = \sum R(\nu) = \sum \log(S(\nu)) \geq \phi(T_0) = 0$$

Dunque sappiamo che possiamo utilizzare il metodo del potenziale per avere un upper bound al costo di M operazioni di cui N sono inserimenti su un albero inizialmente vuoto.

¹Si ricordi che il nodo sarà la root dopo splay.

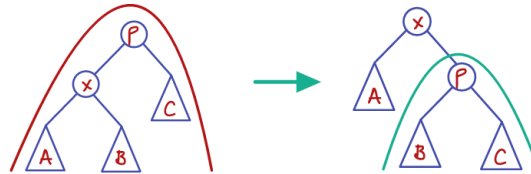
LEMMA Siano $a, b > 0$ $c \geq a+b$ allora $\log(a) + \log(b) \leq 2\log(c) - 2$. Possiamo dimostrarlo:

$$\begin{aligned}
 (a+b)^2 &= a^2 + 2ab + b^2 \\
 &= a^2 - 2ab + b^2 + 4ab \\
 &= (a-b)^2 + 4ab \\
 &\geq 4ab \\
 a+b &\geq 2\sqrt{ab} \\
 c &\geq a+b \geq 2\sqrt{ab} \\
 \log c &\geq 1 + \frac{1}{2}(\log a + \log b) \\
 2\log c - 2 &\geq \log a + \log b
 \end{aligned}$$

Vediamo di dimostrare che il costo ammortizzato dell'operazione $\text{Splay}(x, T)$ è al più $3(R(\text{root}(T)) - R(x)) + 1$. Calcoliamo il costo ammortizzato di ogni operazione zig, zig-zag, zig-zig. Indicheremo con $S_i(x)$, $S_f(x)$ il numero di nodi nel sotto-albero di radice x prima e dopo l'operazione. Allo stesso modo considereremo $R_i(x)$, $R_f(x)$.

Caso Zig

Il costo reale di questa operazione è 1 poiché avviene solo una rotazione. Sappiamo che $\Delta\phi = R_f(x, p) - R_i(x, p)$ e osserviamo che:



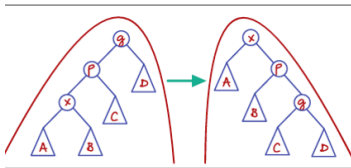
$$S_f(p) \leq S_i(p) \rightarrow R_f(p) \leq R_i(p) \rightarrow \Delta\phi \leq R_f(p) - R_i(x)^2$$

Osserviamo anche che, allo stesso modo, $S_i(x) \leq S_f(x) \rightarrow R_i(x) \leq R_f(x) \rightarrow R_f(x) - R_i(x) \geq 0 \rightarrow \Delta\phi \leq 3(R_f(x) - R_i(x))$. Per cui, $c'_{zig} = 1 + \Delta\phi \leq 1 + 3(R_f(x) - R_i(x))$

²Visto che $\Delta\phi = (R_f(x) - R_f(p)) + (R_f(p) - R_i(p))$ e che abbiamo visto che il rango iniziale è maggiore di quello finale, possiamo concludere che $R_f(p) - R_i(p) = 0$.

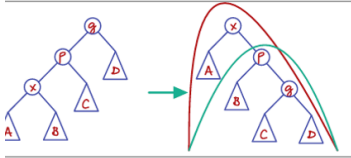
Caso Zig-Zig

Come abbiamo già detto, questa operazione ha costo reale 2. Qui i nodi coinvolti sono 3 per cui $\Delta\phi = R_f(x, p, g) - R_i(x, p, g)$. Analizziamo:



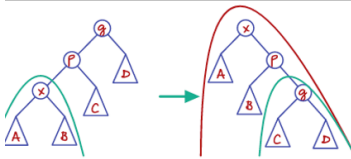
$$1^o \quad S_f(x) = S_i(g) \longrightarrow R_f(x) = R_i(g)$$

$$\longrightarrow \Delta\phi = R_f(p, g) - R_i(x, p)$$



$$2^o \quad S_f(p) \leq S_f(x) \longrightarrow R_f(p) \leq R_f(x)$$

$$\longrightarrow \Delta\phi \leq R_f(x, g) - R_i(x, p)$$



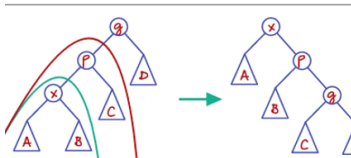
$$3^o \quad S_i(x) + S_p(g) \leq S_f(x) \xrightarrow{\text{DAL LEMMA}} R_i(x) + R_p(g) \leq 2R_f(x) - 2$$

$$\longrightarrow \Delta\phi \leq R_f(x, g) - R_i(x, p) + R_i(x) - R_i(x) =$$

$$= R_f(x) + R_f(g) - R_i(x) - R_i(p) + R_i(x) - R_i(x)$$

$$\leq R_f(x) + 2R_f(x) - 2 - 2R_i(x) - R_i(p) =$$

$$= 3R_f(x) - 2 - 2R_i(x) - R_i(p)$$



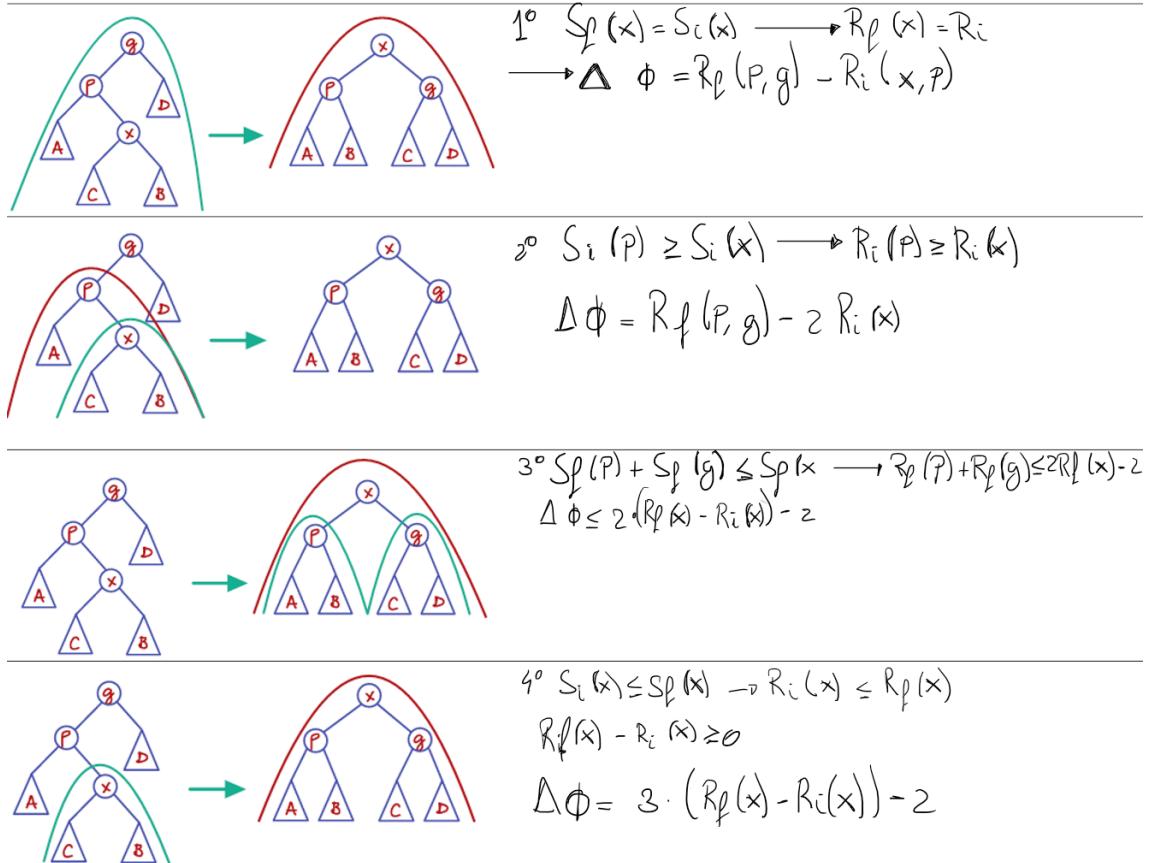
$$4^o \quad S_i(x) \leq S_i(p) \longrightarrow R_i(x) \leq R_i(p)$$

$$\Delta\phi \leq 3(R_f(x) - R_i(x)) - 2$$

Per cui, alla fine di questa analisi possiamo concludere che $c'_{zig-zig} = 2 + \Delta\phi \leq 3(R_f(x) - R_i(x))$

Caso Zig-Zag

Anche in questo caso, l'operazione reale ha costo 2 e sono coinvolti 3 nodi per cui $\Delta\phi = R_f(x, p, g) - R_i(x, p, g)$. Analizziamo:



Da cui segue che, alla fine di questa analisi possiamo concludere che.

$$c'_{zig-zag} = 2 + \Delta \phi \leq 3(R_f(x) - R_i(x))$$

Pertanto, posto $\text{splay}(x) = Z_1, Z_2, \dots, Z_k$ con $Z_j \in \text{zig}, \text{zig-zag}, \text{zig-zig}$ avremo che:

- c'_j = costo ammortizzato dell'operazione Z_j ;
- $R_i^j(x)$ = rango di x poco prima di Z_j ;
- $R_f^j(x)$ = rango di x poco dopo Z_j .

Cerchiamo di calcolare c'_{splay} :

$$\begin{aligned}
c'_{splay} &= \sum_j^k c'_j \leq \sum_j^k 3(R_f^j(x) - R_i^j(x)) + 1 \\
&= \sum_j^k 3(R_f^j(x) - R_f^{j-1}(x)) + 1 \\
&= 3(R_f^k(x) - R_i^0(x)) + 1 = 3(R_f^k(x) - R_i^1(x)) + 1 \\
&= 3(R_i^1(\text{root}(T)) - R_i^1(x)) + 1 \quad \text{Root}(T) = R_f^k \text{ poiché } x \text{ va poi sulla root} \\
&= 3(R(\text{root}(T)) - R(x)) + 1 \quad \text{E, quindi...}
\end{aligned}$$

$$c'_{splay} \leq 3(R(\text{root}(T)) - R(x)) + 1$$

Una sequenza di M operazioni di search, insert e delete sono $M' = O(M)$ operazioni di splay, ciascuna di costo ammortizzato $c'_{splay} \leq 3(R(\text{root}(T_j)) - R(x_j)) + 1 \leq 3R(\text{root}(T_j)) + 1$. Posto che N' sia il massimo numero di nodi nell'albero, avremo che $c'_{splay} \leq 3R(\text{root}(T_j)) + 1 \leq 3\log(N') + 1$. Pertanto:

$$\sum_j^M c_j = \sum_j^{M'} c_{splay_j} \leq \sum_j^{M'} c'_{splay_j} \leq M'(3\log(N') + 1) = O(M\log(N'))$$

Partendo da un albero inizialmente vuoto ed eseguendo N operazioni di inserimento tra le M si ha che una sequenza di M operazioni ha costo $O(M\log N)$ cioè ogni operazione ha tempo ammortizzato logaritmico.

2.3 Top-Down Splay Trees

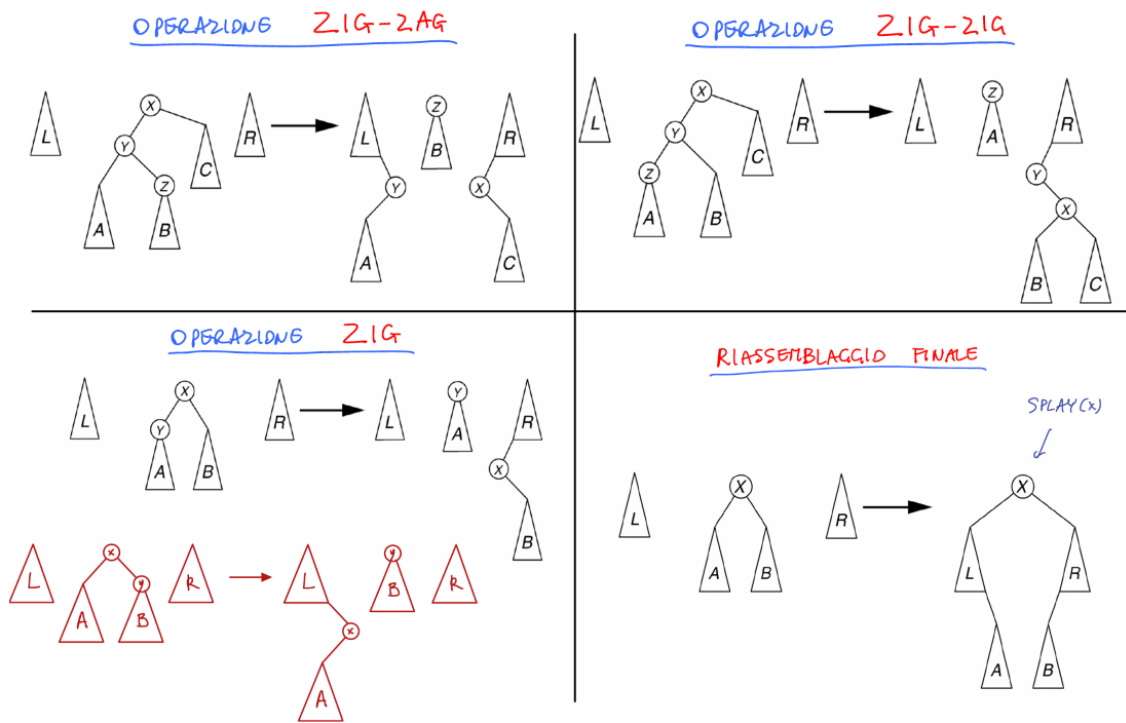
Il tipo di Splay Trees che abbiamo illustrato prima, si chiamano **bottom up splay trees**. Questi sono molto facili da implementare, tuttavia hanno alcuni problemi implementativi dato che le rotazioni danno luogo ad un bottom up e quindi occorre memorizzare il cammino dalla radice al nodo oppure mantenere per ciascun elemento il nodo padre.

La variante che risolve questo problema si chiama Top-Down e, mantenendo il costo logaritmico, risolve questo problema. L'idea è che mentre si procede alla ricerca, si trattano tutti i nodi incontrati. Ogni operazione di splay dividerà l'albero in tre parti:

1. L(left): l'albero di sinistra che ha tutti i nodi più piccoli rispetto ad x ;
2. T' : l'albero centrale su cui si continua la ricerca;
3. R(right): l'albero di destra per cui tutti i nodi sono più grandi rispetto ad x ;

Inizialmente x è la radice dell'albero T e L e R sono vuoti. Una volta scesi di due o uno, applichiamo l'operazione di zig-zag, zig-zig o zig e aggiorniamo i tre alberi L ,

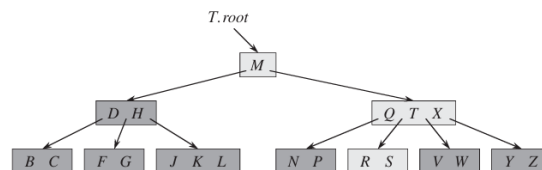
T' e R mantenendo la proprietà $L \leq T' \leq R$. Alla fine gli alberi saranno riordinati come segue:



Capitolo 3

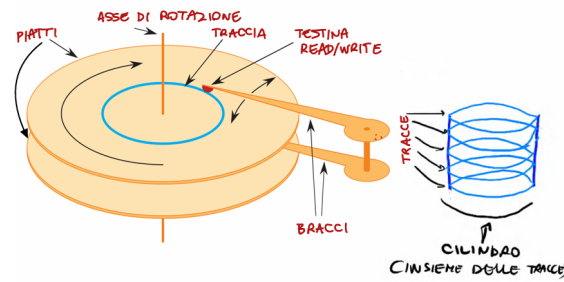
B-Trees

I B-tree sono alberi bilanciati di ricerca particolarmente utili poiché progettati per essere memorizzati sui dischi. Poiché questi dischi sono molto più lenti della memoria ad accesso casuale (RAM), le prestazioni dei B-alberi vengono misurate non soltanto in base al tempo computazionale richiesto per eseguire le operazioni sugli insiemi dinamici, ma anche in base al **numero di accessi al disco**. Poiché il numero di accessi al disco aumenta con l'altezza dell'albero, ciò che si fa' è avere un'altezza quanto più bassa possibile grazie ad un elevato numero di chiavi per nodo. Per cui, se gli alberi rosso neri hanno al più $O(\log_2 n)$ altezza, i b-tree avranno $O(\log_t n)$ altezza.



3.1 Ottimizzazione su Hard Disk

Come accennato, questa struttura dati è parecchio utile perché ottimizza gli accessi all'hard disk. Un'hard disk è composto da più piatti uno sopra l'altro che girano attorno ad una asse comune e, questi, vengono letti mediante un'apposita testina che viene mossa da dei bracci meccanici. Per ammortizzare gli elevati tempi di posizionamento delle testine nelle tracce, le informazioni sono organizzate sui cilindri mediante blocchi della stessa dimensioni dette **pagine**. Risulterà quindi conveniente organizzare le strutture dati in maniera tale che queste sfruttino la suddivisione in pagine così che gli accessi in lettura e scrittura avvengano solo su pagine intere. In tal senso, imposteremo i b-tree in maniera tale che un nodo abbia esattamente la dimensione di una pagina, così facendo, andremo a leggere al massimo $\log_t n$ nodi che corrispondono all'altezza dell'albero.



3.2 Definizione e altezza di un B-tree

Un B-Tree T è un albero con radice in $root[T]$ che gode delle seguenti proprietà:

1. Ciascun nodo x ha i seguenti campi:
 - $n[x]$ numero delle chiavi in x ordinate in **ordine crescente** cioè $key_1[x] \leq key_2[x] \leq \dots \leq key_{n[x]}[x]$.
 - $leaf[x]$ campo booleano tale che è vero se x è una foglia e falso se è un nodo intero.
2. Ciascun nodo interno x contiene $n[x] + 1$ puntatori ai figli indicati come $c_1[x], c_2[x], \dots, c_{n[x]+1}[x]$.
3. Le chiavi $x.key_i$ separano gli intervalli delle chiavi memorizzate in ciascun sotto albero: Se k_i denota una chiave nel sotto albero di radice $c_i[x]$ per $i = 1, \dots, n[x] + 1$, vale che $k_i \leq key_1[x] \leq k_2 \leq key_2[x] \leq \dots \leq key_{n[x]}[x] \leq k_{n[x]+1}$. Proprio perché vale questa proprietà i b-tree sono alberi di ricerca.
4. Tutte le foglie hanno la stessa profondità che è l'altezza dell'albero.
5. Ci sono limiti superiori e inferiori al numero di chiavi che si possono avere in un nodo. Questi possono essere espressi in base ad un intero $t \geq 2$ che è chiamato **grado minimo**.
 - Ogni nodo, deve avere almeno $t - 1$ chiavi. Ogni nodo interno, tranne la radice, quindi ha almeno t figli.
 - Ogni nodo può contenere al massimo $2t - 1$ chiavi. Quindi, un nodo interno può avere al massimo $2t$ figli. Diciamo che un nodo è pieno se contiene esattamente $2t - 1$ chiavi.

Teorema Altezza di un B-Albero

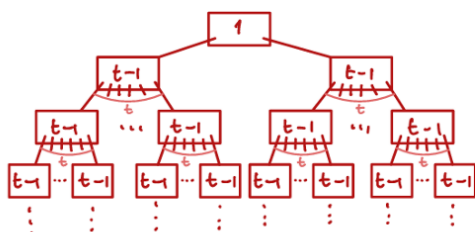
Se $n \geq 1$, allora per qualsiasi B-Albero T di n chiavi con altezza h e grado minimo $t \geq 2$, si ha che:

$$\log_{2t} \frac{n+1}{2t} \leq h \leq \log_t \frac{n+1}{2}$$

Dimostriamolo:

Siano n_h il minimo numero di chiavi e sia N_h il massimo numero di chiavi.

$$m_h \leq m \leq N_h$$

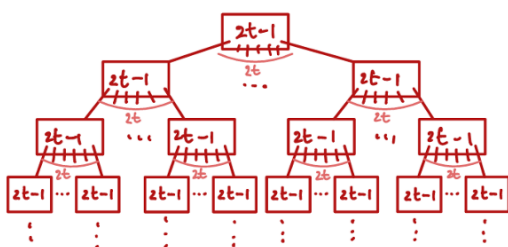


livello	# nodi	# charri
0	1	1
1	2	$2(t-1)$
2	$2t$	$2t(t-1)$
3	$2t^2$	$2t^2(t-1)$
$\vdots$		
i	$2t^{i-1}$	$2t^{i-1}(t-1)$
$\vdots$		
h	$2t^{h-1}$	$2t^{h-1}(t-1)$

$$m_h = 1 + 2(t-1) \cdot \sum_{i=1}^h t^{i-1}$$

$$= 1 + 2(t-1) \cdot \sum_{j=0}^{h-1} t^j = 1 + 2(t-1) \frac{t^h - 1}{t - 1} = 1 + 2(t^h - 1) = 2t^h - 1$$

$$m_h \leq m \rightarrow 2t^h - 1 \leq m \rightarrow t^h \leq \frac{m+1}{2} \rightarrow h \leq \log_t \frac{m+1}{2}$$



livello	# nodi	# charri
0	1	1
1	$2t$	$2t(t-1)$
2	$(2t)^2$	$(2t)^2(t-1)$
3	$(2t)^3$	$(2t)^3(t-1)$
$\vdots$		
i	$(2t)^i$	$(2t)^i(t-1)$
$\vdots$		
h	$(2t)^h$	$(2t)^h(t-1)$

$$N_h = (2t-1) \sum_{i=0}^h (2t)^i = (2t-1) \cdot \frac{(2t)^{h+1} - 1}{2t - 1}$$

$$= (2t)^{h+1} - 1$$

$$m \leq N_h \rightarrow m \leq (2t)^{h+1} - 1 \rightarrow m+1 \leq (2t)^{h+1} \rightarrow \log_{2t}(m+1) \leq h+1 \rightarrow$$

$$\rightarrow \log_{2t}(m+1) - 1 \leq h \rightarrow \log_{2t} \frac{m+1}{2t} \leq h$$

Da questi calcoli segue che $\log_{2t} \frac{n+1}{2} \leq h \leq \log_t \frac{n+1}{2}$ per cui si dimostra che $h = \Theta(\log(n))$.

3.3 Operazioni di base sui B-Tree

Per quanto riguarda le operazioni sui B-Tree, descriveremo:

- B-Tree-Search
- B-Tree-Create
- B-Tree-Insert
- B-Tree-Delete

Supporremo che la radice sia sempre in memoria e che per tutti i nodi passati come parametro deve essere chiamata la procedura read.

3.4 B-Tree-Search(x,k)

In questa procedura quello che sarà fatto sarà analizzare gli intervalli in un nodo e, nel caso il numero cercato fosse in quell'intervallo, si scende ancora con il nodo fino a quando non si trova ciò che si cerca.

Algorithm 1 B-TREE-SEARCH(x, k)

```

1:  $i \leftarrow 1$ 
2: while  $i \leq n[x]$  and  $k > key_i[x]$  do  $\triangleright$  Fino a quando la chiave corrente è minore
3:    $i \leftarrow i + 1$ 
4: end while
5: if  $i \leq n[x]$  and  $k = key_i[x]$  then  $\triangleright$  Se non abbiamo analizzato tutte le chiavi e
   abbiamo trovato k
6:   return  $(x, i)$   $\triangleright$  Allora abbiamo trovato la chiave cercata
7: end if
8: if  $leaf[x]$  then  $\triangleright$  Altrimenti, se il nodo è una foglia...
9:   return NIL  $\triangleright$  ...la chiave non è presente
10: else
11:   DISK-READ( $c_i[x]$ )  $\triangleright$  Altrimenti, leggiamo da disco il figlio
12:   return B-TREE-SEARCH( $c_i[x], k$ )  $\triangleright$  E cerchiamo ricorsivamente
13: end if
```

La complessità sarà $O(h) = O(\log_t n)$ per quanto riguarda gli accessi al disco e $O(th) = O(t \log_t n)$ per quanto riguarda il tempo di cpu.

3.5 B-Tree-Create

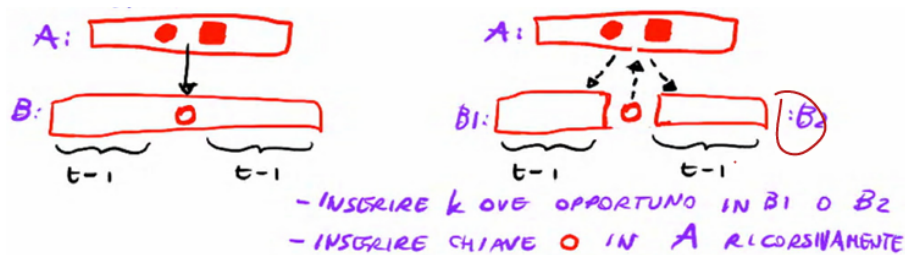
Questa operazione consiste nella creazione di un albero vuoto per cui, semplicemente, andremo a creare la radice.

Algorithm 2 B-TREE-INITIALIZE()

- | | |
|--|--|
| 1: $x \leftarrow \text{Allocate-Node}()$ | ▷ Allocazione di un nuovo nodo |
| 2: $\text{leaf}[x] \leftarrow \text{true}$ | ▷ Il nodo appena creato è una foglia |
| 3: $n[x] \leftarrow 0$ | ▷ Numero di chiavi nel nodo inizialmente 0 |
| 4: $\text{Disk-Write}(x)$ | ▷ Scriviamo il nodo su disco |
| 5: $\text{root}[T] \leftarrow x$ | ▷ Impostiamo la radice dell'albero |
-

3.6 Inserimento in un B-Tree

Si chiama innanzitutto la B-Tree-Search per localizzare la foglia dove fare l'inserimento e, se questa non è piena, allora si inserisce k nella posizione opportuna. Altrimenti, si divide la foglia in due, il mediano va' nel nodo sopra e la chiave si inserisce dov'è di giusto.



Inoltre, per far sì che il tutto non risalga alla radice, si faranno le suddivisione mentre si scende per localizzare la foglia in cui inserire la chiave e si utilizzerà un'ulteriore procedura chiamata B-Tree-Split-Child(x, i) con x nodo non pieno e i nodo figlio di x .

Algorithm 3 B-TREE-SPLIT-CHILD(x, i)

```

1:  $z \leftarrow \text{ALLOCATE-NODE}()$  ▷ Si crea un nuovo nodo
2:  $y \leftarrow x.c_i$  ▷ Si identifica il nodo da dividere
3:  $z.\text{leaf} \leftarrow y.\text{leaf}$  ▷ Il nuovo nodo prende lo stesso status di foglia di y
4:  $z.n \leftarrow t - 1$  ▷ Il nuovo nodo avrà t-1 chiavi perché dividiamo un nodo pieno
   che ha al più  $2t-1$ 
5: for  $j \leftarrow 1$  to  $t - 1$  do ▷ Le chiavi di y vengono divise dove c'è il mediano
6:    $z.\text{key}_j \leftarrow y.\text{key}_{j+t}$ 
7: end for
8: if not  $y.\text{leaf}$  then ▷ Anche i puntatori ai sotto alberi si trasferiscono
9:   for  $j \leftarrow 1$  to  $t$  do
10:     $z.c_j \leftarrow y.c_{j+t}$ 
11:   end for
12: end if
13:  $y.n \leftarrow t - 1$  ▷ Viene aggiornata anche la dimensione di y
14: for  $j \leftarrow x.n + 1$  downto  $i + 1$  do
15:    $x.c_{j+1} \leftarrow x.c_j$ 
16: end for
17:  $x.c_{i+1} \leftarrow z$  ▷ Il nodo genitore viene aggiornato per accogliere z
18: for  $j \leftarrow x.n$  downto  $i$  do ▷ Si shiftano per accogliere il mediano
19:    $x.\text{key}_{j+1} \leftarrow x.\text{key}_j$ 
20: end for
21:  $x.\text{key}_i \leftarrow y.\text{key}_t$  ▷ Si sposta il mediano
22:  $x.n \leftarrow x.n + 1$ 
23:  $\text{DISK-WRITE}(y)$  ▷ Si salva il tutto
24:  $\text{DISK-WRITE}(z)$ 
25:  $\text{DISK-WRITE}(x)$ 

```

Questa procedura ha costo $O(1)$ per quanto riguarda gli accessi al disco e $\Theta(t)$ per quanto riguarda il tempo di CPU.

Analizzata questa procedura passiamo all'inserimento:

Algorithm 4 B-TREE-INSERT(T, k)

```

1:  $r \leftarrow T.root$ 
2: if  $r.n == 2t - 1$  then
3:    $s \leftarrow \text{ALLOCATE-NODE}()$        $\triangleright$  Se la radice è piena, creiamo un nuovo nodo
4:    $T.root \leftarrow s$                    $\triangleright$  Che diventerà la nuova radice
5:    $s.leaf \leftarrow \text{false}$ 
6:    $s.n \leftarrow 0$ 
7:    $s.c_1 \leftarrow r$                    $\triangleright$  La vecchia radice diventa il nuovo figlio
8:   B-TREE-SPLIT-CHILD( $s, 1$ )           $\triangleright$  E il nuovo figlio viene diviso in due
9:   B-TREE-INSERT-NONFULL( $s, k$ )         $\triangleright$  Procedura a seguito
10: else
11:   B-TREE-INSERT-NONFULL( $r, k$ )         $\triangleright$  Se il nodo non è pieno facciamo
      l'inserimento
12: end if

```

Come possiamo notare, vi è un'altra procedura che è B-Tree-Insert-Nonfull che effettua l'inserimento effettivo sul nodo non vuoto. In pratica questa procedura fa' sì che si scenda sempre su un nodo non pieno, così facendo non abbiamo problemi nell'inserimento.

Algorithm 5 B-TREE-INSERT-NONFULL(x, k)

```

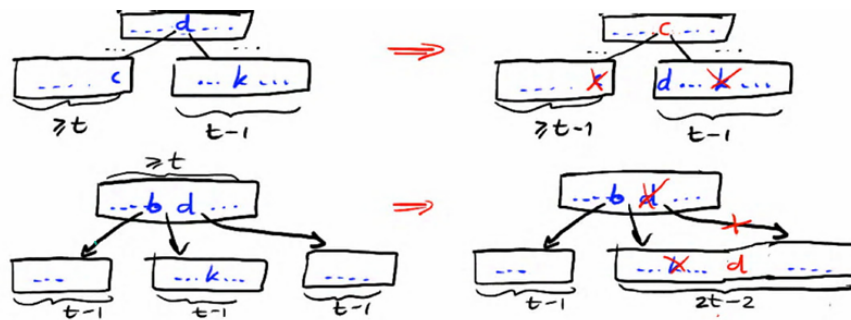
1:  $i \leftarrow x.n$ 
2: if  $x.leaf$  then                   $\triangleright$  Se è una foglia si inserisce subito
3:   while  $i \geq 1$  and  $k < x.key_i$  do
4:      $x.key_{i+1} \leftarrow x.key_i$      $\triangleright$  Le chiavi vengono shiftate per l'inserimento
5:      $i \leftarrow i - 1$ 
6:   end while
7:    $x.key_{i+1} \leftarrow k$                $\triangleright$  Si fa l'inserimento
8:    $x.n \leftarrow x.n + 1$ 
9:   DISK-WRITE( $x$ )
10: else
11:   while  $i \geq 1$  and  $k < x.key_i$  do  $\triangleright$  Viene determinato il figlio a cui scendere
12:      $i \leftarrow i - 1$ 
13:   end while
14:    $i \leftarrow i + 1$ 
15:   DISK-READ( $x.c_i$ )
16:   if  $x.c_i.n == 2t - 1$  then         $\triangleright$  Se è pieno si effettua una split
17:     B-TREE-SPLIT-CHILD( $x, i$ )
18:     if  $k > x.key_i$  then
19:        $i \leftarrow i + 1$ 
20:     end if
21:   end if
22:   B-TREE-INSERT-NONFULL( $x.c_i, k$ )     $\triangleright$  Si rieffettua la chiamata per fare
      l'inserimento in un nodo non pieno
23: end if

```

Per quanto riguarda la complessità, avremo che la NonFull sarà $\Theta(h)$ per gli accessi al disco e $\Theta(th)$ per il tempo di CPU, per l'inserimento invece, avremo le stesse complessità.

3.7 Cancellazione da un B-Tree

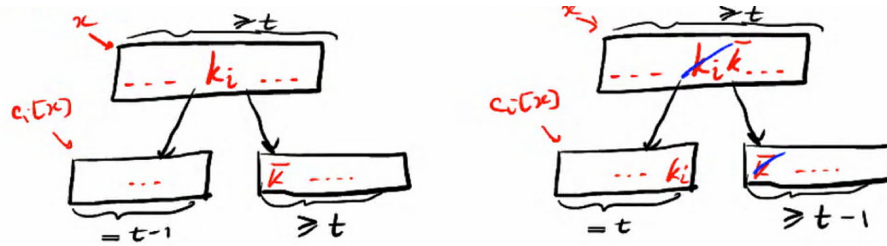
Per effettuare la cancellazione da un B-Tree innanzitutto utilizzeremo, anche qui, la procedura B-Tree-Search che ci indicherà il nodo dove si trova la chiave k da eliminare. Se tale nodo è una foglia con almeno t chiavi, allora si cancella la chiave, altrimenti, il nodo sarà definito **magro**, per cui avrà al massimo $t-1$ chiavi al suo interno e si dovrà procedere diversamente. Si andrà a guardare da una foglia sorella o dal nodo padre per verificare la possibilità di prestito di una chiave o di, eventualmente unione con un nodo sorella.



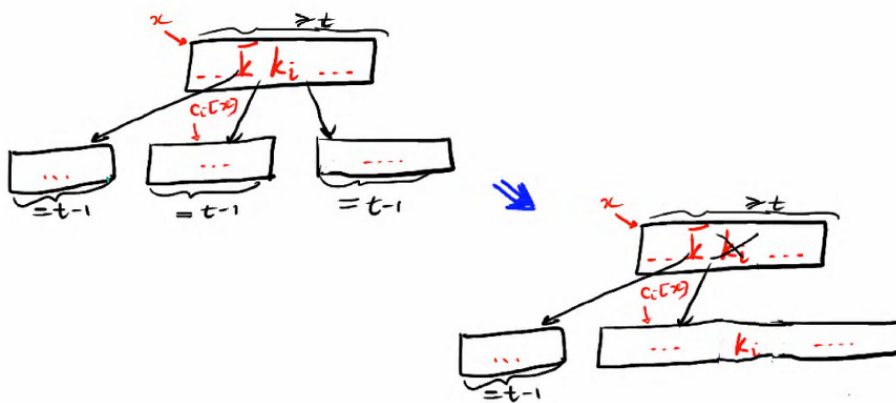
Se il nodo contenente la chiave k è, invece, un nodo interno si sostituisce k con la chiave immediatamente più piccola e si cancelli il predecessore di k che risiederà su una foglia. Per evitare, anche in questo caso, di propagare il tutto fino alla radice, si farà in modo che i nodi del cammino dalla radice a k non siano magri. Pertanto:

1. Se k è nel nodo x ed x è una foglia si cancella semplicemente k da x .
2. Se k è nel nodo x ed x è interno:
 - Se il figlio y di x che precede k ha almeno t chiavi, si determini il predecessore k' di k nel sotto albero radicato in y e lo si sostituisca a k .
 - Situazione simmetrica a quella di prima per il figlio z di x che segue k . In questo caso si prenderà il successore.
 - Se sia y che z hanno $t-1$ chiavi, si uniscano k e il nodo z a y , si cancelli k da x , si deallochi z e si cancelli ricorsivamente da k e da y .
3. Se la chiave non è presente sul nodo interno x , si determini la radice su cui continuare la ricerca:
 - Se $c_i[x]$ è magro, si eseguano i passi 3 e si proceda ricorsivamente a eliminare k .

- Se $c_i[x]$ ha un fratello immediato non magro si proceda così:



- Se sono entrambi magri si proceda invece:



- Se $c_i[x]$ non è magro, si proceda ricorsivamente a cancellare la chiave k a partire dal nodo $c_i[x]$.

Capitolo 4

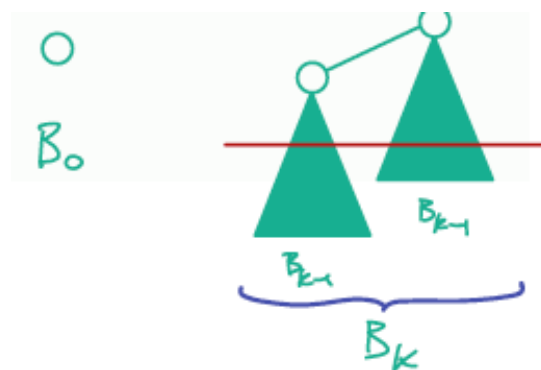
Heap Binomiali

Gli heap binomiali fanno parte di una famiglia di Heap detta mergeable Heap che consistono in degli heap che supportano, oltre che le classiche operazioni, anche l'unione tra due heap.

4.1 Alberi binomiali

Prima di parlare di heap binomiali, è necessario parlare di alberi binomiali che sono la componente fondamentale utilizzata negli heap binomiali. Per ogni $k \in \mathbb{N}$ esiste un albero binomiale B_k di grado k definito in base alla seguente ricorsione:

- B_0 è l'albero formato da un solo nodo.
- Dato B_{k-1} definiamo B_k come la combinazione di due alberi di grado $k-1$ per cui la radice più grande ¹ va alla sinistra della radice più piccola. Sulla base di questa proprietà, possiamo dire che gli alberi binomiali così definiti sono **ordinati**.

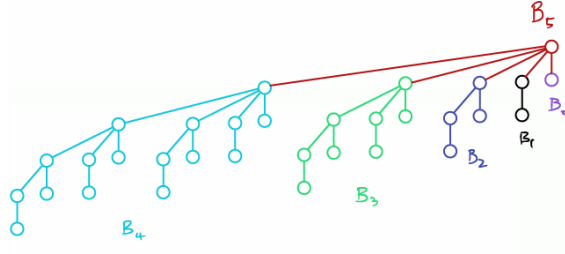


Gli alberi binomiali seguono alcune particolari proprietà importanti da avere in mente:

1. L'albero B_k di grado k ha esattamente 2^k nodi.

¹O più piccola nel caso di un max heap

2. L'altezza di B_k è k .²
3. B_k ha $\binom{k}{i}$ nodi a profondità i .
4. La radice di B_k ha grado k ed ogni altro nodo in B_k ha grado inferiore a k . Inoltre i figli di B_k sono radici di $B_{k-1}, B_{k-2}, \dots, B_0$ in questo esatto ordine.



Queste proprietà possono essere dimostrate tramite induzione.

DIMOSTRAZIONE

Caso base:

- B_0 ha 1 nodo che equivale a 2^0 .
- L'altezza di B_0 è 0.
- B_0 ha $\binom{0}{0} = 1$ nodi a profondità 0.
- La radice di B_0 ha grado 0.

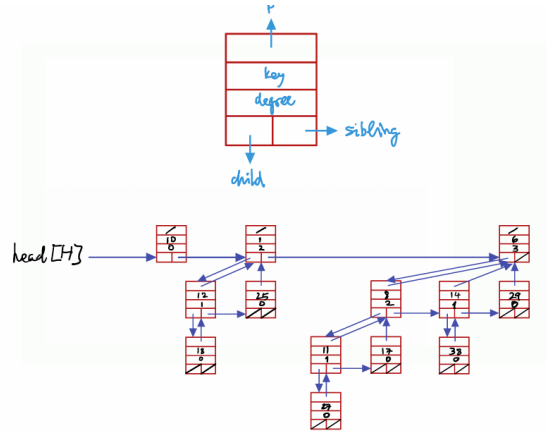
Passo induttivo:

1. B_k ha $\#(B_k) = \#(B_{k-1}) + \#(B_{k-1}) = 2^{k-1} + 2^{k-1} = 2^k$.
2. L'altezza di B_k è uguale a $h(B_{k-1}) + 1 = (k-1) + 1 = k$.
3. Considerando un albero B_k , questo è composto dagli alberi B'_{k-1}, B''_{k-1} . Supponiamo che B'_{k-1} sia l'albero a cui attacca la radice di B''_{k-1} . Noteremo che, per quanto riguarda i nodi al livello d dell'albero B'_{k-1} , questi non saranno cambiati poiché l'albero è rimasto fermo. La situazione è diversa per l'albero B''_{k-1} i cui livelli sono scesi di uno. Da qui, avremo che:

$$Nodi_d = \binom{k-1}{d} + \binom{k-1}{d-1} = \binom{k}{d}$$

4. Il grado della radice di B_k è $deg(B_{k-1}) + 1 = k-1 + 1 = k$. Ciascun nodo di B_{k-1} ha, per ipotesi induttiva, un grado inferiore a $k-1$ per cui anche inferiore a k . Il primo figlio della radice di B_k è radice di B_{k-1} per cui è già in ordine. Di conseguenza, per ipotesi induttiva, i successivi $k-1$ figli sono radici di $B_{k-2}, \dots, B_1$.

²Cammino più lungo dalla radice ad una foglia.

**COROLLARIO**

Sia B un albero binomiale con n nodi, allora ogni nodo in B ha grado al più $\log(n)$.

DIMOSTRAZIONE

Per qualche $k \in \mathbb{N}$ avremo $B = B_k$, quindi $n = 2^k$. Ogni nodo ha grado $\leq k$ ma $k = \log(n)$ da qui la tesi.

4.2 Heap binomiali

Uno Heap Binomiale H non è altro che una foresta di alberi binomiali ordinati per grado crescente tali che:

- Ogni albero in H gode della proprietà di min-heap.
- Per ogni $k \in \mathbb{N}$, H contiene solo un albero di grado k .

Da qui naturalmente notiamo come, per ogni albero, la radice sia la chiave minima e, per ogni heap con n nodi, ci sono al più $\lfloor \log(n) \rfloor + 1$ alberi binomiali. Ogni nodo è composto dalla chiave, dal suo grado, dal padre, dai fratelli e da un puntatore ad un solo figlio.³

4.3 Operazioni Heap Binomiali

Tra le varie operazioni basilari troviamo:

- Make-Binomial-Heap().
- Binomial-Heap-Minimum(H).
- Binomial-Link(y, z).
- Binomial-Heap-Union(H_1, H_2).
- Binomial-Heap-Insert(H, x).

³Nel caso volessimo trovare tutti gli altri figli, ci basterebbe ispezionare tutta la lista di fratelli del figlio

- Binomial-Heap-Extract-min(H).
- Binomial-Heap-Decrease-key(H, x, k).
- Binomial-Heap-Delete(x).

4.3.1 Make-Binomial-Heap()

Questa operazione permette la creazione di un heap binomiale vuoto.

Algorithm 6 MAKE-BINOMIAL-HEAP()

```

1:  $H \leftarrow \text{ALLOCATE-HEAP}()$ 
2:  $\text{head}[H] \leftarrow \text{NIL}$ 
3: return  $H$ 

```

La complessità di questo algoritmo sarà $O(1)$.

4.3.2 Binomial-Heap-Minimum(H)

Questa operazione permette di trovare il minimo nello heap.

Algorithm 7 BINOMIAL-HEAP-MINIMUM(H)

```

1:  $y \leftarrow \text{NIL}$ 
2:  $x \leftarrow \text{head}[H]$ 
3:  $\text{min} \leftarrow \infty$ 
4: while  $x \neq \text{NIL}$  do
5:   if  $\text{key}[x] < \text{min}$  then
6:      $\text{min} \leftarrow \text{key}[x]$ 
7:      $y \leftarrow x$ 
8:   end if
9:    $x \leftarrow \text{sibling}[x]$ 
10: end while
11: return  $y$ 

```

La complessità di questo algoritmo è $O(\log(n))$ poiché, come detto prima, è possibile che vi siano al più $\log(n)$ alberi binomiali.

4.3.3 Binomial-Link(y, z)

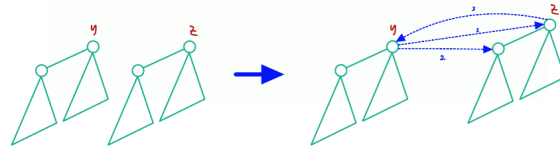
Grazie a questa operazione possiamo unire due alberi y, z dello stesso grado per dare vita ad un albero di grado maggiore.

Algorithm 8 BINOMIAL-LINK(y, z)

```

1:  $p[y] \leftarrow z$ 
2:  $\text{sibling}[y] \leftarrow \text{child}[z]$ 
3:  $\text{child}[z] \leftarrow y$ 
4:  $\text{degree}[z] \leftarrow \text{degree}[z] + 1$ 

```



4.3.4 Binomial-Heap-Union(H_1, H_2)

Questa operazione è la chiave di tutto lo Heap. In pratica permette di unire due heap binomiali rispettando tutte le proprietà sopra citate. L'idea alla base di questa procedura è quella di eseguirla come se fosse una somma tra due numeri binari, eseguendola in questo modo.

$$\begin{aligned}
 103 + 54 &= 2^0 + 2^1 + 2^1 + 2^2 + 2^2 + 2^4 + 2^5 + 2^5 + 2^6 \\
 &= 2^0 + 2^2 + 2^3 + 2^4 + 2^6 + 2^6 + 2^7
 \end{aligned}$$

Come possiamo vedere, non facciamo altro che mettere in ordine il tutto e cominciamo ad eseguire le somme di volta in volta e, quando incontriamo tre numeri uguali,⁴ ne ignoriamo uno e facciamo la somma solo tra gli altri due. Vediamo al procedura:

⁴Che nel caso degli alberi saranno le radici con lo stesso grado

Algorithm 9 BINOMIAL-HEAP-UNION(H_1, H_2)

```

1:  $H \leftarrow \text{MAKE-BINOMIAL-HEAP}()$   $\triangleright$  Creazione di un nuovo albero
2:  $\text{head}[H] \leftarrow \text{BINOMIAL-HEAP-MERGE}(H_1, H_2)$   $\triangleright$  Si uniscono i due heap nello
   heap appena creato mettendo in ordine di grado gli alberi
3: free the objects  $H_1$  and  $H_2$  but not the lists they point to
4: if  $\text{head}[H] = \text{NIL}$  then
5:   return  $H$ 
6: end if
7:  $\text{prev-}x \leftarrow \text{NIL}$ 
8:  $x \leftarrow \text{head}[H]$ 
9:  $\text{next-}x \leftarrow \text{sibling}[x]$ 
10: while  $\text{next-}x \neq \text{NIL}$  do  $\triangleright$  Scorriamo la lista
11:   if  $(\text{degree}[x] \neq \text{degree}[\text{next-}x])$  or  $(\text{sibling}[\text{next-}x] \neq \text{NIL} \text{ and } \text{degree}[\text{sibling}[\text{next-}x]] = \text{degree}[x])$  then  $\triangleright$  Se abbiamo due gradi diversi tra il
     primo e il secondo o sia il primo che il terzo hanno lo stesso grado, scorriamo
12:      $\text{prev-}x \leftarrow x$ 
13:      $x \leftarrow \text{next-}x$ 
14:   else if  $\text{key}[x] \leq \text{key}[\text{next-}x]$  then  $\triangleright$  Se hanno lo stesso grado e il primo è
     più piccolo del secondo, il secondo si attacca al primo.
15:      $\text{sibling}[x] \leftarrow \text{sibling}[\text{next-}x]$ 
16:      $\text{BINOMIAL-LINK}(\text{next-}x, x)$ 
17:   else  $\triangleright$  Caso speculare a quello prima ma il primo si attacca al secondo
18:     if  $\text{prev-}x = \text{NIL}$  then
19:        $\text{head}[H] \leftarrow \text{next-}x$   $\triangleright$  Se necessario, aggiorniamo la testa
20:     else
21:        $\text{sibling}[\text{prev-}x] \leftarrow \text{next-}x$ 
22:     end if
23:      $\text{BINOMIAL-LINK}(x, \text{next-}x)$ 
24:      $x \leftarrow \text{next-}x$ 
25:   end if
26:    $\text{next-}x \leftarrow \text{sibling}[x]$ 
27: end while
28: return  $H$ 

```

Analizziamo la complessità di questo algoritmo. Chiamiamo n_1, n_2 il numero di nodi dei rispettivi alberi. Sappiamo che

$$n \geq n_1 + n_2 \leq \lfloor \log(n_1) \rfloor + \lfloor \log(n_2) \rfloor + 2 \leq 2\lfloor 2\log(n) \rfloor + 2$$

Per cui, la complessità del merge è $O(\log(n))$. Ogni iterazione del ciclo, x o va avanti di una posizione o viene fatto un link che ha tempo costante. Per tanto, questo algoritmo ha complessità $O(\log(n))$.

4.3.5 Binomial-Heap-Insert(H, x)

Tramite questa operazione, facciamo un'inserimento nello heap che non è altro che la creazione di un heap con un solo nodo con cui poi si fa l'unione.

Algorithm 10 BINOMIAL-HEAP-INSERT(H, x)

```

1:  $H' \leftarrow \text{MAKE-BINOMIAL-HEAP}()$ 
2:  $p[x] \leftarrow \text{NIL}$ 
3:  $child[x] \leftarrow \text{NIL}$ 
4:  $sibling[x] \leftarrow \text{NIL}$ 
5:  $degree[x] \leftarrow 0$ 
6:  $head[H'] \leftarrow x$ 
7:  $H \leftarrow \text{BINOMIAL-HEAP-UNION}(H, H')$ 

```

La complessità sarà $O(\log(n))$ poiché dipenderà tutto dalla union.

4.3.6 Binomial-Heap-Extract-min(H)

Questa operazione consiste nell'eliminazione del minimo.

Algorithm 11 BINOMIAL-HEAP-EXTRACT-MIN(H)

```

1: Troviamo la radice  $x$  con chiave minima e la rimuoviamo stacciamo dalla lista.
2:  $H' \leftarrow \text{MAKE-BINOMIAL-HEAP}()$ 
3: Eliminiamo la radice  $x$  dalla parte che abbiamo staccato e facciamo puntare la testa del nuovo heap alla lista dei figli di  $x$  però invertita. Invertendola avremo di nuovo l'ordine crescente
4:  $H \leftarrow \text{BINOMIAL-HEAP-UNION}(H, H')$      $\triangleright$  Riuniamo il tutto tramite la union
5: return  $x$ 

```

Anche in questo caso avremo che la complessità sarà $O(\log(n))$.

4.3.7 Binomial-Heap-Decrease-key(H, x, k)

Questa operazione permette di portare la chiave del nodo x ad un valore k inferiore al valore della chiave precedente. In questo caso supporremo già di avere un puntatore che punta al nodo x .

Algorithm 12 BINOMIAL-HEAP-DECREASE-KEY(H, x, k)

```

1: if  $k > key[x]$  then
2:     error "new key is greater than current key"
3: end if
4:  $key[x] \leftarrow k$ 
5:  $y \leftarrow x$ 
6:  $z \leftarrow p[y]$ 
7: while  $z \neq \text{NIL}$  and  $key[y] < key[z]$  do
8:     exchange  $key[y] \leftrightarrow key[z]$ 
       If  $y$  and  $z$  have satellite fields, exchange them, too.
9:      $y \leftarrow z$ 
10:     $z \leftarrow p[y]$ 
11: end while

```

In questo caso la complessità sarà anche qui $O(\log(n))$ poiché non facciamo altro che scambiare le chiavi fino a far salire quella cambiata al punto giusto dell'albero che avrà al massimo $\log(n)$ livelli.

4.3.8 Binomial-Heap-Delete(x)

Tramite l'operazione di delete non facciamo altro che andare ad eliminare uno specifico nodo x . E, grazie alle operazioni che abbiamo definito prima, questa è un'operazione ormai banale.

Algorithm 13 BINOMIAL-HEAP-DELETE(H, x)

- 1: BINOMIAL-HEAP-DECREASE-KEY($H, x, -\infty$)
 - 2: BINOMIAL-HEAP-EXTRACT-MIN(H)
-

La complessità sarà anche qui $O(\log(n))$.

4.4 Analisi ammortizzata di un inserimento in uno heap binomiale inizialmente vuoto

Scegliamo come funzione potenziale $\Phi(H)$ il numero di alberi binomiali in uno heap. Per cui, se H_0 è un albero binomiale vuoto, avremo che $\Phi(H) \geq \Phi(H_0)$. Il costo reale c_i è dato da:

$$c_i = \# \text{Make-binomial-heap} + \# \text{binomial-link}$$

Il costo ammortizzato di una make-binomial-heap è:

$$c'_i = c_i + \Delta\Phi = 1 + 0 = 1$$

Dunque, il costo di un inserimento è

$$\begin{aligned} c'_i &= c_i + \Delta\Phi \\ &= (1 + \#Link_i) + (\#alberi_i - \#alberi_{i-1}) = \\ &= (1 + \#Link_i) + ((\#alberi_{i-1} + 1 - \#Link_i) - \#alberi_{i-1}) = 2 \end{aligned}$$

Quindi n inserimenti in un heap binomiale inizialmente vuoto possono essere effettuati in tempo $O(n)$.

Capitolo 5

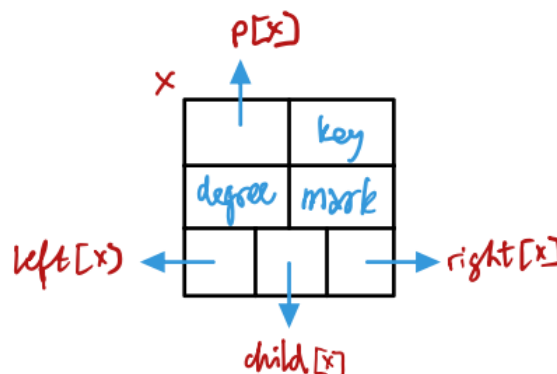
Heap di Fibonacci

Uno Heap di Fibonacci è anch'esso una collezione di alberi binomiale che, però, non sono ordinati ma mantengono comunque la proprietà di min heap. In un certo senso, possiamo dire che questi somiglino agli Heap binomiali con alcune differenze sostanziali:

1. Se negli heap binomiali abbiamo una lista ordinata, qui abbiamo una **lista non ordinata** e che è **circolare**.
2. Negli heap di Fibonacci è ammesso avere più alberi dello stesso grado.
3. Gli alberi in questo caso, pur seguendo una certa logica, non risultano essere ordinati e non sono binomiali.

Un'altra cosa da sottolineare è che questi alberi possono essere "degradati" attraverso alcune procedure di cui parleremo in seguito che, però, proprio grazie alla degradazione, avranno tempi ammortizzati costanti. Dimosteremo, infatti, che Extract min e Delete hanno complessità $O(D(n))$ dove $D(n)$ rappresenta il massimo grado di un nodo nell'heap che sarà $\leq n - 1$ e, ¹per ciò, dimostreremo che $D(n) = O(\log n)$.

Andiamo a guardare la struttura interna di un nodo di un heap di Fibonacci.

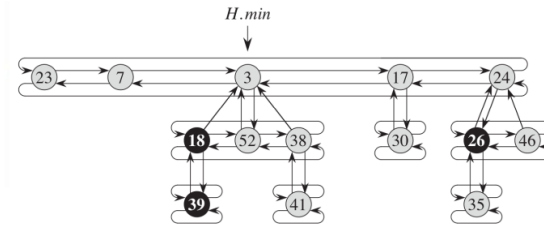


Come possiamo vedere, questi nodi hanno vari campi:

¹Ciò accade perché la peggiore situazione possibile, avendo n nodi, è che ad un solo nodo colleghiamo tutti gli altri e, questo, avrebbe quindi grado $n-1$.

- $p[x]$: sostanzialmente è un puntatore al nodo padre.
- $left[x]$ — $right[x]$: sono dei puntatori ai fratelli sinistri e destri. Naturalmente, scorrere la lista da sinistra o da destra non cambierà i nodi che troveremo dato che si tratta di una **lista circolare**.
- $child[x]$: puntatore che si riferisce al figlio di x . In questo caso ne abbiamo solo uno non perché i nodi possano avere un solo figlio, ma perché da questo troviamo tutti gli altri grazie ai campi $left$ e $right$.
- $degree[x]$ — $deg[x]$: sono entrambi lo stesso campo e vanno a dire il numero di figli che questi nodi hanno.
- $mark[x]$: questo è un campo booleano che ci dice se un nodo ha perso dei figli dall'ultima volta che è diventato a sua volta figli. Vedremo che grazie a questo campo potremo effettuare alcune procedure interessanti.

Definiremo, inoltre, la radice dello heap come $H.min$ o come $min[H]$ che conterrà, per l'appunto il minimo. Questo sarà l'unico accesso allo heap che avremo. E definiremo anche $H.n$ o $n[H]$ che indicherà il numero di chiavi in H .



5.1 Funzione potenziale

La funzione potenziale Φ che utilizzeremo sarà

$$\Phi(H) = t(H) + 2m(H) \geq 0$$

$$t(H) = \# \text{alberi nella lista delle radici}$$

$$m(H) = \# \text{nodi marcati in } H$$

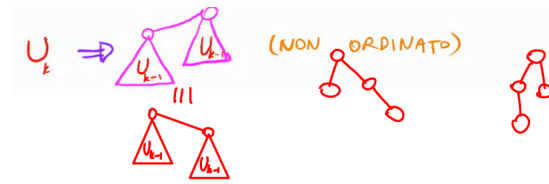
Anche in questo caso, sappiamo per certo che se $\Phi(H_0)$ è l'heap di Fibonacci vuoto, allora avremo che $\Phi(H) \geq \Phi(H_0) \geq 0$. Per le analisi che effettueremo utilizzeremo un upper bound $D(n)$ sul massimo grado di un qualunque nodo nello heap di fibonacci con n nodi e, dimostreremo, che questo $D(n) = O(\log(n))$.

5.2 Alberi binomiali non ordinati

Per ogni albero $k \in N$, esiste un albero **non ordinato** U_k di grado k , definito come:

- U_0 è un albero formato da un solo nodo.

- Dato U_{k-1} , definiamo U_k come l'albero formato due copie di U_{k-1} nel seguente modo



Anche in questo caso possiamo notare la presenza di alcune proprietà fondamentali che caratterizzano questi tipi di alberi. Per cui, per ogni $k = 0, 1, \dots$ valgono:

- U_k ha 2^k nodi.
- L'altezza di U_k è k .
- U_k ha $\binom{k}{i}$ nodi a profondità i .
- La radice di U_k ha grado k ed ogni altro nodo in U_k ha grado inferiore a k . Inoltre i figli di U_k sono radici di $U_0, U_1, \dots, U_{k-1}$ ma **senza alcun particolare ordine**.

5.3 Operazioni su Heap di Fibonacci

Le principali operazioni che troviamo su questo tipo di strutture dati sono:

- Make-Heap;
- Insert;
- Minimum;
- Extract-Min;
- Union;
- Decrease_key;
- Delete.

Andando a considerare tutte le operazioni meno che decrease_key e delete, avremo che lo heap di Fibonacci è una collezione di alberi binomiali non ordinati. E, si osserva, che $D(n) = O(\log(n))$ almeno quando lo heap è formato da alberi binomiali non ordinati e varrà che se $D'(n)$ è il massimo gradi di un nodo in uno heap senza mai usare operazioni di delete e decrease key, allora

$$D'(n) \leq D(n)$$

. Cominciamo ad analizzare le operazioni, partiamo con l'idea di ritardare il più possibile il lavoro di ristrutturazione dello Heap, per cui, molte operazioni che descriveremo adesso sembreranno particolarmente semplici dato che, il lavoro effettivo, verrà poi sviluppato in seguito.

5.3.1 Make-Fibonacci-Heap()

Questa operazione permette la creazione di un nuovo heap di Fibonacci inizialmente vuoto.

Algorithm 14 MAKE-FIBONACCI-HEAP()

```

1:  $H \leftarrow \text{ALLOCATE-NODE}()$ 
2:  $n[H] \leftarrow 0$ 
3:  $\text{min}[H] \leftarrow \text{NIL}$ 
4: return  $H$ 

```

La complessità di questo algoritmo è banale, poiché $\Delta t = 0, \Delta m = 0$ allora avremo che $\Delta \Phi = 0$. Pertanto, $c' = c + \Delta \Phi = c = O(1)$.

5.3.2 Fib-Heap-Insert(H, x)

Questa operazione inserisce un nuovo nodo all'interno dello heap inserendolo, semplicemente, all'interno della lista iniziale e, se necessario, cambiando anche il minimo.

Algorithm 15 FIB-HEAP-INSERT(H, x)

```

1:  $\text{degree}[x] \leftarrow 0$  ▷ Settiamo tutti i parametri a 0 o NIL
2:  $p[x] \leftarrow \text{NIL}$ 
3:  $\text{child}[x] \leftarrow \text{NIL}$ 
4:  $\text{left}[x] \leftarrow x$ 
5:  $\text{right}[x] \leftarrow x$ 
6:  $\text{mark}[x] \leftarrow \text{false}$ 
7: concatenate the root list containing  $x$  with root list  $H$ 
8: if  $\text{min}[H] = \text{NIL}$  or  $\text{key}[x] < \text{key}[\text{min}[H]]$  then ▷ Aggiorniamo il minimo
9:    $\text{min}[H] \leftarrow x$ 
10: end if
11:  $n[H] \leftarrow n[H] + 1$  ▷ Aggiorniamo il numero di nodi presenti

```

Analizzando la complessità, noteremo che questa volta $\Delta t = 1$ e non cambierà, però, Δm , dunque $\Delta \Phi = 1$. La complessità sarà, dunque, $c' = c + \Delta \Phi = c + 1 = O(1)$.

5.3.3 Minimum(H)

L'estrazione del minimo rimane banale poiché questa rimane sempre la radice del nostro Heap.

Algorithm 16 FIB-HEAP-INSERT(H, x)

```

1: return ( $\text{min}(H)$ )

```

La complessità sarà $\Theta(1)$ poiché la funzione potenziale rimane invariata.

5.3.4 Fib-Heap-Union(H_1, H_2)

Questa è la procedura per cui due Heap di Fibonacci vengono uniti. Semplicemente, verrà creato un nuovo Heap e su questo si concateneranno le due liste scegliendo il giusto minore da utilizzare.

Algorithm 17 FIB-HEAP-UNION(H_1, H_2)

```

1:  $H \leftarrow \text{MAKE-FIB-HEAP}()$ 
2:  $H.min \leftarrow H_1.min$ 
3: concatena la lista delle radici di  $H_2$  con la lista delle radici di  $H$ 
4: if ( $H_1.min = \text{NIL}$ ) or ( $H_2.min \neq \text{NIL}$  and  $H_2.min.key < H_1.min.key$ ) then
5:    $H.min \leftarrow H_2.min$ 
6: end if
7:  $H.n \leftarrow H_1.n + H_2.n$ 
8: return  $H$ 

```

La complessità in questo caso rimane $O(1)$ poiché non cambia il numero di alberi alla radice, considerando che $t(H) = t(H_1) + t(H_2)$ non stiamo aggiungendo alcun nodo rispetto a quelli già esistenti.

5.3.5 Fib-Heap-Extract-Min(H)

In questo momento arriva il lavoro più importante da considerare. Il problema nasce in questo momento dal fatto che, dopo aver estratto il minimo, noi non abbiamo idea di dove sia il nuovo valore minimo, per cui, abbiamo bisogno di effettuare una ricerca e, nel mentre, sistemare il tutto.

Algorithm 18 FIB-HEAP-EXTRACT-MIN(H)

```

1:  $z \leftarrow H.min$ 
2: if  $z \neq \text{NIL}$  then
3:   for ciascun figlio  $x$  di  $z$  do
4:     aggiungi  $x$  alla lista delle radici di  $H$   $\triangleright$  Tutti i figli vanno alla lista della radice
5:      $x.p \leftarrow \text{NIL}$ 
6:   end for
7:   rimuovi  $z$  dalla lista delle radici di  $H$ 
8:   if  $z == z.right$  then  $\triangleright$  Verifichiamo che non fosse l'unico elemento nella lista
9:      $H.min \leftarrow \text{NIL}$ 
10:  else
11:     $H.min \leftarrow z.right$ 
12:  end if
13:   $\text{CONSOLIDATE}(H)$   $\triangleright$  Operazione di sistemazione dell'albero
14:   $H.n \leftarrow H.n - 1$ 
15: end if
16: return  $z$ 

```

Vediamo quindi adesso la componente fondamentale della procedura che è il consolidamento della lista delle radici in cui si ripetono i seguenti passi affinché ogni radice nella lista delle radici non avrà un **grado distinto**:

1. Trovare due radici x e y nella list adelle radici con lo stesso grado. E, supponiamo che $x.key \leq y.key$ senza perdere generalità.
2. Colleghiamo y a x : eliminiamo y dalla lista delle radici e trasformiamo y in un figlio di x . Questa operazione è svolta dalla procedura di Fib-Heap-Link. L'attributi $x.degree$ verrà incrementato e la marcatura di y sarà eliminata se presente.

Algorithm 19 CONSOLIDATE(H)

```

1: Sia  $A[0..D(H.n)]$  un nuovo array ▷ Array dei gradi
2: for  $i \leftarrow 0$  to  $D(H.n)$  do
3:    $A[i] \leftarrow \text{NIL}$ 
4: end for
5: for ciascun nodo  $w$  nella lista delle radici di  $H$  do ▷ Controlliamo per ogni
   nodo il suo grado nell'array
6:    $x \leftarrow w$ 
7:    $d \leftarrow x.degree$ 
8:   while  $A[d] \neq \text{NIL}$  do ▷ Se c'è un altro nodo con lo stesso grado di  $x$ 
9:      $y \leftarrow A[d]$ 
10:    if  $x.key > y.key$  then
11:      scambia  $x$  con  $y$ 
12:    end if
13:    FIB-HEAP-LINK( $H, y, x$ ) ▷ Effettuiamo il Link
14:     $A[d] \leftarrow \text{NIL}$  ▷ Resettiamo il grado nell'array
15:     $d \leftarrow d + 1$ 
16:  end while
17:   $A[d] \leftarrow x$  ▷ Aggiungiamo il nuovo grado nell'array dei gradi
18: end for
19:  $H.min \leftarrow \text{NIL}$  ▷ Resettiamo il minimo
20: for  $i \leftarrow 0$  to  $D(H.n)$  do ▷ Troviamo il nuovo minimo tra le radici
21:   if  $A[i] \neq \text{NIL}$  then
22:     if  $H.min == \text{NIL}$  then ▷ Primo nodo a venir visto
23:       crea una lista di radici per  $H$  che contiene solo  $A[i]$ 
24:        $H.min \leftarrow A[i]$ 
25:     else
26:       inserisce  $A[i]$  nella lista delle radici di  $H$  ▷ Via via, li andiamo
       inserendo nella lista
27:       if  $A[i].key < H.min.key$  then
28:          $H.min \leftarrow A[i]$ 
29:       end if
30:     end if
31:   end if
32: end for

```

Per completezza, riportiamo anche la procedura di link.

Algorithm 20 FIB-HEAP-LINK(H, y, x)

- 1: *Rimuove y dalla lista delle radici di H*
 - 2: *Trasforma y in un figlio di x , incrementando $x.degree$*
 - 3: $y.mark \leftarrow \mathbf{false}$
-

Andiamo ad analizzare la complessità di questa operazione. Per quanto riguarda l'extract min, vediamo che l'unica operazione rilevante è quella dove si spostano i figli della vecchia radice alla lista della radice e questa operazione ha complessità $O(D(n))$. Dopodiché, analizzando la consolidate troviamo:

- $O(D(n))$ derivante dall'operazione di creazione dell'array dei gradi e la sua inizializzazione.
- $O(t(H))$ derivante dall'operazione di consolidamento effettivo dell'albero tramite i vari link.
- $O(D(n))$ derivante dalla ricerca del minimo nelle radici inserite che saranno $t(H)$.

Il costo reale sarà quindi

$$\begin{aligned}
 &= O(D(n)) + \text{Derivante dall'operazione di estrazione del minimo} \\
 &+ D(n) + t(H) \text{ Derivante dal processo di consolidamento delle radici} \\
 &= O(D(n) + t(H))
 \end{aligned}$$

Per un totale di:

$$\begin{aligned}
 \Delta t &\leq (D(n) + 1) - t(H) \quad \Delta m \leq 0 \\
 \Delta \Phi &= \Delta t + 2\Delta m \leq D(n) + 1 - t(H) \\
 c' &= c + \Delta \Phi = O(D(n) + t(H)) + D(n) + 1 - t(H) = O(D(n))
 \end{aligned}$$

5.3.6 Decrease Key

L'operazione di Decrease key va a selezionare un nodo per poi diminuire la chiave. Una volta che questa è diminuita si controlla se è una radice e, se non lo è, allora si stacca il nodo mediante l'operazione di cut e poi si eseguono dei tagli a cascata. Infine, si controlla se il minimo è cambiato o meno.

Algorithm 21 FIB-HEAP-DECREASE-KEY(H, x, k)

```

1: if  $k > x.key$  then
2:   error "la nuova chiave è maggiore di quella corrente"
3: end if
4:  $x.key \leftarrow k$ 
5:  $y \leftarrow x.p$ 
6: if  $y \neq \text{NIL}$  and  $x.key < y.key$  then
7:   CUT( $H, x, y$ ) ▷ Inseriamo il nodo nella lista delle radici
8:   CASCADING-CUT( $H, y$ )
9: end if
10: if  $x.key < H.min.key$  then
11:    $H.min \leftarrow x$  ▷ Se è cambiato il minimo, lo aggiorniamo
12: end if

```

Algorithm 22 CUT(H, x, y)

```

1: Rimuove  $x$  dalla lista dei figli di  $y$ , decrementando  $y.degree$ 
2: Aggiunge  $x$  alla lista delle radici di  $H$ 
3:  $x.p \leftarrow \text{NIL}$ 
4:  $x.mark \leftarrow \text{false}$ 

```

Algorithm 23 CASCADING-CUT(H, y)

```

1:  $z \leftarrow y.p$ 
2: if  $z \neq \text{NIL}$  then
3:   if  $y.mark == \text{false}$  then
4:      $y.mark \leftarrow \text{true}$ 
5:   else
6:     CUT( $H, y, z$ )
7:     CASCADING-CUT( $H, z$ )
8:   end if
9: end if

```

Analizziamo la complessità di questa procedura. Finalmente entra in gioco la questione dei nodi marchiati per cui vedremo che cambierà la complessità. Supponiamo che CUT abbia costo costante e dia vita a d cascading cut. Consideriamo che al più un nodo verrà marcato ma almeno $d - 1$ verranno smarcati dai tagli a cascata per cui:

$$\Delta m \leq 1 - (d - 1) = 2 - d$$

Supponiamo quindi che la procedura cascading cut venga richiamata d volte. Avremo che il costo reale sarà $\Theta(d)$, per cui:

$$\begin{aligned}
\Phi(H) &= t(H) + 2m(H) \\
\Phi(H') &\leq (t(H) + d) + 2(m(H) - (d - 1) + 1) = \\
&= t(H) + d + 2m(H) - 2d + 2 + 2 = \\
&= t(H) + 2m(H) - d + 4
\end{aligned}$$

Per cui, segue che $c' = O(d) + \Phi(H') - \Phi(H) = O(d) - d + 4 = O(1)$

5.3.7 Delete

Infine, banalmente avremo la delete che consiste semplicemente in una decrease key al minimo possibile per poi effettuare un'estrazione del minimo.

Algorithm 24 FIB-HEAP-DELETE(H, x)

- 1: FIB-HEAP-DECREASE-KEY($H, x, -\infty$)
 - 2: FIB-HEAP-EXTRACT-MIN(H)
-

La complessità di questa operazione sarà $O(D(n))$.

5.4 Proprietà di Fibonacci

Definiamo la serie di Fibonacci in questo modo:

$$\begin{aligned} F_0 &= 0, & F_1 &= 1 \\ F_k &= F_{k-1} + F_{k-2} & k &\geq 2 \end{aligned}$$

Lemma 1

Questo primo lemma ci dice che $F_{k+2} = 1 + \sum_0^k F_i$ $k \geq 0$.

Dimostrazione

Caso base: $k = 0$

$$F_{k+2} = F_2 = 0 + 1 = 1 \rightarrow 1 + \sum_0^k F_i = 1 + \sum_0^0 F_i = 1 + 0 = 1$$

Passo induttivo:

$$F_{k+1+2} = F_{k+3} = F_{k+2} + F_{k+1} = 1 + \sum_0^k F_i + F_{k+1} = 1 + \sum_0^{k+1} F_i$$

Prima di procedere con il prossimo lemma, andiamo a fare alcune considerazioni su quello che viene definito il **rapporto aureo**. Infatti, questo è strettamente legato alla serie di Fibonacci come vedremo. Il valore del rapporto aureo è $\Phi = \frac{1+\sqrt{5}}{2}$ e, si può dimostrare che $1 + \Phi = \Phi^2$. Inoltre, è importante affermare che $2 > \Phi$.

Lemma 2

$F_{k+2} \geq \Phi^k$ per $k \geq 0$

Dimostrazione

Caso base: $k = 0$

$$\begin{aligned} k = 0 : & \quad F_{k+2} = F_2 = 1 = \Phi^0 = \Phi^k \\ k = 1 : & \quad F_{k+2} = F_3 = 2 > \Phi = \Phi^1 = \Phi^k \end{aligned}$$

Passo induttivo: $k \geq 1$ e, ipotesi induttiva: $F_{h+2} \geq \Phi^h \forall 0 \leq h < k$.

$$\begin{aligned} F_{(k+1)+2} &= F_{k+3} = F_{k+1} + F_{k+2} = \\ &= F_{(k-1)+2} + F_{k+2} \geq \Phi^{k-1} + \Phi^k = \\ &= \Phi^{k-1}(1 + \Phi) = \Phi^{k-1}\Phi^2 = \Phi^{k+1} \end{aligned}$$

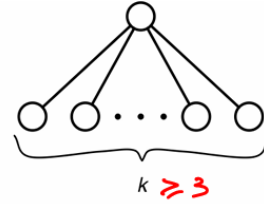
Lemma 3

Sia x un nodo in uno heap di Fibonacci e sia $\text{degree}[x] = k$, siano $y_1, \dots, y_k$ i figli di x nell'ordine in cui sono stati innestati in x . Allora $\text{degree}[y_i] \geq i-2$, per $i = 3, 4, \dots, k$.

Dimostrazione

Sia $i \geq 3$, notiamo che quando y_i è innestato in x al tempo T , il nodo x ha già $y_1, \dots, y_{i-1}$ tra i suoi figli, per cui $\text{degree}_{T-1}[y_i] = \text{degree}_{T-1}[x] \geq i-1$. Dall'istante T , y_i può aver perduto al più un figlio e, quindi, $\text{degree}[y_i] \geq i-2$.

Per ogni $k \geq 3$, trovare una sequenza di operazioni sugli heap di Fibonacci che, a partire da una famiglia vuota di heap, ne costruisca uno formato da un *solo* albero avente la forma riportata nella figura a lato, oppure stabilire che una siffatta sequenza non esiste.



SUPPONIAMO CHE SIA
POSSIBILE COSTRUIRE
UN HEAP COME QUELLO
A LATO.



SIANO y_1, y_2, y_3 I PRIMI NODI AD ESSERE STATI
INNESTATI IN x NELL'ORDINE DATO.

PER IL LEMMA PRECEDENTE SI DOVREBBE
AVERE $\text{deg}[y_2] \geq 1$, ASSURDO.

PERTANTO UNA SIFFATTA SEQUENZA NON PUÒ
ESISTERE.

Lemma 4

Sia x un nodo in uno heap di Fibonacci e sia $\text{degree}[x] = k$, allora $\text{size}[x] \geq F_{k+2}$, dove $\text{size}[x]$ è il numero di nodi nel sottoalbero radicato in x . **Dimostrazione**
Innanzitutto poniamo:

$$s_j = \min.\text{size}[z]$$

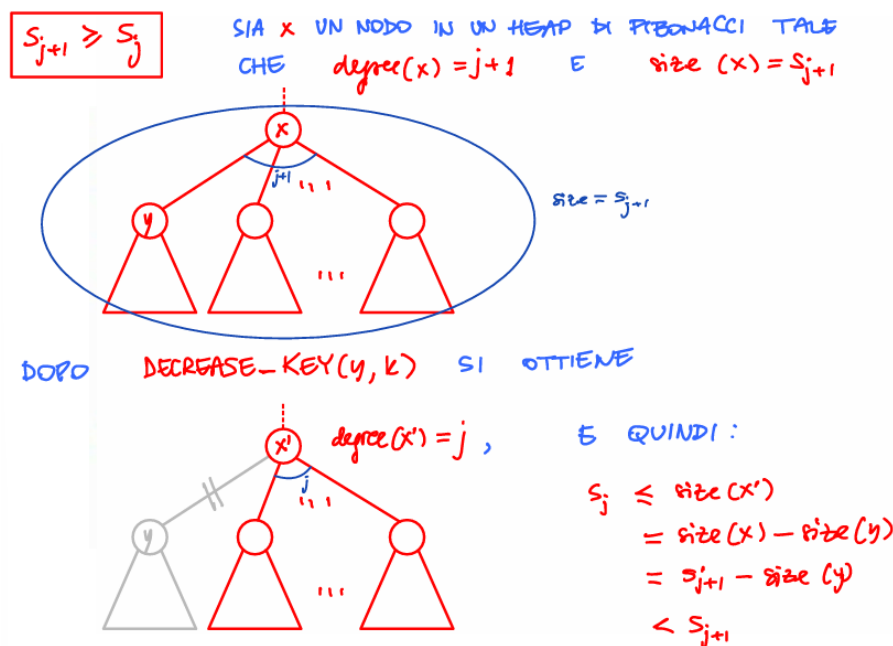
$$\text{degree}[z] = j$$

$$z \in H$$

$$H \in H_u \quad H_u = \text{Famiglia di tutti gli Heap di Fibonacci}$$

Si ha che

$$s_0 = 1, s_1 \geq 2, \quad s_{j+1} \geq s_j \quad j \in \mathbb{N}$$



L'obiettivo sarà quindi dimostrare che

$$s_k \geq F_{k+2} \quad \text{se } \text{deg}[x] = k, \quad \text{size}[x] \geq s_k \geq F_{k+2}$$

Dimostrazione

Dimostriamo innanzitutto che

$$s_0 = 1, s_1 \geq 2, \quad s_{j+1} \geq s_j \quad j \in N$$

Se $k = 0$, allora vuol dire che $\text{degree}[x] = 0$, per cui, il nodo non ha alcun figlio e, quindi $\text{size}[x] = 1$.

Se $k = 1$, allora $\text{degree}[x] = 1$, quindi x ha un solo figlio, per cui $\text{size}[x] \geq 2$

Andiamo adesso per induzione, per cui, supponiamo l'ipotesi $\text{size}[x] \geq F_{j+2}$, sia corretta per ogni $j < k$. Dimostriamo che $s_j \geq F_{j+2} \quad \forall j = 0, 1, \dots$

$$j = 0: \quad s_j = 1, \quad F_{j+2} = 1$$

$$j = 1: \quad s_j \geq 2, \quad F_{j+2} = F_3 = 2$$

$$j \geq 2: \quad s_j = \text{size}[y_1] + \text{size}[y_2] + \dots + \text{size}[y_j] + 1 \geq \quad \text{Segue dal lemma 3}$$

$$\geq 1 + s_{j-2} + s_{j-3} + \dots + s_{j-2} + 1 = \quad \text{Siccome } s_j \geq F_j + 2, \text{ possiamo applicare il Lemma 1}$$

$$= 2 + \sum_{i=2}^j s_{i-2} \geq 2 + \sum_{i=2}^j F_i = 1 + \sum_{i=0}^j F_i = F_{j+2}$$

Poiché $\text{degree}[x] = k$ si ha che $\text{size}[x] \geq s_k \geq F_{k+2}$.

Corollario 1

$$\text{size}[x] \geq \Phi^{\text{degree}[x]}$$

Corollario 2

$$D(n) \leq \lfloor \log_{\Phi}(n) \rfloor \text{ da cui } D(n) = O(\log(n)).$$

Dimostrazione

Sia x un nodo in uno heap di Fibonacci con n nodi. Si ha che $n \geq \text{size}[x] \geq \Phi^{\text{degree}[x]}$, pertanto:

$$\begin{aligned} \text{degree}[x] &\leq \lfloor \log_{\Phi}(n) \rfloor \\ D(n) = \max \text{degree}[x] &\leq \lfloor \log_{\Phi}(n) \rfloor \end{aligned}$$

Capitolo 6

Strutture Dati per Insiemi Disgiunti

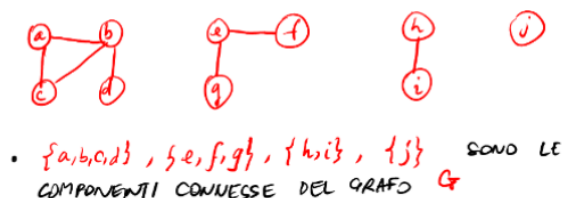
Le strutture dati per insiemi disgiunti sono utili alla rappresentazione efficace di famiglie dinamiche soggette a operazioni di:

- **Make-Set(x)**: Introduzione di un singolo $\{x\}$.
- **Union(x,y)**: Unione degli insiemi disgiunti che contengono, rispettivamente, x e y .
- **Find-Set(x)**: Ricerca dell'insieme che contiene x .

Per l'analisi di queste strutture saranno prese in considerazione m operazioni di Make-Set, Union e Find-Set, comprendenti n operazioni di Make-Set. Varrà che $n \leq m$ e che il numero di union sia $\leq n - 1$ dato che non le operazioni di union si fanno a due a due.

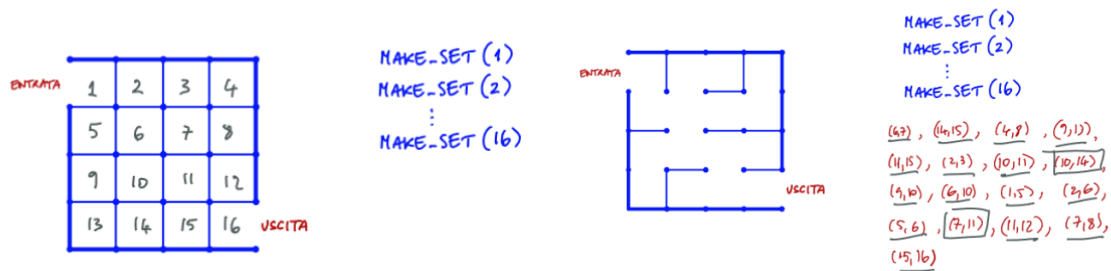
6.1 Componenti Connesse in un Grafo

Sia $G = (V, E)$ un grafo non orientato e siano $u \sim v$ per $u, v \in V$ se esiste un cammino da u a v in G . La relazione di $\sim$ è una relazione di equivalenza su V e le **componenti connesse** di G sono le classi di equivalenza di $\sim$.



6.2 Generazione di Labirinti

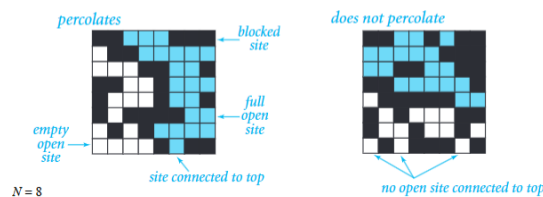
Allo stesso modo, possiamo anche generare dei labirinti sfruttando queste operazioni dato che, effettuando una serie di union, riusciremo a generare una sola componente connessa equivalente a un labirinto.



Per scegliere quali Union fare, si crea una lista di tutti i muri presenti e la si randomizza. Dopodiché scorri tutta la lista e, se i due elementi sono già nella stessa componente, non fai nulla, altrimenti li unisci tramite una union.

6.3 Percolazione

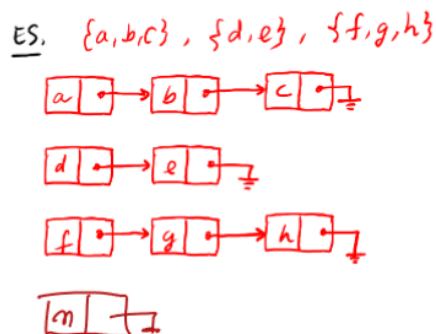
Un'altra interessante applicazione è la verifica della **percolazione** in un solido. Questa consiste nel passaggio lento di un liquido in uno specifico solido in considerazione. Immaginiamo questa situazione:



Come possiamo vedere, a sinistra esiste un percorso dalla parte sopra alla parte sotto del blocco, per cui abbiamo percolazione; al contrario, nel blocco destro notiamo come l'acqua non possa passare a causa di blocchi non vuoti. Questo tipo di eventi può essere simulato in una griglia $N \times N$ per cui, ogni blocco, può essere pieno con probabilità p e vuoto con probabilità $1 - p$. Noteremo che, maggiore sarà p e maggiore sarà la probabilità di percolazione.

6.4 Implementazione di Insiemi Disgiunti

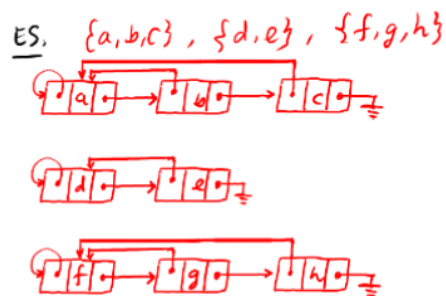
Andiamo per gradi a vedere come poter implementare questo tipo di strutture dati. La scelta più banale ricade su una semplice lista di nodi, per cui:



Per quanto riguarda le operazioni vediamo che:

- **Make-Set:** Ovvio, ci basta aggiungere sempre in testa.
- **Find-Set:** Possiamo utilizzare un riferimento alla Tail per capire a quale insieme si appartiene.
- **Union:** L'implementazione non è poi così semplice.

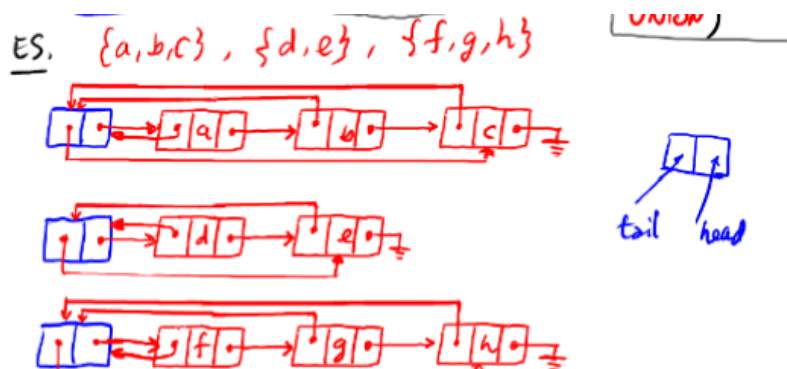
Per ciò, decidiamo di modificare la nostra lista facendo sì che in ogni nodo vi sia un puntatore al **rappresentante** e uno al prossimo nodo nella lista.



In questo caso vediamo che:

- **Make-Set:** Ovvio, come prima.
- **Find-Set:** Mediante il calcolo di $repr[x]$ è immediato.
- **Union:** Effettuiamo una concatenazione e successivamente aggiorniamo tutti i campi rappresentante.

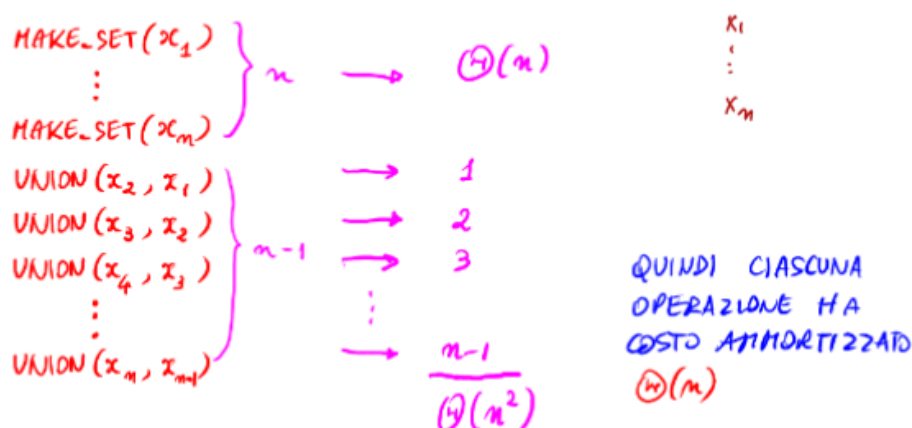
Ci rendiamo però conto come l'unione richieda necessariamente di scorrere tutta la lista per trovare la tail. Per evitare ciò, possiamo tenere un **nodo fittizio** all'inizio della lista che contiene le informazioni che riguardano la tail e la head per evitare di scorrere tutta la lista.



Infine, al fine di ottimizzare ulteriormente il processo, possiamo anche eliminare il campo tail e fare tutte le unioni in testa dopo l'head.



Consideriamo una sequenza di $2n - 1$ operazioni e ragioniamo su queste:



I costi delle varie union derivano principalmente dall'aggiornamento del campo rappresentante delle varie liste che, come possiamo notare, sono sempre in ordine $union(x_i, x_{i-1})$. Questo è scelto per mostrare il caso peggiore poiché la lista più lunga è sempre x_{i-1} ed è quella che, quindi, subirà tutti gli aggiornamenti. Da qui, intuitivamente, possiamo applicare un'euristica chiamata **Unione per Ranghi** in cui appendiamo la lista più corta a quella più lunga così da fare meno aggiornamenti possibili. Per fare ciò, tenendo ancora in considerazione il nodo fittizio, aggiungiamo anche il campo **size** che ci indica la lunghezza della lista che servirà proprio per fare il confronto durante l'unione. Da qui, la seguente analisi:

- SIA DATA UNA SEQUENZA $\mathcal{S}$ DI OPERAZIONI MAKE_SET, UNION E FIND_SET E PONIAMO:

$n = \# \text{ MAKE_SET}$

$m = \# \text{ MAKE_SET} + \# \text{ UNION} + \# \text{ FIND_SET}$

$k = \# \text{ AGGIORNAMENTI AI CAMPI repr}$

- LA COMPLESSITA' DELLA SEQUENZA $\mathcal{S}$ E' CHIARAMENTE

$O(m + k)$

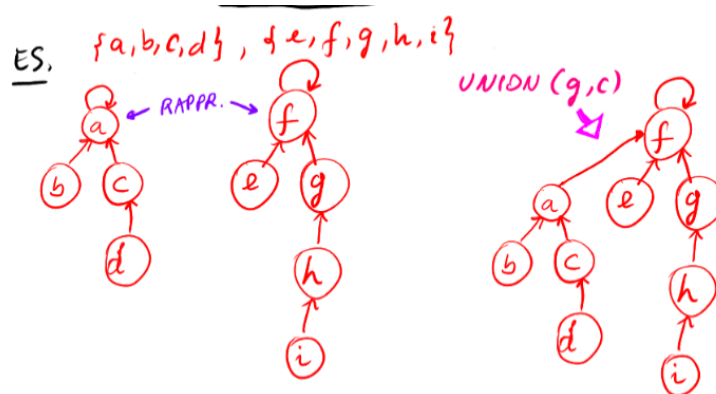
	AMMORTIZED
FIND_SET(x)	$O(1)$
MAKE_SET(x)	$O(\lg m)$
UNION(x, y)	$O(1)$

Questa analisi è fatta poiché, grazie all'euristica di unione per ranghi, ogni aggiornamento di rappresentante, la lista contenente x aumenta di almeno 2. Quindi

il campo $repr[x]$ può cambiare al più $\lfloor \log(n) \rfloor^1$ volte. Di conseguenza, il numero totale di aggiornamenti del campo $repr$ di tutti i nodi è $O(n \log(n))$, per cui, la complessità della sequenza è $O(m + n \log(n))$.

6.5 Insiemi Disgiunti mediante Alberi Disgiunti

Un modo per migliorare ulteriormente l'implementazione è tramite l'utilizzo di alberi disgiunti con euristiche di **unione dei ranghi** e **compressione dei cammini**.



Definiamo in maniera opportuna il rango dell'albero legandolo al numero di archi del cammino semplice più lungo dal nodo x a una foglia. Anche in questo caso si sceglie di innestare il rango più piccolo in quello più alto e, nel caso di parità, si sceglie arbitrariamente.

L'euristica della compressione dei cammini consiste nell'ottimizzazione della find-set per iterazioni successive. Quindi, quello che facciamo è prendere tutti i nodi incontrati nell'esecuzione di una find-set e li innestiamo alla radice.



Per quanto riguarda le complessità utilizzando le euristiche vediamo:

¹Ciò deriva dal fatto che se x appartiene a una lista lunga n , possono esserci state al massimo $\lfloor \log(n) \rfloor$ union, e quindi può aver cambiato un massimo di $\lfloor \log(n) \rfloor$ rappresentanti.

- L'UNIQUE PER RANGHI (SENZA COMPRESSIONE DEI CAMMINI)
DA' LUOGO AD UNA COMPLESSITA' $O(m \lg n)$
- LA COMPRESSIONE DEI CAMMINI (SENZA UNIQUE PER RANGHI)
DA' LUOGO AD UNA COMPLESSITA' $O(m + f \cdot (1 + \log_{2+f/m} n))$,
DOVE $f = \# \text{ FIND_SET}$
- UTILIZZANDO ENTRAMBE LE EURISTICHE, SI
OTTIENE UNA COMPLESSITA' $O(m \alpha(n))$, DOVE
 $\alpha(n)$ E' UN'INVERSA DELLA FUNZIONE DI ACKERMANN
- IN PRATICA $\alpha(n) \leq 4$, SEBBENE $\alpha(n) \rightarrow +\infty$.

Capitolo 7

Alberi ricoprenti minimi

Gli alberi ricoprenti minimi sono dei particolari alberi per cui, a partire da un grafo non orientato connesso $G = (V, E)$ e una funzione peso $w : E \rightarrow R$, restituisce un albero $T = (V, E')$ con $E' \subseteq E$ tale che per ogni altro albero $T_1 = (V, E'_1)$ si abbia che

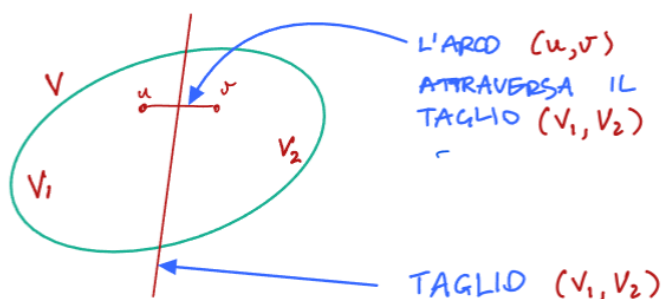
$$w(T) = \sum_{e \in E'} w(e) \leq \sum_{e \in E} w(e) = w(T_1)$$

Cioè, tutti i nodi devono essere connessi e gli archi che li collegano devono avere costo minimo rispetto a tutti gli altri possibili collegamenti.

Naturalmente, il nostro obiettivo sarà quello di non effettuare una ricerca esaustiva poiché questa ha un costo su n vertici di n^{n-2} , cioè, un costo esponenziale.

7.1 Taglio e Colorazione

Sia $G = (V, E)$ un grafo non orientato e connesso con funzione peso $w : E \rightarrow R$, definiamo un **taglio** di G , una partizione (V_1, V_2) di V non banale. Un arco (u, v) tale che $u \in V_1, v \in V_2$ si dice che **attraversa il taglio**.



Faremo utilizzo di un **processo di colorazione**, per cui, finché possibile applicheremo i seguenti passi:

- **Passo Blu:** si seleziona un taglio non attraversato da archi blu e, tra gli archi non colorati, prendiamo quello di peso minimo e lo coloriamo di blu.

- **Passo Rosso:** si seleziona un ciclo semplice privo di archi rossi, tra gli archi non colorati del ciclo selezioniamo quello di peso massimo e lo coloriamo di rosso.

Durante tutto questo processo di colorazione varrà il seguente invariante del colore per cui “ogni passo della procedura di colorazione esiste un albero ricoprente minimo contenente tutti gli archi blu e nessun arco rosso.”

Teorema

Ogni esecuzione del processo di colorazione, colora tutti gli archi e mantiene l'invariante di colore. Come conseguenza di questo teorema, avremo che alla fine della procedura di colorazione, l'insieme di archi blu forma un MST. Possiamo dimostrare questa cosa:

Sia E' l'insieme di archi blu alla fine del processo di colorazione e sia $T = (V, E'')$ un MST tale che $E' \subseteq E''$ che esiste grazie all'invariante di colore. Poiché E'' non contiene archi rossi, si ha che $E'' \subseteq E'$, da cui $E'' = E'$. Quindi, (V, E') è un MST.

Adesso dimostriamo, invece, la conservazione della proprietà d'invariante di colore:

Caso base

Inizialmente l'invariante di colore è banalmente dimostrata poiché non ci sono né archi blu né rossi.

Induzione

Sia $T = (V, E')$ un MST che contiene tutti gli archi blu al passo k e nessun arco rosso. Al passo $k + 1$ possiamo avere due casi:

1. **Colora l'arco che attraversa il taglio (V_1, V_2) di blu:** Se $e \in T$, T verifica banalmente l'invariante di colore poiché l'arco era già presente nell'albero. Se $e \notin T$ si consideri il cammino da u a v . Sia (u_1, v_1) un arco che attraversa il taglio (V_1, V_2) , questo sarà colorato. Si consideri T' tale che:

$$T' = (T \setminus \{u_1, v_1\}) \cup \{u, v\}$$

Poiché $w(u, v) \leq w(u_1, v_1)$ si ha che $w(T') \leq w(T) \leq w(T')$ da cui $w(T') = w(T)$, cioè T' è un MST che contiene tutti gli archi blu al passo $k + 1$.

2. **Colora di rosso l'arco con peso massimo nel ciclo:** Ancora una volta, se $e \notin T$ non c'è nulla da dimostrare poiché l'arco rosso non è in T . Se $e \in T$, $T \setminus \{e\}$ ha due componenti connesse che determinano una partizione (V_1, V_2) di V . Sia f un arco sul cammino $(u_2, \dots, u_r, u_1)$, f non può essere blu perché non è in T e non può essere nemmeno rosso perché fa parte del ciclo scelto per effettuare il passo rosso. Dunque, f non è colorato e avremo che $w(e) \geq w(f)$, sia T'' tale che:

$$T'' = (T \setminus \{e\}) \cup \{f\}$$

Questo è uno spanning tree e, poiché $w(e) \geq w(f)$ avremo che $w(T) \leq w(T'') \leq w(T)$ e ciò, implica che $w(T'') = w(T)$ il che verifica l'invariante di colore.

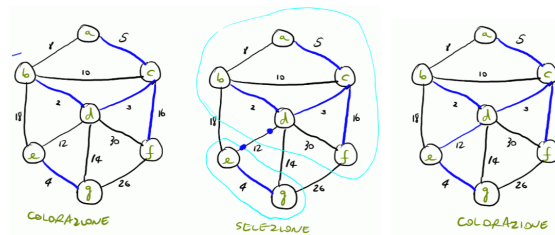
Non ci resta, infine, che dimostrare che al termine dell'esecuzione, tutti gli archi sono colorati:

Sia $e = (u, v)$ un arco non ancora colorato e sia T un MST che verifica l'invariante di colore. Se $e \in T$, cancellando e da T si ottengono due componenti connesse che determinano un taglio (V_1, V_2) . Questo taglio non è attraversato da alcun arco blu, per cui, possiamo applicare un passo blu. Se $e \notin T$, sia $\pi_{u,v}$ un cammino da u a v in T , si consideri il ciclo semplice $\pi_{u,v}; e$ tale ciclo non contiene archi rossi e contiene archi non colorati, per cui possiamo applicare un passo rosso.

7.2 Algoritmo di Boruvka

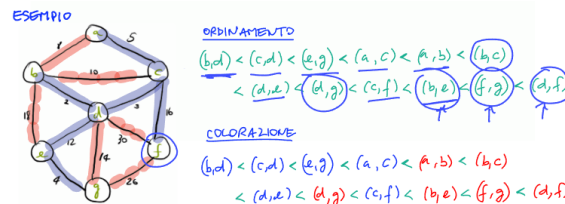
Questo algoritmo suppone che gli archi siano ordinati totalmente da un ordinamento che è in accordo con quello sui pesi, cioè che $w(e) < w(e') \rightarrow e < e'$. Si applicherà il seguente passaggio finché possibile:

Per ciascun albero blu, si selezioni l'arco incidente minimo non ancora colorato e lo si colori di blu. Doppiamo immaginarci questo processo come un unico step che fanno tutti gli alberi contemporaneamente e non come un processo sequenziale, per cui, può capitare che a una interazione due nodi scelgano lo stesso arco. Vediamone un esempio:



7.3 Algoritmo di Kruskal

Seguendo un ordine crescente per costi di archi, se l'arco corrente è contenuto in un albero blu¹, allora lo si colori di rosso, altrimenti lo si colori di blu.



Per dimostrare che questo algoritmo è corretto, procederemo per induzione:

- **op(e_i) colora e_i di blu:** sia $e_i = (u_i, v_i)$, si ha che u_i, v_i appartengono a due alberi distinti S_i, T_i ². Consideriamo il taglio $(S_i, V \setminus S_i)$, questo è attraversato dall'arco e_i e, tutti gli archi $e : w(e) < w(e_i)$ sono già colorati a causa del

¹Cioè se l'arco considerato connette due nodi che sono già connessi nello stesso albero blu

²Altrimenti non potremmo colorarlo di blu

funzionamento dell'algoritmo. Per cui, e_i ha peso minimo tra tutti gli archi non colorati nel taglio, dunque può essere colorato di blu.

- **op(e_i) colora e_i di rosso:** sia $e_i = (u_i, v_i)$, u_i, v_i appartengono allo stesso albero blu, sia π il cammino da u_i a v_i in T , si consideri il ciclo $\pi_i; e_i$, tale ciclo non contiene archi rossi ed e è l'unico arco non colorato, per cui possiamo fare un passo rosso.

Per gestire questo algoritmo, possiamo sfruttare ciò che abbiamo fatto con le strutture dati per insiemi disgiunti, ottenendo la seguente complessità:

COMPLESSITÀ

$O(E \log V)$

INFATTI, SI HA:

- $O(E \log V)$: ORDINAMENTO DEGLI ARCHI
- UNION-FIND:
 - $|V|$ MAKESET
 - $2|E|$ FIND-SET
 - $|V|-1$ UNION

$2(|V|+|E|)-1 = O(E)$ OPERAZIONI DI CUI $|V|$ MAKESET

$\rightarrow (2(|V|+|E|)-1) \alpha(V) = O(E \log V)$

7.4 Algoritmo di Prim

Passiamo, infine, all'algoritmo di Prim. Si seleziona un nodo s detto **sorgente** e si esegua per $|V| - 1$ volte la seguente operazione:

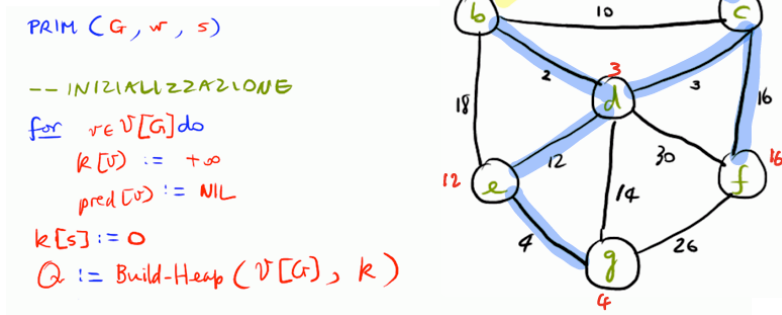
- si seleziona l'albero blu T contenente S .
- Si selezioni l'arco minimo incidente su T e lo si colori di blu.

In base a questo algoritmo possono esserci più MST in base alla sorgente da cui si parte. Tuttavia, se vale che la funzione peso è **iniettiva**, allora G ha un unico MST e ciò si può dimostrare nel seguente modo: Siano $T_1 \neq T_2$ due MST di G , sia $(u, v) \in E[T_1] \setminus E[T_2]$, sia π un cammino da u a v in T_2 , ovviamente $\pi \not\subseteq T_1$. Sia

$$(u', v') \in (E[T_2] \setminus E[T_1]) \cap \pi \subseteq E[T_1] \Delta E[T_2]$$

Pertanto $w(u', v') > w(u, v)$, sia $T'_2 = T_2 \setminus \{(u', v')\} \cup \{(u, v)\}$ si ha che T'_2 è un albero e $w(T'_2) = w(T_2) - w(u', v') + w(u, v) < w(T_2)$ e ciò sarebbe assurdo perché abbiamo detto che T_2 è un MST.

Vediamo un esempio di esecuzione dell'algoritmo di Prim:



```

-- COSTRUZIONE
while Q ≠ ∅ do
    u := Extract_Min (Q)
    for v ∈ Adj[u] do
        if v ∈ Q and w(u,v) < k[v] then
            Decrease_Key (Q, v, w(u,v))
            pred[v] := u
return {(pred[v], v) : v ∈ V[G] \ {s}}

```

$O(V)$ Extract_Min
 $O(E)$ Decrease_Key

Infine, diamo uno sguardo alla complessità di questo algoritmo, tenendo in considerazione le diverse implementazioni di Heap che abbiamo visto nei precedenti capitoli:

COMPLESSITA'

INIZIALIZZAZIONE: $O(V)$

COSTRUZIONE: $|V|$ Extract_Min
 $|E|$ Decrease_Key

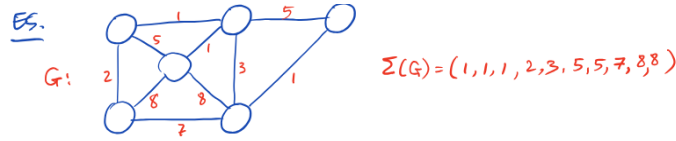
	heap binario	heap binom.	heap di Fibon.
INIZIALIZZAZIONE	$O(V)$	$O(V)$	$O(V)$
$ V $ Extract_Min	$O(V \lg V)$	$O(V \lg V)$	$O(V \lg V)$
$ E $ Decrease_Key	$O(E \lg V)$	$O(E \lg V)$	$O(E)$
	$O(E \lg V)$	$O(E \lg V)$	$O(E + V \lg V)$

7.5 Signature di uno Spanning Tree

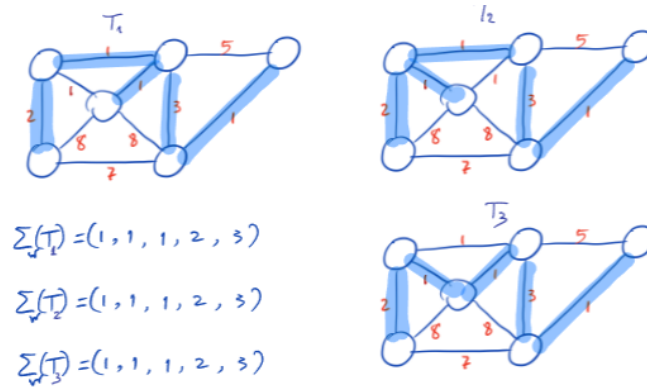
Siano $r_1 = (V, T_1)$ e siano $r_2 = (V, T_2)$ due spanning di un medesimo grafo, definiamo **distanza** tra r_1 e r_2 ponendo:

$$d(r_1, r_2) = d(T_1, T_2) = \frac{|T_1 \Delta T_2|}{2}$$

Sia $G = (V, E)$ un grafo non orientato con una funzione peso $w : E \rightarrow R$, la **segnatura** $\sum_w(G)$ di G è la sequenza dei pesi degli archi di G in ordine non decrescente:



Supponiamo adesso di prendere i seguenti MST e le relative signature:



Come possiamo notare, tutte le signature di questi MST sono uguali.

Teorema Unicità della signature di MST

Due qualunque MST di uno stesso grafo pesato non orientato e connesso hanno la stessa segnatura.

Sia (G, w) un grafo non orientato connesso e pesato contenente degli MST con signature diverse. In particolare, siano $r_1 = (V, T_1)$ e $r_2 = (V, T_2)$ due MST con signature distinte tali che la loro distanza $d(T_1, T_2)$ sia minima.

Sia $e = (u, v)$ un arco di peso minimo in $T_1 \Delta T_2^3$, e supponiamo che $e \in T_1$ e $e \notin T_2$. Sia (V_1, V_2) il taglio di G indotto cancellando dall'MST r_1 l'arco e .

Sia π un cammino in r_2 da $u \in V_1$ a $v \in V_2$, e sia e' un arco in π che attraversa il taglio, allora $e' \in T_2 \setminus T_1$. Per la minimalità di $w(e)$ in $T_1 \Delta T_2$ si ha che $w(e) \leq w(e')$. Pertanto, posto $T'_2 = (T_2 \setminus \{e'\}) \cup \{e\}$ si ha che $r'_2 = (V, T'_2)$ è uno spanning tree di G , per cui:

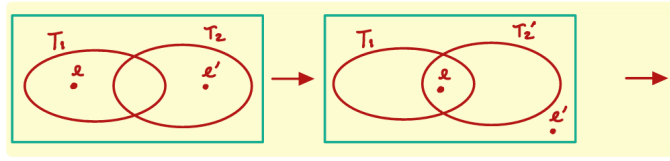
$$w(T_2) \leq w(T'_2) = w(T_2) - w(e') + w(e) \leq w(T_2)$$

Da cui $w(T'_2) = w(T_2)$ e, quindi, $r'_2 = (V, T'_2)$ è un MST di G . Notiamo che:

$$\sum_w(r'_2) = \sum_w(r_2) \neq \sum_w(r_1)$$

E quindi:

³Cioè l'insieme composto dagli elementi univoci di entrambi gli insiemi



→ $T_1 \Delta T_2' = (T_1 \Delta T_2) \setminus \{e, e'\}$, E QUINDI

$$d(T_1, T_2') = \frac{|T_1 \Delta T_2'|}{2} = \frac{|T_1 \Delta T_2|}{2} - 1 < d(T_1, T_2),$$

CONTRADDICENDO LA MINIMALITÀ DI $d(T_1, T_2)$.

7.6 Minimalità della Signature di un MST

Siano $\sigma = (s_1, \dots, s_k)$ e $\sigma' = (s'_1, \dots, s'_k)$ due k-uple di numeri reali, poniamo $\sigma < \sigma'$ se σ precede σ' lessicograficamente, cioè:

$$s_1 = s'_1, s_2 = s'_2, \dots, s_{l-1} = s'_{l-1}, s_l < s'_l$$

Queste relazioni godono delle seguenti proprietà:

- La relazione di precedenza è **transitiva**.
- Sia (G, w) un grafo non orientato pesato, con $G = (V, E)$ e sia $w' : E \rightarrow R$ una qualunque funzione peso tale che:
 - $w'(e) \leq w(e)$ per cui $e \in E$
 - $w'(f) < w(f)$ per qualche $f \in E^4$

Allora la segnatura di (G, w') precede la segnatura di (G, w) .

Teorema

Uno spanning tree è un MST se e solo se la sua segnatura è **lessicograficamente minima**.

Dimostrazione

Supponiamo esista un grafo pesato (G, w) non orientato e connesso con $G = (V, E)$ i cui MST hanno una segnatura σ non minima. Siano r, r' con $r = (V, T)$, $r' = (V, T')$ una coppia di spanning tree tali che:

- r sia un MST
- $\sigma' = \sum_w(r') < \sum_w(r) = \sigma$
- $d(r, r')$ sia minima
- $e \in T \Delta T'$ di peso minimo con $e = (u, v)$.

Distinguiamo adesso i due casi:

⁴Cioè almeno un elemento deve essere diverso

- $e \in T \setminus T'$: Sia (V_1, V_2) il taglio G indotto cancellando e dall'MST r . Sia π un cammino r' da u a v e sia e' un arco in π che attraversa il taglio. Allora $w(e) \leq w(e')$. Poiché $(T' \setminus \{e'\}) \cup \{e\}$ è uno spanning tree, si ha che:

$$\sum_w (T' \setminus \{e'\} \cup \{e\}) \leq \sum_w (T') < \sum_w (T)$$

Inoltre:

$$d((T' \setminus \{e'\}) \cup \{e\}, T) < d(T', T)$$

E ciò contraddice la minimalità di $d(T', T)$.

- $e \in T' \setminus T$: Sia (V_1, V_2) il taglio G indotto cancellando dallo spanning tree r' . Sia π un cammino in r da u a v e sia e' un arco in π che attraversa il taglio, allora $e' \in T \setminus T'$. Al solito, $(T' \setminus \{e'\}) \cup \{e\}$ è uno spanning tree di $G = (V, E)$. Poiché $w(e) \leq w(e')$ si ha che:

$$w(T) \leq w((T' \setminus \{e'\}) \cup \{e\}) \leq w(T')$$

Dunque: $\sum_w (T') < \sum_w (T) = \sum_w ((T' \setminus \{e'\}) \cup \{e\})$, inoltre $d(T', (T' \setminus \{e'\}) \cup \{e\}) < d(T', T)$ e ciò contraddice la minimalità di $d(T', T)$.

Dimostriamo adesso il viceversa.

Sia r uno spanning tree con segnatura minima e sia r' un mst di G . Per la prima parte del teorema r' è minima, dunque $\sum_w(r) = \sum_w(r')$ da cui si dimostra banalmente che anche r è un mst.

7.7 Clustering

Supponiamo di avere una collezione di oggetti che è necessario inserire in dei gruppi di oggetti simili. È necessario avere una **funzione distanza** tra gli oggetti in modo da avere un grado di somiglianza tra questi. Il problema del **clustering** è il problema di suddividere opportunamente un dato insieme di oggetti in modo tale che oggetti vicini in termini della funzione distanza stabilita, appartengano allo stesso cluster e oggetti lontani a cluster diversi.

Per fare ciò, sfruttiamo l'algoritmo di Kruskal precedentemente mostrato. Grazie a questo si ricercano cluster che assicurino la massima distanza tra ciascuna coppia di cluster distinti.

Sia $U = \{p_1, p_2, \dots, p_n\}$ un insieme di oggetti dotato di una funzione distanza $d : U \times U \rightarrow R$ tale che:

- $d(p_i, p_i) = 0$ per ogni $i = 1, \dots, n$
- $d(p_i, p_j) > 0$ per ogni $1 \leq i, j \leq n$
- $d(p_i, p_j) = d(p_j, p_i)$ per ogni $1 \leq i, j \leq n$.

Sia fissato $k \leq n$ assegnato e chiamato **numero di cluster**. Una partizione $C_1, C_2, \dots, C_k$ di U tale che $c_i \neq 0$ per ogni $i = 1, \dots, k$, questa è detta k -clustering

di U . Dato un k -clustering $C = \{C_1, C_2, \dots, C_k\}$ di U , la **separazione di C** è data da:

$$d(C_i, C_j) = \min d(p, q) : p \in C_i, q \in C_j \quad \text{Distanza tra due cluster}$$

$$sep(C) = \min_{i \neq j} d(C_i, C_j) \quad \text{Grado di separazione di } C$$

Dato un insieme U con funzione d e un intero k , definiamo il problema del k -clustering a separazione massima per (U, d) il problema di trovare un k -clustering C di U tale che:

$$sep(C) \geq sep(C')$$

Per ogni altro k -clustering C' di U .

7.7.1 Algoritmo di Kruskal Clustering

Dopo avere effettuato opportune modifiche, tramite l'algoritmo di Kruskal è possibile risolvere il problema del clustering a separazione massima.

Clustering := $\{\{p_1\}, \{p_2\}, \dots, \{p_m\}\}$
- si consideri il grafo completo G su U
- seguendo un ordinamento non-decrescente degli archi di G
(rispetto alla distanza d) e sino a quando $|Clustering| = k$
si eseguono le istruzioni:
** sia (p_i, p_j) l'arco di G da trattare*
** se p_i e p_j appartengono a cluster C, C' diversi,*
si uniscano C e C' in un unico cluster
return Clustering

In pratica, le modifiche che abbiamo effettuato sono:

- L'algoritmo si ferma quando il numero di alberi della foresta è k e non 1.
- L'algoritmo restituisce le componenti connesse della foresta trovata.

La complessità di questo algoritmo è $O(n^2 \log(n))$. Possiamo osservare che la separazione del cluster così calcolato è maggiore o uguale al peso $d(p_i, p_j)$ dell'arco (p_i, p_j) che l'algoritmo avrebbe preso in considerazione se non si fosse fermato. Ciò è dovuto al fatto che l'algoritmo si è fermato prima di aggiungere quest'ultimo arco e, quindi, tutti gli archi prima di lui sono più corti. Consideriamo adesso due punti p e q che sono in cluster diversi e che definiscono la separazione, l'arco tra di loro non può essere già stato processato, altrimenti p e q sarebbero nello stesso cluster e, se questo arco non è ancora stato processato, deve essere perché la sua lunghezza è almeno uguale a quella del prossimo arco in lista. Dunque, deve per forza essere che $sep(C) \geq d(p_i, p_j)$.

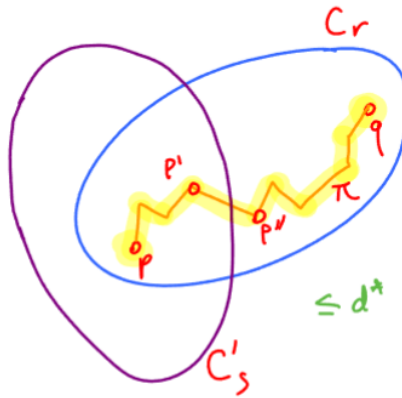
Correttezza dell'algoritmo

Sia $C^* = \{C_1, \dots, C_k\}$ il k -clustering costruito dall'algoritmo. Come osservato prima $sep(C^*) \geq d^*$ dove d^* è il peso dell'arco che sarebbe stato preso in considerazione all'iterazione successiva. Sia $C' = \{C'_1, \dots, C'_2\}$ un qualunque altro clustering

distinto da C^* , dobbiamo verificare che $\text{sep}(C') \leq \text{sep}(C^*)$. Dato che $C^* \neq C'$, almeno un cluster di C^* ne interseca due distinti di C' . Sia $C_r \in C^*$ tale che C_r interseca due cluster distinti di C' :

- Sia $p \in C_r$ e sia $C'_s \in C'$ tale che $p \in C'_s$. Cioè p è l'elemento contenuto in entrambi.
- Sia $q \in C_r$ tale che $q \notin C'_s$. Cioè q è contenuto solo in C_r .

Poiché p, q stanno nello stesso cluster C_r , essi sono connessi nel grafo da un cammino π i cui archi hanno peso $\leq d^*$. Quindi su π vi sono due nodi contigui p', p'' tali che $p' \in C'_s, p'' \notin C'_s$, ma $d(p', p'') \leq d^*$ e, quindi, $\text{sep}(C') \leq d^* \leq \text{sep}(C^*)$ da cui la tesi.

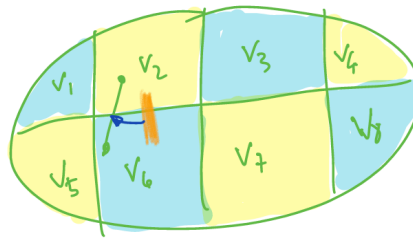


7.7.2 Minimum Spanning Tree in Grafi Sparsi

Definiamo una **contrazione** di un grafo **non orientato** $G = (V, E)$ un grafo $G' = (V', E')$ tale che:

- $V' = \{V_1, V_2, \dots, V_k\}$ è una partizione di V .⁵
- Ogni sottografo di $G = (V, E)$ indotto dagli insiemi di vertici $V_1, V_2, \dots, V_k$ è connesso.⁶
- Per ogni coppia $v_i, v_j \in V'$ con $i \neq j$ si ha che:⁷

$$(v_i, v_j) \in E' \leftrightarrow (\exists v_i \in V_i, v_j \in V_j) : (v_i, v_j) \in E$$



⁵Raggruppiamo i nodi in gruppi.

⁶In ogni gruppo i nodi devono essere connessi.

⁷Ogni gruppo è connesso se esiste un arco da un nodo di uno a uno dell'altro.

Dato un grafo non orientato e connesso $G = (V, E)$ con funzione peso $w : E \rightarrow R$, descriveremo una procedura **MST-Reduce** che costruisce:

- Un grafo **contratto** $G' = (V', E')$ di $G = (V, E)$ con funzione peso $w' : E' \rightarrow R$ dove V' è una partizione di V tale che $|V'| \leq |V|/2$.⁸

- Una **funzione** $orig' : E' \rightarrow E$ tale che:⁹

$$(u', v') \in E', orig'(u', v') = (u, v) \rightarrow (u \in u', v \in v', w'(u', v') = w(u, v))$$

- Un insieme di archi $T \subseteq E$ tale che:

$$(\forall (u_1, u_2) \in T)(\exists V_i \in V') : u_1, u_2 \in V_i$$

tali che per ogni **MST** a di (G', w') :¹⁰

$$T \cup orig'[a] \text{ è un MST di } (G, w)$$

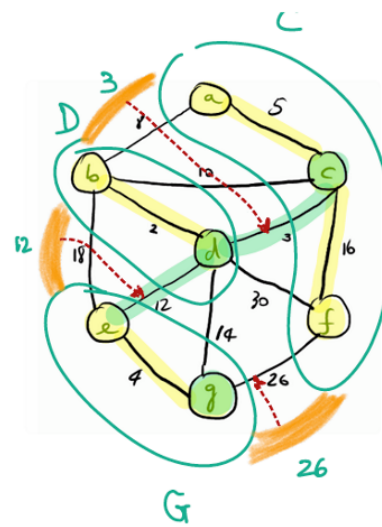
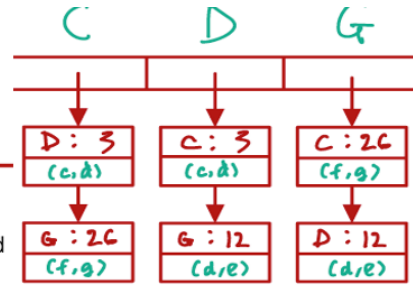
Gli archi di T che si trovano "dentro" il gruppo V_i formano l'Albero di Copertura Minimo (MST) del sottografo indotto dai vertici di V_i .

MST-REDUCE($G, orig, w, T$)

```

1 for each  $v \in V[G]$  do
2    $mark[v] \leftarrow FALSE$ 
3   MAKE-SET( $v$ )
4 for each  $u \in V[G]$  do
5   if  $mark[u] = FALSE$  then
6     choose  $v \in Adj[u]$  such that  $w[u, v]$  is minimized
7     UNION( $u, v$ )
8      $T \leftarrow T \cup \{orig[u, v]\}$ 
9      $mark[u] \leftarrow mark[v] \leftarrow TRUE$ 
10  $V[G'] \leftarrow \{FIND-SET(v) : v \in V[G]\}$ 
11  $E[G'] \leftarrow \emptyset$ 
12 for each  $(x, y) \in E[G]$  do
13    $u \leftarrow FIND-SET(x)$ 
14    $v \leftarrow FIND-SET(y)$ 
15   if  $u \neq v$  then
16     if  $(u, v) \notin E[G']$  then
17        $E[G'] \leftarrow E[G'] \cup \{(u, v)\}$ 
18        $orig[u, v] \leftarrow orig[x, y]$ 
19        $w[u, v] \leftarrow w[x, y]$ 
20     else if  $w[x, y] < w[u, v]$  then
21        $orig[u, v] \leftarrow orig[x, y]$ 
22        $w[u, v] \leftarrow w[x, y]$ 
23 construct adjacency lists  $Adj$  for  $G'$ 
24 return  $G', orig', w$ , and  $T$ 

```



L'algoritmo si compone di varie fasi:

⁸Ogni partizione ha al più la metà dei nodi originali

⁹Ogni arco nella che collega due partizioni, è un arco reale del grafo iniziale.

¹⁰Gli archi in T sono quelli che sono finiti dentro i supernodi. Siccome li abbiamo usati per la contrazione, siamo sicuri che facciano parte della soluzione ottimale

1. All'inizio ogni nodo è da solo (interpretato come insieme a se' stante) e viene marcato come FALSE.
2. L'algoritmo scorre tutti i nodi. Se trova un nodo u che non è ancora stato toccato (cioè marcato con FALSE) allora si cerca l'arco più leggero che parte da u e arriva a v e si fondono i due gruppi e marca u e v come true.
3. I nuovi nodi non sono altro che i "rappresentanti" (trovati con FIND-SET) dei gruppi che abbiamo creato prima. Se abbiamo unito A, B, C , il nuovo nodo sarà il nome del rappresentante di quel gruppo.
4. L'algoritmo scorre tutti gli archi originali (x, y) del grafo vecchio e per ogni arco, guarda a quale "regione" appartengono gli estremi e se la regione di partenza è diversa, controlliamo due casi:
 - Caso 1: Se non c'è ancora un arco tra la Regione u e la Regione v nel nuovo grafo, lo creiamo usando il peso dell'arco (x, y) .
 - Caso 2: Se esiste già un arco tra u e v , controlliamo il peso e teniamo solo quello minore.
5. Infine, costruiamo la lista di adiacenza per G' .

Capitolo 8

Cammini Minimi in un Grafo

Il problema dei cammini minimi su grafi è uno dei problemi più importanti e studiati al mondo e trova importanti applicazioni in vari campi come le reti di telecomunicazioni ecc.

Richiamiamo alcune definizioni:

- Sia V un insieme di nodi
- Sia $E \subset V \times V : (v, v) \notin E : \forall v \in V$.
- La coppia $G = (V, E)$ è un grafo orientato con V l'insieme dei nodi ed E l'insieme degli archi.
- Sia $w : E \rightarrow R$ una funzione peso o costo, (G, w) si dice grafo pesato.

Dato $G = (V, E)$ con $V = \{1, \dots, n\}$ la **matrice di adiacenza** associata al grafo è la matrice A di dimensione $n \times n$ tale che A_{ij} è 1 se $(i, j) \in E$ o 0 altrimenti.

Dato $G = (V, E)$ le **liste di adiacenze** associate al grafo sono gli insieme $Adj_g(u) = \{v \in V : (u, v) \in E\}$.

Sia $G = (V, E)$ un grafo con una funzione peso $w : E \rightarrow R$, siano $u, v \in V$ si dice **cammino da u a v** è una sequenza $\pi = (v_0, \dots, v_k)$ di nodi contigui di G tale che:

- $v_0 = u, v_k = v$.
- $(v_{i-1}, v_i) \in E$ per $i = 1, \dots, k$

Poniamo $w(\pi) = \sum_{i=1}^k w(v_{i-1}, v_i)$ con k la lunghezza del cammino, v_0 la testa e v_k la coda.

Siano $\pi_1 = (v_0, \dots, v_k)$ e $\pi_2 = (v_k, \dots, v_l)$ due cammini, scriviamo $\pi_1; \pi_2$ per indicare la **concatenazione di π_1 e π_2** cioè il cammino che è la somma dei due cammini sia in termini di percorso che in termini di peso.

Poniamo $\delta(u, v) = \min\{w(\pi) : \pi \in path(G, u, v)\}$. Sia un cammino $\pi \in paths(G; u, v)$ è **minimo** se $w(\pi) = \delta(u, v)$. Da questo possiamo definire vari problemi:

1. Dato un grafo pesato calcolare $\delta(u, v)$.
2. Dato un grafo pesato e dato un vertice appartenente al grafo detto **sorgente**, calcolare $\delta(s, v)$ per ogni $v \in V$.

3. Dato un grafo pesato e dato un vertice detto **destinazione**, calcolare $\delta(v, t)$ per ogni $v \in V$.
4. Dato un grafo pesato calcolare $\delta(u, v)$ per ogni vertice $u, v \in V$.

Un grafo **ammette cammini minimi da s** se $\delta(s, u) \neq -\infty$ per ogni vertice $u \in V$. Un grafo **ammette cammini minimi** se $\delta(u, v) \neq -\infty$ per ogni vertice $u, v \in V$.

Diciamo che un grafo ammette cammini minimi se e solo se ammette cammini minimi da ciascuno dei suoi nodi.

Una condizione necessaria e sufficiente perché un grafo pesato ammetta cammini minimi da una sorgente s è che **non vi sia alcun ciclo di peso negativo raggiungibile da s**.

Condizione necessaria e sufficiente perché un **grafo ammetta cammini minimi** è che non contenga archi di peso negativo. Segue che se la funzione peso non è negativa allora ammette sempre cammini minimi. Allo stesso modo se un grafo è aciclico ammette sempre cammini minimi.

Sia $\pi = (v_0, \dots, v_k)$ un cammino minimo in $G = (V, E)$ e $w : E \rightarrow R$, allora il **sottocammino** $\pi_{ij} = v_i, \dots, v_j$ per ogni $0 \leq i \leq j \leq k$ è minimo. Si può dimostrare banalmente perché se esistesse un altro cammino minimo da i a j , allora quello di partenza non sarebbe più minimo.

Per cercare di memorizzare tutti i cammini minimi a singola sorgente (anziché andare a memorizzare tutto in delle liste), possiamo utilizzare **un grafo dei cammini minimi** che non è altro che un grafo composto da tutti gli archi che compongono i cammini da s a tutti i vertici.

Sia $\min_paths(G; s) \neq \emptyset$ allora $paths(G'_s; s) = \min_paths(G; s)$. Dobbiamo dimostrare la doppia inclusione:

- CASO $\min_paths \subseteq paths$: Dalla definizione di G'_s segue immediatamente che $\min_paths \subseteq paths$.
- CASO $paths(G'_s; s) \subseteq \min_paths$ Immaginiamo che esista almeno un percorso π dentro G'_s che però NON è un cammino minimo nel grafo originale ($\pi \notin \min_paths(G; s)$). Tra tutti questi ipotetici percorsi "sbagliati", prendiamo quello con il **minor numero di archi**. Poiché π è il "primo" percorso sbagliato se togliamo l'ultimo pezzo, quello che rimane deve essere giusto. Quindi il peso di questo pezzo è esattamente la distanza minima: $w(\pi') = \delta(s, v_{k-1})$. Il peso di tutto il percorso π è la somma del prefisso ottimo + l'ultimo arco:

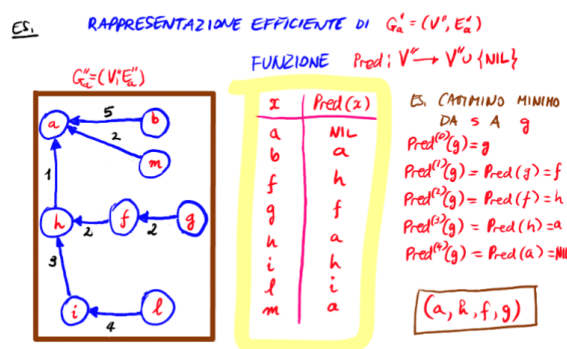
$$w(\pi) = w(\pi') + w(v_{k-1}, v_k) = \delta(s, v_{k-1}) + w(v_{k-1}, v_k)$$

Consideriamo un vero cammino $\bar{\pi}$ che passa per quell'ultimo arco, sappiamo che per le proprietà dei cammini minimi la distanza vera per arrivare a v_k , cioè $\delta(s, v_k)$, è uguale alla distanza per arrivare a v_{k-1} più il peso dell'arco:

$$\delta(s, v_k) = \delta(s, v_{k-1}) + w(v_{k-1}, v_k)$$

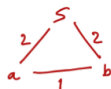
Ma qui arriviamo a un assurdo poiché abbiamo che $w(\pi) = \delta(s, v_{k-1}) + w(v_{k-1}, v_k)$ e che $\delta(s, v_k) = \delta(s, v_{k-1}) + w(v_{k-1}, v_k)$. che dimostra che $w(\pi) = \delta(s, v_k)$ che contraddice l'ipotesi che π non è minimo.

Sia (G, w) un grafo pesato e sia $s \in V$ una sorgente. Supponiamo che il grafo ammetta cammini minimi da s , allora esiste un **albero di cammini minimi** $G_s'' = (V'', E_s'') : V'' \subseteq V, E_s'' \subseteq E$ radicato in s contenente esattamente un cammino minimo da s in ciascun nodo raggiungibile da esso. Per costruirlo, ci basta fare una visita BFS o DFS da s nel grafo dei cammini minimi. Per rappresentare in maniera efficiente questo albero, utilizziamo la funzione $pred$ per cui ogni nodo ha un riferimento al precedente verso la radice.



Dato un grafo non orientato $G = (V, E)$ con funzione peso $w : E \rightarrow R$ e sorgente $s \in V$, **non è necessariamente vero** che tra i suoi alberi di cammini minimi da s ci debba essere un mst.

$$w(s, a) = 2, \quad w(s, b) = 2, \quad w(a, b) = 1.$$



C'È UN UNICO ALBERO DI CAMMINI MINIMI (V, T) DA s PER (G, w) , CON $T = \{(s, a), (s, b)\}$



M $\&$ (V, T) NON È UN MST PER (G, w) . ■

8.1 Algoritmi per Cammini Minimi da Una Sorgente

Descriveremo algoritmi basati su **tecniche di Label Correctin/Setting**. In pratica, dato un grafo $G = (V, E)$ con funzione peso $w : E \rightarrow R$ e sorgente $s \in V$, viene mantenuta una funzione $d : V \rightarrow R \cup \{\inf\}$ che sarà la nostra **stima della distanza** e faremo in modo tale che valga sempre che $d \geq \delta_{G_s}$ e che alla fine della computazione valga $d = \delta_{G_s}$.

Possiamo iniziare a impostare le distanze come segue:

$$\begin{aligned} d[s] &= 0; \\ d[v] &= \inf; \forall v \in V/\{s\} \end{aligned}$$

L'albero dei cammini minimi sarà rappresentato da un array di predecessori costruito insieme al calcolo della distanza δ_{G_s} . Per farlo, basterà inizialmente porre il predecessore di ogni nodo a *null*.

```

procedure Initialize-Single-Source( $G, s$ )
  for  $v \in V[G]$  do
     $d[v] := +\infty$ 
     $Pred[v] := NIL$ 
   $d[s] := 0$ 

```

Per aggiornare queste due funzioni, utilizzeremo la così detta procedura di **Relax** che sarà alla base di molti algoritmi che vedremo. In pratica, se la distanza $d[v]$ è maggiore rispetto alla distanza $d[u] + w(u, v)$, allora abbiamo trovato un cammino migliore che va dalla sorgente a v che passa prima da u .

```

Procedure RELAX( $u, v; w$ )
  if  $d[v] > d[u] + w(u, v)$  then
     $d[v] := d[u] + w(u, v)$ 
     $Pred[v] := u$ 

```

Da questo, segue l'algoritmo **Generic Single-Source Shortest-Path (GSSSP)**:

```

Procedure GSSSP( $G, s, w$ )
  Initialize-Single-Source( $G, s$ )
  while  $\exists (u, v) \in E[G]$  TALE CHE  $d[u] + w(u, v) < d[v]$  do
    - sia  $(u, v)$  tale che  $d[u] + w(u, v) < d[v]$ 
    RELAX( $u, v; w$ )

```

Durante l'esecuzione di GSSSP, varrà che:

- La funzione d decrescerà monotonicamente.
- Se $d[x] < \inf$, allora esiste un cammino da s a x di peso minore o uguale a $d[x]$.
- Vale sempre $\delta_{G_s} \leq d$, cioè, $\delta_{G_s}(x) \leq d[x]$.

Passiamo a dimostrare questi punti:

- La prima proprietà segue direttamente per induzione sul numero di esecuzione della procedura relax. A ogni relax, il valore d può solo diminuire.
- La seconda si dimostra per induzione:
 - **Caso Base:** Inizialmente abbiamo che $d[x] < \inf$ solo per $x = s$ e, in particolare $d[s] = 0$.
 - **Passo induttivo:** Sia x un nodo in G tale che $d[x] < \inf$ dopo una certa relax. Indicheremo d^- e d^+ il valore della funzione d immediatamente prima e immediatamente dopo. Abbiamo anche qui due casi:
 - * **Caso 1:** $d^-[x] = d^+$ cioè non cambia niente, poiché $d^-[x] < \inf$, per ipotesi induttiva esiste un cammino da s a x di peso $w(\pi_{sx}) \leq d^-[x]$.
 - * **Caso 2:** $d^+[x] < d^-[x]$ allora in questo caso avremo che $d^+[x] = d^-[u] + w(u, x) \leq \inf$. Per ipotesi induttiva, esiste un cammino reale π_{su} che va dalla sorgente s fino a u e il suo peso è $w(\pi_{su}) \leq d^-[u]$. Creiamo un nuovo cammino π_{sx} semplicemente "incollando" l'arco (u, x) alla fine del cammino π_{su} e questo, avrà peso:

$$w(\pi_{sx}) = w(\pi_{su}) + w(u, x) \leq d^-[u] + w(u, x) = d^+[x]$$

- Anche il terzo punto si può dimostrare per induzione:
 - **Caso Base:** Anche in questo caso, poiché $\delta_{G_s}(s) \leq 0$ e per ogni $x \in V$ vale che $\delta_{G_s}(x) \leq \inf = d[x]$, allora prima della procedura relax vale che $\delta_{G_s} \leq d$.
 - **Passo Induttivo:** Supponiamo che valga $\delta_{G_s} \leq d$ poco prima di relax $(u, v; w)$, sia d^- e d^+ come prima. Per ipotesi induttiva $\delta_{G_s} \leq d^-$ e $d^+[x] = d^-[x] \forall x \neq v$. Per il punto due esiste, esiste un cammino π_{sx} tale che $w(\pi_{sx}) \leq d^+[x]$ e, dunque, $\delta_{G_s}(x) \leq w(\pi_{sx}) \leq d^+[x]$. Quindi vale quanto detto.

Lemma:

Sia (G, w) tale che $G = (V, E)$ e $w : E \rightarrow R$ e sia $s \in V$. Sia $f : V \rightarrow R \cup \{\inf\}$ tale che $f \geq \delta_{G_s}$ e $f(s) \leq 0$, allora le seguenti condizioni sono equivalenti:

- (a): $(\forall (u, v) \in E), \quad f(v) \leq f(u) + w(u, v).$ ¹
- (b): $f = \delta_{G_s}.$ ²

Dimostriamolo:

1. Ipotesi per assurdo

Supponiamo per assurdo che valga la prima proprietà (a) ma che $f \neq \delta_{G_s}$.

¹Per ogni arco del grafo, la stima per arrivare a v è già migliore (o uguale) di quella che otterremmo passando per u

²La nostra funzione f coincide esattamente con le distanze minime vere. Abbiamo trovato la soluzione perfetta.

2. Definizione dell'insieme P_d

Consideriamo l'insieme dei cammini in cui la stima è incoerente (stima maggiore del peso reale):

$$P_d = \{\pi \in \text{paths}(G; s) : f(\text{tail}(\pi)) > w(\pi)\}$$

Osserviamo che la sorgente s non appartiene all'insieme ($s \notin P_d$), infatti:

$$f(\text{tail}(s)) = f(s) \leq 0 = w(s)$$

Poiché $f \neq \delta_{G_s}$ (e sapendo che vale sempre $f \geq \delta_{G_s}$), esiste almeno un nodo v tale che $f(v) > \delta_{G_s}(v)$. Dunque esiste un cammino π_{sv} tale che $w(\pi_{sv}) < f(v)$, il che implica che $\pi_{sv} \in P_d$. L'insieme non è vuoto.

3. Scelta del cammino minimo

Sia $\bar{\pi} \in P_d$ un cammino di **lunghezza minima**. Sia $k = \text{length}(\bar{\pi})$.

Poiché $s \notin P_d$, deve valere $k \geq 1$.

Sia $\bar{\pi} = (v_0, \dots, v_k)$ con $v_0 = s$. Poiché $\bar{\pi} \in P_d$, vale che:

$$f(v_k) = f(\text{tail}(\bar{\pi})) > w(\bar{\pi})$$

4. Analisi del prefisso

Consideriamo il cammino $\bar{\pi}'$ che coincide con $\bar{\pi}$ privato dell'ultimo arco (v_{k-1}, v_k) .

Per la **minimalità** di $\bar{\pi}$, si ha che $\bar{\pi}' \notin P_d$. Quindi la stima sul penultimo nodo è corretta:

$$f(v_{k-1}) = f(\text{tail}(\bar{\pi}')) \leq w(\bar{\pi}')$$

5. Contraddizione

Mettendo insieme le relazioni ottenute:

$$\begin{aligned} f(v_k) &> w(\bar{\pi}) && (\text{poiché } \bar{\pi} \in P_d) \\ &= w(\bar{\pi}') + w(v_{k-1}, v_k) && (\text{scomposizione del peso}) \\ &\geq f(v_{k-1}) + w(v_{k-1}, v_k) && (\text{poiché } \bar{\pi}' \notin P_d) \end{aligned}$$

Otteniamo quindi la disuguaglianza stretta:

$$f(v_k) > f(v_{k-1}) + w(v_{k-1}, v_k)$$

Ciò contraddice l'ipotesi iniziale (a) secondo cui $f(v) \leq f(u) + w(u, v)$ deve valere per ogni arco del grafo.

Dimostriamo adesso il viceversa.

1. Ipotesi e Assurdo

Supponiamo che valga (b), ovvero che $f = \delta_{G_s}$.

Supponiamo per assurdo che non valga (a), cioè che esista un arco $(u, v) \in E$ tale che:

$$f(v) > f(u) + w(u, v)$$

2. Esistenza del cammino ottimo verso u

Poiché $-\infty < \delta_{G_s}(u) = f(u) < +\infty$, esiste sicuramente un cammino minimo π_{su} da s a u tale che:

$$w(\pi_{su}) = \delta_{G_s}(u)$$

3. Costruzione del cammino verso v

Consideriamo il cammino π_{sv} ottenuto concatenando π_{su} con l'arco (u, v) :

$$\pi_{sv} = \pi_{su}; (u, v)$$

Il peso di questo cammino è:

$$w(\pi_{sv}) = w(\pi_{su}) + w(u, v)$$

4. Contraddizione

Riprendiamo la disuguaglianza dell'ipotesi assurda e sostituiamo i valori:

$$\begin{array}{ll} f(v) > f(u) + w(u, v) & \text{(Ipotesi assurda)} \\ \delta_{G_s}(v) > \delta_{G_s}(u) + w(u, v) & \text{(Poiché } f = \delta_{G_s} \text{)} \\ \delta_{G_s}(v) > w(\pi_{su}) + w(u, v) & \text{(Sostituzione peso cammino)} \\ \delta_{G_s}(v) > w(\pi_{sv}) & \text{(Definizione di } \pi_{sv} \text{)} \end{array}$$

Abbiamo ottenuto che $\delta_{G_s}(v) > w(\pi_{sv})$.

Questo è **assurdo**, perché per definizione $\delta_{G_s}(v)$ è il peso del cammino minimo da s a v , e non può essere strettamente maggiore del peso di un altro cammino esistente (π_{sv}).

Pertanto, l'ipotesi assurda cade e deve valere:

$$(\forall (u, v) \in E) \quad f(v) \leq f(u) + w(u, v)$$

Lemma:

Da quanto detto prima, una condizione necessaria e sufficiente perché l'algoritmo GSSSP termini è che $d = \delta_{G_s}$. Banalmente se GSSSP si ferma vale che $d[v] \leq d[u] + w(u, v)$ per tutti i nodi v e, per il lemma precedente, $d = \delta_{G_s}$. Allo stesso modo si può dimostrare poiché se vale che $d = \delta_{G_s}$ allora per il lemma precedente si ha $d[v] \leq d[u] + w(u, v)$, dunque termina GSSSP.

Teorema:

Supponiamo che (G, w) ammetta cammini minimi³ da una sorgente s . Siano $d : V \rightarrow R \cup \{\inf\}$ e $pred : V \rightarrow V \cup \{null\}$ le funzioni calcolate dopo una sequenza di relax. Definiamo:

- V_T : Sono i nodi che abbiamo raggiunto finora.
- E_T : Sono gli archi che collegano ogni nodo al suo padre.

Allora le seguenti proprietà valgono prima e dopo l'esecuzione di $relax(u, v; w)$:

³Cioè che non vi siano cicli di peso negativo

1. **T è un albero radicato in s :** Significa che seguendo i padri si arriva sempre alla sorgente s , senza girare in tondo.
2. **Coerenza delle distanze:** Se cammino dentro questo albero da un nodo x a un nodo y , la stima $d[y]$ è sempre maggiore o uguale alla stima $d[x]$ più il peso del cammino da x a y .

Dimostriamolo indicando con $d', pred'$ il nuovo valore della funzione d e $pred$ dopo la relax.

1. Se la relax non fa nulla, cioè $d' = d$ e $pred' = pred$, allora il teorema è banalmente dimostrato. Se invece abbiamo che:
 - $d'[v] = d[u] + w(u, v)$ e $d'[z] = d[z]$ per tutti gli altri nodi.
 - $pred'[v] = u$ e $pred'[z] = pred[z]$ per tutti gli altri nodi.

Il rischio è che T' non sia più un albero radicato in s , allora v sarebbe necessariamente su T_{su} e si avrebbe che:

$$\begin{aligned} d[v] + w(T_{vu}) &\leq d[u] \\ d[v] + w(T_{vu}) + w(u, v) &\leq d[u] + w(u, v) \\ w(T_{vu}) + w(u, v) &\leq d[u] + w(u, v) - d[v] < 0 \end{aligned}$$

Cioè $T_{vu}; (u, v)$ sarebbe un ciclo di peso negativo e ciò è un assurdo perché altrimenti il problema non ammetterebbe cammini minimi.

2. Dimostriamo adesso la seconda proprietà, cioè che $d'[x] + w(T'_{xy}) \leq d'[y]$. Per farlo, controlliamo le coppie x, y il cui percorso è cambiato e ciò accade solo se il percorso passa per il nuovo arco (u, v) . Immaginiamo di avere la seguente situazione:

$$x \rightarrow \dots \rightarrow u \rightarrow v \rightarrow \dots \rightarrow y$$

Possiamo scomporre il percorso da x a u , l'arco (u, v) e il percorso da v a y e vediamo cosa cambia. Notiamo che per $x \rightarrow u$ valeva già la vecchia proprietà, cioè $d[x] + w(T_{xu}) \leq d[u]$. Per quanto riguarda il nodo v , abbiamo appena aggiornato il valore: $d'[v] = d[u] + w(u, v)$. Infine, per il pezzo $v \rightarrow y$, le distanze relative nell'albero non sono peggiorate.

Corollario:

Se l'algoritmo GSSSP termina, allora abbiamo che:

- $d = \delta_{G_s}$: I valori numerici calcolati sono esatti.
- T è un Albero di Cammini Minimi: L'insieme dei puntatori ai "padri" ($Pred$) forma una struttura ad albero che contiene effettivamente i percorsi più brevi possibili dalla sorgente verso tutti gli altri nodi.

Si dimostra banalmente poiché, sia x un nodo raggiungibile da s e sia T_{sx} il cammino nell'albero T da s a x , allora per il lemma precedente abbiamo:

$$\delta_{G_s}(x) \leq w(T_{sx}) = d[s] + w(T_{sx}) \leq d[x] = \delta_{G_s}(x)$$

Da cui $w(T_{sx}) = \delta_{G_s}(x)$ cioè il cammino è minimo, dunque T è un albero di cammini minimi.

Lemma: Se (G, w) ammette cammini minimi, l'esecuzione di GSSSP termina dopo al massimo $\lfloor e(|V[G]| - 1)! \rfloor$ passi.

8.1.1 GSSSP Ottimizzato

Se $s \rightsquigarrow \dots \rightsquigarrow u \rightarrow v$ è un cammino minimo da s a v e a un certo punto dell'esecuzione vale che $d[u] = \delta_{G_s}(u)$, allora dopo l'esecuzione della relax dell'arco tra (u, v) per la proprietà di sottostruttura ottima avremo che $d[v] = \delta_{G_s}(v)$. Infatti, dopo la relax avremo che:

$$\delta_{G_s}(v) \leq d[v] \leq d[u] + w(u, v) = w(\pi_{su} + w(u, v)) = \delta_{G_s}(v)$$

Da cui segue che $d[v] = \delta_{G_s}(v)$. Dunque se riusciamo ad accertarci che $d[u] = \delta_{G_s}(u)$ per qualche nodo, converrà chiamare la relax su ogni nodo adiacente. Introduciamo quindi il seguente algoritmo:

```

Procedure SCAN(u; G, w)
  for v ∈ AdjG(u) do
    RELAX(u, v; w)

```

Da cui segue la versione ottimizzata di GSSSP che chiameremo OGSSSP:

```

Procedure OGSSSP(G, s, w)
  Initialize-Single-Source(G, s)
  S := ∅ // rappresenta l'insieme dei nodi x per cui e' stato
         // accertato che vale d[x] = δGs(x)
  while ∃ v ∈ V[G] \ S tale che d[v] > δGs(v) do
    SCAN(v; G, w)
    S := S ∪ {v}
  return d, Pred;

```

Teorema

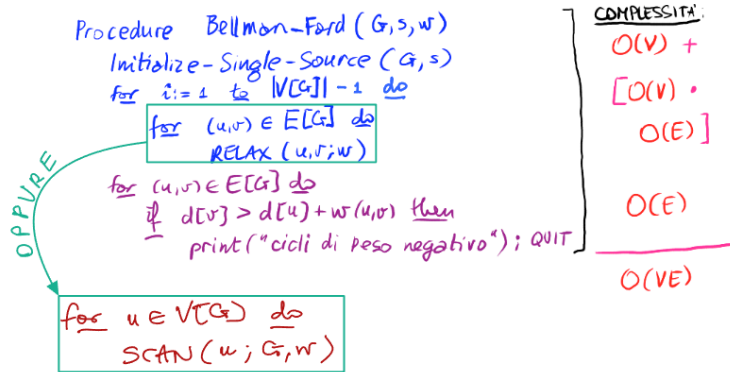
Se ci sono ancora **nodi da visitare** ($V \setminus S \neq \emptyset$), allora esiste sicuramente un nodo v tra questi per cui **la stima attuale è perfetta**: $d[v] = \delta_{G_s}(v)$.

Dimostriamolo:

Prendiamo un qualsiasi u non ancora visitato e immaginiamo un cammino minimo che vada dalla sorgente a u : $\pi = (u_0, u_1, \dots, u_k)$. Dato che s è stato visitato (è in S) e u no, seguendo questi archi arriveremo a un nodo che non è in S e questo lo indicheremo come u_{i_0} . Il predecessore era in S e, dunque era già ottimo, per cui, se rilassiamo l'arco tra u_{i_0} e il suo predecessore, allora anche u_{i_0} sarà ottimo. La complessità totale di questo algoritmo è $O(V + E)$ + Tempo per scegliere il nodo giusto cioè il tempo che ci metto a trovare $d[u] = \delta_{G_s}(u)$ e questo dipenderà dagli algoritmi che decidiamo.

8.1.2 Algoritmo di Bellman-Ford

L'algoritmo di Bellman-Ford non effettua alcun tipo di ottimizzazione nella scelta del "nodo giusto", per cui, quello che fa' è scegliere a ogni iterazione tutti gli archi del grafo e prova a rilassarli. Lo pseudocodice è il seguente:



Come possiamo vedere, le iterazioni del ciclo sono $|V| - 1$ poiché un cammino non può avere più di $|V| - 1$ nodi.

Un'altra cosa interessante è che questo algoritmo funziona anche nel caso di archi con cicli di peso negativo poiché se a una prossima iterazione, vi fosse un **arco che si può ancora rilassare, allora vuol dire che abbiamo archi di peso negativo**, dunque, l'algoritmo Bellman-Ford ce lo segnala.

La complessità è quindi $O(V)$ per l'inizializzazione, $O(V) \cdot O(E)$ per il ciclo annidato e, infine, $O(E)$ per l'ultimo controllo. In totale la complessità sarà $O(VE)$.

8.1.3 Algoritmo di Dijkstra

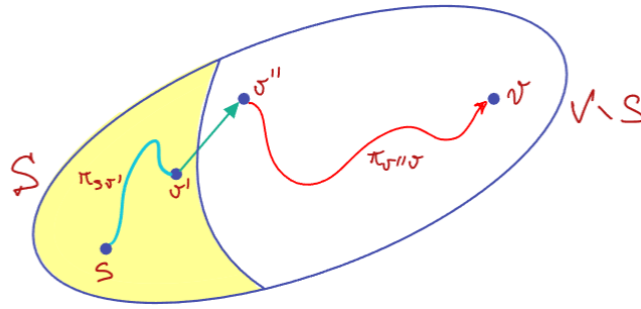
Sia $G = (V, E)$ un grafo privo di archi di peso negativo e sia $s \in V$ una sorgente assegnata. Quello che ci suggerisce di fare l'algoritmo OGSSSP è prendere un nodo per cui $d[v] = \delta_{G_s}(v)$ dai nodi non visitati. L'algoritmo propone di scegliere il nodo che ha il valore $d[v]$ minimo tra tutti i nodi non visitati. Consideriamo due casi:

- $s \notin S$: All'inizio, l'insieme dei visitati è vuoto. L'unico nodo con distanza finita è la sorgente s ($d[s] = 0$) che è il minimo tra i nodi non visitati.
- **Caso** $s \in S$, $v \in V \setminus S$ e $d[v] \neq \delta_{G_s}(v)$:

In questo caso, poiché la stima non è ottima, allora $\delta_{G_s}(v) < +\infty$ e dunque esiste un cammino minimo reale π_{sv} da s a v . Tale cammino si può scomporre come:

$$\pi_{sv} = \pi_{sv'}; (v', v''); \pi_{v''v}$$

dove l'arco $v' \rightarrow v''$ attraversa la frontiera tra l'insieme S e l'insieme $V \setminus S$.



Poiché il cammino $\pi_{sv''}; (v', v'')$ è parte di un cammino minimo, vale che $d[v'] = \delta_{G_s}(v')$. Dunque, chiamando $relax(v', v''; w)$, avremo che $d[v''] = \delta_{G_s}(v'')$ per quanto visto precedentemente. Seguendo questo ragionamento vediamo che:

$$\begin{aligned}
 d[v] &> \delta_{G_s}(v) = w(\pi_{sv}) \\
 &= w(\pi_{sv'}; (v', v'')) + w(\pi_{v''v}) \\
 &= \delta_{G_s}(v'') + w(\pi_{v''v}) \\
 &\geq \delta_{G_s}(v'') \quad (\text{poiché } w \geq 0) \\
 &= d[v''] \\
 &\geq \min\{d[u] : u \in V \setminus S\}
 \end{aligned}$$

Dunque, avremo che:

$$d[v] > \min\{d[u] : u \in V \setminus S\}$$

E, quindi, possiamo dire che per ogni nodo $\bar{v} \in V \setminus S$ tale che $d[\bar{v}] = \min\{d[u] : u \in V \setminus S\}$ (ovvero il nodo scelto dall'algoritmo) vale necessariamente che:

$$d[\bar{v}] = \delta_{G_s}(\bar{v})$$

Da ciò, segue l'algoritmo di Dijkstra:

<u>ALGORITMO DI DIJKSTRA</u>	
<pre> Procedure Dijkstra (G, s, w) Initialize-Single-Source (G, s) Q := make-queue (V[G], d) while Q ≠ ∅ do v := Extract-Min (Q, d) SCAN(v; G, w) </pre>	COMPLESSITÀ $\mathcal{O}(V) +$ $\mathcal{O}(V) +$ $[V \cdot \text{costo}(\text{Extract-Min}) +$ $\mathcal{O}(E) \cdot \text{costo}(\text{Decrease-Key})]$

La complessità dell'algoritmo può dunque variare in base a come implementiamo la coda Q :

ARRAY	HEAP BINARIO	HEAP "u" FIBONACCI
$O(V)$	$O(V)$	$O(V)$
$O(1)$	$O(V)$	$O(V)$
$O(V^2)$	$O(V \log V)$	$O(V \log V)$
$O(E)$	$O(E \log V)$	$O(E)$
$O(V^2)$	$O((V+E) \log V)$	$O(E \log V)$

8.1.4 Algoritmo DAG

Sia $G = (V, E)$ un grafo aciclico on $w : E \rightarrow R$ e sia $s \in V$ una sorgente assegnata. Sia $u \in V/S$ tale che $d[u] > \delta_{G_s}(u)$, allora $\delta_{G_s}(u) < \inf$ e dunque esiste un cammino minimo da s a u .

Sia u' il predecessore di u in π_{su} , poiché $d[u] > \delta_{G_s}(u)$, deve valere che $u' \in V/S$,⁴ pertanto, vale che u ha un predecessore $u' \in V/S$ e, quindi, per ogni nodo $v \in V/S$ i cui predecessori appartengono tutti ad S , vale che $d[v] = \delta_{G_s}(v)$. Cioè **se ho già visitato tutti i nodi che potrebbero "portare" a v (tutti i suoi potenziali padri), allora la stima di v non può più cambiare.**

Da queste considerazioni, possiamo descrivere questo algoritmo:

CAMMINI MINIMI DA UNA SORGENTE IN GRAFI ACICLICI

Procedure DAG-Shortest-Paths(G, s, w)

Initialize-Single-Source(G, s)

sia $\prec$ un ordinamento topologico di G

for $v \in V[G]$ seguendo l'ordinamento $\prec$ do

SCAN($v; G, w$)

COMPLESSITA'

] $O(V)$

] $O(V+E)$

] $O(V+E)$

$O(V+E)$

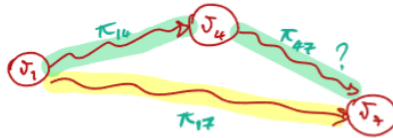
8.2 Cammini Minimi tra Tutte le Coppie

Da adesso sarà necessario considerare la seguente notazione:

- $d[v] \rightarrow d[s, v]$ stima della distanza da s a v .
- $pred[v] \rightarrow pred[s, v]$ predecessore di v in un cammino da s a v .

Questo è necessario poiché stiamo passando in un contesto con più sorgenti. Passiamo dunque da un'array a una matrice. Immaginiamo una situazione di questo tipo, siano v, v_4, v_7 dei nodi nel grafo come segue:

⁴Questa cosa deve essere vera poiché se u' fosse stato visitato, allora avremmo già effettuato la relax u', u .



Vediamo che se:

$$d[v_1, v_7] > d[v_1, v_4] + d[v_4, v_7]$$

Ci converrà porre:

$$d[v_1, v_7] = d[v_1, v_4] + d[v_4, v_7]$$

$$Pred[v_1, v_7] = Pred[v_4, v_7]$$

Da qui segue la seguente generalizzazione di *relax*:

```

G-RELAX (x, y, z; w)
  if d[x, z] > d[x, y] + d[y, z] then
    d[x, z] := d[x, y] + d[y, z]
    Pred[x, z] := Pred[y, z]
  
```

Anche la funzione di inizializzazione diventa:

```

Initialize-All-Pairs(G, w)
  for u in V[G] do
    for v in V[G] do
      if (u, v) in E[G] then
        d[u, v] := w(u, v)
        Pred[u, v] := u
      else
        d[u, v] := NIL
        if u != v then
          d[u, v] := +infinity
        else
          d[u, v] := 0
  
```

8.2.1 Bellman-Ford per Coppie

Una possibile soluzione si ottiene iterando l'algoritmo Bellman-Ford come segue:

```

G-BELLMAN-FORD (G, w)
  Initialize-All-Pairs(G, w)
  for s in V[G] do
    for i = 1 to |V[G]|-1 do
      for (u, v) in E[G] do
        G-RELAX (s, u, v)
  
```

Tale algoritmo ha complessità $O(V^2E)$ per grafi, cioè con $E = \Theta(V^2)$, tale complessità diventa $O(V^4)$. Questo algoritmo non è molto efficiente nel costruire cammini lunghi. Immaginiamo questa situazione:

$$v_1 \rightarrow v_2 \rightarrow v_3 \cdots \rightarrow v_7$$

Con l'approccio Bellman-ford abbiamo bisogno di calcolare prima $g_relax(v_1, v_2, v_3)$, poi $g_relax(v_1, v_3, v_4)$, poi $g_relax(v_1, v_4, v_5)$, poi $g_relax(v_1, v_5, v_6)$ e, infine, $g_relax(v_1, v_6, v_7)$. Ma tante altre combinazioni di g_relax sono possibili. Esiste un modo per ridurre il numero di chiamate g_relax ?

8.2.2 Algoritmo Floyd-Warshall

L'algoritmo in questione, si focalizza sui **vertici intermedi** di un cammino minimo che sarebbero i vertici che sono tra il vertice sorgente e quello di arrivo. Su un cammino semplice $p = (v_0, \dots, v_l)$ i vertici intermedi sono $(v_1, \dots, v_{l-1})$. Organizzeremo il lavoro in "blocchi" che chiameremo y_i e, in sostanza, quello che faremo a ogni iterazione è controllare se passare dal blocco y_i aiuta ad arrivare a destinazione. L'idea è che se seguiremo un ordine qualunque di rilassamento di questi blocchi, otterremo quello che ci serve poiché:

- Al passo k , calcoliamo i cammini minimi tra tutte le coppie (i, j) potendo usare come nodi intermedi (ponti) solo i nodi che appartengono all'insieme $\{y_1, y_2, \dots, y_k\}$.
- Al passo $k + 1$, aggiungiamo il nodo y_{k+1} all'insieme dei nodi intermedi utilizzabili e vediamo se questo nuovo "ponte" ci permette di accorciare qualche percorso.

Alla fine di questa procedura avremo considerato tutti i nodi come intermediari piuttosto che andare a confrontare tutti i nodi come a casaccio e non è necessario che li analizziamo secondo un particolare ordine.

ALGORITMO DI FLOYD-WARSHALL

```

Floyd-Warshall ( $G, w$ )
  Initialize-All-Pairs( $G, w$ )
  for  $y \in V(G)$  do
    for  $x \in V(G)$  do
      for  $z \in V(G)$  do
         $G\_RELAX(x, y, z)$ 
  
```

$\text{if } d[x, z] > d[x, y] + d[y, z] \text{ then}$
 $d[x, z] := d[x, y] + d[y, z]$
 $Pred[x, z] := Pred[y, z]$

COMPLESSITÀ: $O(V^3)$

Dimostriamo che questo algoritmo funziona. Sia $v_1, \dots, v_n$ l'ordine in cui gli elementi $y \in V$ vengono processati nel ciclo for più esterno. Per $1 \leq i \leq n$ e $u, v \in V$ poniamo $\delta^i(u, v)$ come il peso di un cammino semplice da u a v i cui nodi interni sono $\{v_1, \dots, v_i\}$. Possiamo dimostrare per induzione che dopo l' i -esima prova del ciclo for più esterno valgono:

- $d[u, v]$ peso di un cammino π da u a v i cui nodi interni sono in $\{v_1, \dots, v_i\}$ e, tale che $w(\pi) \leq \delta^i(u, v)$.
- $pred[u, v]$ predecessore di v in un cammino π da u a v i cui nodi interni sono in $\{v_1, \dots, v_i\}$ e, tale che $w(\pi) = d[u, v]$

Per tanto, se il grafo iniziale ammette cammini minimi, allora varrà che:

- $d[u, v] = \delta_{G,w}(u, v)$ per ogni coppia u, v
- $pred[u, v]$ predecessore di v in un cammino semplice da u a v .

In sostanza, avremo che $d[u, u] = 0$ per ogni nodo e un cammino da u a v sarà dato dalla successione di predecessori di v . Esiste questo algoritmo per trovare il cammino:

```

Build_Min_Path (u, v; G, Pred)
  if  $u \neq v$  and  $Pred[u, v] = nil$  then
    return "v non e' RAGGIUNGIBILE DA u"
  if  $u = v$  then
    return <u>
  else
    return Append( Build_Min_Path (u, Pred[u, v]; G, Pred), v )

```

Nel caso in cui il grafo non ammette cammini minimi, alla fine dell'esecuzione di floyd warshall si ha che:

$$d[u, v] \leq \delta^n(v, v) \leq w(\pi) < 0$$

Cioè avremo una condizione **necessaria e sufficiente perché ammetta cammini minimi** è che alla fine dell'esecuzione di floyd warshall si abbia che $d[u, u] = 0$ per ogni vertice.

Immaginiamo di voler costruire un nuovo grafo G^* (o una matrice) che ha un arco tra u e v se e solo se esiste un modo per raggiungere v partendo da u nel grafo originale. Per fare ciò, possiamo modificare la funzione peso in maniera tale che ogni arco abbia costo 1 e poi lanciando Floyd Warshall notiamo che se v è raggiungibile da u , allora $d[u, v] \neq \text{inf}$ altrimenti sarà inf . Esiste una versione ottimizzata di Floyd Warshall per fare questa cosa che sfrutta gli and e gli or logici per ottimizzare il tempo di calcolo sulla CPU.

8.3 Rompere la barriera $O(V^3)$

Con l'algoritmo Floyd Warshall arriviamo a una complessità di $O(V^3)$ e adesso ci domandiamo, possiamo fare di meglio?

Nel caso in cui non vi siano archi di peso negativo, possiamo utilizzare Dijkstra V volte che, implementandolo con gli **Heap di Fibonacci** ha complessità di $O(V \log V + E)$ e, quindi, per V iterazione abbiamo un costo totale di $O(V^2 \log V + VE)$. Il nostro nuovo algoritmo è asintoticamente migliore di V^3 se e solo se il grafo è sparso $E = o(V^2)$

$$V^2 \log V + VE = o(V^3)$$

Poiché:

$$\lim_{V \rightarrow \infty} \frac{V^2 \log V + VE}{V^3} = \lim_{V \rightarrow \infty} \frac{\log V}{V} + \lim_{V \rightarrow \infty} \frac{E}{V^2} = 0$$

8.3.1 Algoritmo di Johnson per Grafi Sparsi

Questo algoritmo funziona anche con archi di peso negativo e sfrutta Dijkstra e Bellman-Ford come subroutines per ottenere una complessità di $O(V^2 \log V + VE)$. Per fare ciò, l'algoritmo usa la tecnica del **riassegnamento dei pesi** $w \rightarrow \hat{w}$. Poiché Dijkstra si "rompe" se vede numeri negativi, Johnson trasforma il grafo originale (G, w) in un nuovo grafo $(G, \hat{w})$ dove tutti i pesi sono non-negativi ($\hat{w} \geq 0$). È necessario che questa modifica venga effettuata in maniera tale che per ogni coppia (u, v) il cammino π_{uv} sia minimo in entrambi.

Lemma:

Sia $G = (V, E)$ un grafo con funzione peso $w : E \rightarrow R$ e sia $h : V \rightarrow R$ una funzione qualunque, si ponga:

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v)$$

Per ogni $(u, v) \in E$. Allora per ogni cammino in G avremo che π è un cammino minimo in (G, w) se e solo se π è un cammino minimo in $(G, \hat{w})$. Inoltre, (G, w) ammette cammini minimi se e solo se $(G, \hat{w})$ li ammette.

Dimostrazione:

Sia $\pi = (v_0, \dots, v_k)$ un cammino in G , allora:

$$\hat{w}(\pi) = w(\pi) + h(v_0) - h(v_k)$$

Questo deriva da:

$$\begin{aligned} \hat{w}(\pi) &= \sum_{i=1}^k \hat{w}(v_{i-1}, v_i) \\ &= \sum_{i=1}^k (w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)) \quad \text{SOMMATORIA TELESCOPICA} \\ &= \sum_{i=1}^k w(v_{i-1}, v_i) + \sum_{i=1}^k (h(v_{i-1}) - h(v_i)) \\ &= \sum_{i=1}^k w(v_{i-1}, v_i) + h(v_0) - h(v_k) \\ &= w(\pi) + h(v_0) - h(v_k). \end{aligned}$$

Dato che $h(\text{Partenza})$ e $h(\text{Arrivo})$ sono costanti fisse per quel viaggio, se il cammino A era più breve del cammino B nel vecchio sistema, lo sarà anche nel nuovo.

Sia π un cammino minimo in $(G, \hat{w})$ da u a v . Se π non fosse minimo in (G, w) , allora esisterebbe un cammino π' da u a v tale che $w(\pi') < w(\pi)$ ma

$$\hat{w} = w(\pi') + h(u) - h(v) < w(\pi) + h(u) - h(v) = \hat{w}(\pi)$$

Contraddicendo la minimalità di π in $(G, \hat{w})$, pertanto π è minimo in (G, w) . Allo stesso modo, se π è minimo in (G, w) lo sarà anche in $(G, \hat{w})$. Inoltre, se $\pi = (v_0, v_k)$ è un ciclo con $v_0 = v_k$, allora $\hat{\pi} = w(\pi) + h(v_0) - h(v_k) = w(\pi)$. Di conseguenza, (G, w) ammette cicli se e solo se li ammette anche $(G, \hat{w})$.

Vediamo adesso **come scegliere** $h : V \rightarrow R$. Vogliamo che i nuovi pesi siano tutti

non negativi ($\hat{w} \geq 0$) per poter usare Dijkstra. Dalla formula vista prima, questo significa:

$$w(u, v) + h(u) - h(v) \geq 0 \quad \Rightarrow \quad h(v) - h(u) \leq w(u, v)$$

Estendiamo, dunque, G con un nuovo nodo s e con archi (s, v) per ogni $v \in V$ con $w(s, v) = 0$. Supponiamo che (G', w) ammetta cammini minimi da s , si ponga $h(v) = \delta_{(G', w)}(s, v)$ ⁵ per ogni $v \in V$. Allora, per ogni arco $(u, v) \in E$ se π_{su} è un cammino minimo da s a u si ha che:

$$\begin{aligned} \delta_{(G', w)}(s, u) + w(u, v) &= w(\pi_{su}) + w(u, v) \\ &= w(\pi_{su}; (u, v)) \geq \delta_{(G', w)}(s, v) \end{aligned}$$

QUINDI:

$$\hat{w}(u, v) = w(u, v) + \delta_{(G', w)}(s, u) - \delta_{(G', w)}(s, v) \geq 0$$

Vediamo adesso il codice:

```

JOHNSON( $G, w$ )
1. compute  $G'$ , where  $V[G'] = V[G] \cup \{s\}$ ,
    $E[G'] = E[G] \cup \{(s, v) : v \in V[G]\}$ , and
    $w(s, v) = 0$  for all  $v \in V[G]$ 
2. if BELLMAN-FORD( $G', w, s$ ) = FALSE then
3.   print "the input graph contains a negative-weight cycle"
4. else
5.   for each vertex  $v \in V[G]$  do
     set  $h(v)$  to the value of  $\delta(s, v)$  computed by the
     Bellman-Ford algorithm
6.   for each edge  $(u, v) \in E[G]$  do
7.      $\hat{w}(u, v) := w(u, v) + h(u) - h(v)$ 
8.   let  $D = (d_{uv})$  and  $\Pi = (\pi_{uv})$  be two new  $n \times n$  matrix
9.   for each vertex  $u \in V[G]$  do
10.    run DIJKSTRA( $G, \hat{w}, u$ ) to compute
        $\hat{\delta}(u, v)$  and  $\text{Pred}_u[v]$  for all  $v \in V[G]$ 
11.    for each vertex  $v \in V[G]$  do
12.       $d_{uv} := \hat{\delta}(u, v) - h(u) + h(v)$ 
13.       $\pi_{uv} := \text{Pred}_u[v]$ 
14.   return  $D, \Pi$ 

```

$O(V)$ [lines 1-4]
 $O(VE + V^2)$ [lines 2-4]
 $O(V)$ [line 5]
 $O(E)$ [lines 6-8]
 $V(V \log V + E)$
 V^2 [lines 9-13]
 $E = o(V^2) \rightarrow V^2 \log V + VE = o(V^3)$ $O(V^2 \log V + VE)$

⁵Questo valore sarà procurato dall'algoritmo di Bellman-Ford. Probabilmente, la distanza minima δ sarà negativa a causa degli archi di peso negativo.

Capitolo 9

Reti di Flusso

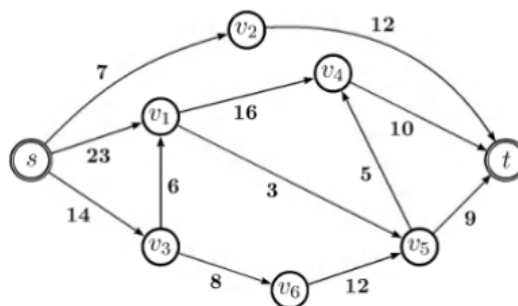
Una rete è un grafo pesato a cui nodi e archi sono associati dei valori detti **pesi**. In una rete, gli archi sono dei canali dentro cui vi fluiscono dei beni che possono essere rappresentate per mezzo di grandezze **discrete** o **continue** e possono rappresentare dei valori **assoluti** o dei valori **relativi** come, ad esempio, per unità di tempo. In questo contesto i pesi degli archi rappresentano la **capacità e dei costi** mentre i pesi sui nodi possono rappresentare la quantità di beni che entrano in quei nodi o che ne escono.

Formalmente una rete di flusso è definita come un grafo orientato $G = (V, E)$ senza cappi e tale che:

- A ogni arco $(u, v) \in E$ è associato un valore non negativo, $c(u, v) \geq 0$, detto **capacità** dell'arco. Se l'arco (u, v) non è presente in E allora assumiamo che esso abbia capacità nulla, cioè $c(u, v) = 0$.
- Nella rete di flusso ci sono due vertici speciali: il vertice **sorgente**, indicato con il simbolo s , e il vertice **pozzo** (o **destinazione**), indicato con il simbolo t .

I rimanenti vertici sono detti **vertici di transizione**.

- Ogni vertice giace su qualche cammino dalla sorgente al pozzo. Il grafo è quindi connesso e pertanto vale $|E| \geq |V| - 1$.



Data una rete $G = (V, E)$ con capacità $c : V \times V \rightarrow R_0^+$ un **flusso reale** in G è una funzione $\varphi : V \times V \rightarrow R_0^+$ soddisfa i seguenti requisiti:

- **Vincolo di Capacità:** per ogni arco $(u, v) \in V \times V$ si richiede che $\varphi(u, v) \leq c(u, v)$ che ci dice che non possiamo fare passare più risorse di quanta sia la capacità dell'arco.
- **Conservazione del flusso (legge di Kirchhoff):** Per tutti gli archi $u \in V - \{s, t\}$ si richiede che:

$$\sum \varphi(u, v) \text{ flusso uscente} = \sum \varphi(v, u) \text{ flusso entrante}$$

Cioè, tutto quello che entra in un nodo intermedio deve anche uscire. Questa regola vale, naturalmente, per tutti i nodi tranne che per la sorgente e la destinazione.

Chiameremo **flusso reale** la quantità $\varphi(u, v)$ lungo l'arco (u, v) . Definiamo il **Valore del flusso reale** come:

$$|\varphi| = \sum_{v \in V} \varphi(s, v) - \sum_{v \in V} \varphi(v, s)$$

Il flusso totale della rete è calcolato guardando solo la Sorgente (s). È la somma di tutto ciò che esce da s meno tutto ciò che rientra in s .

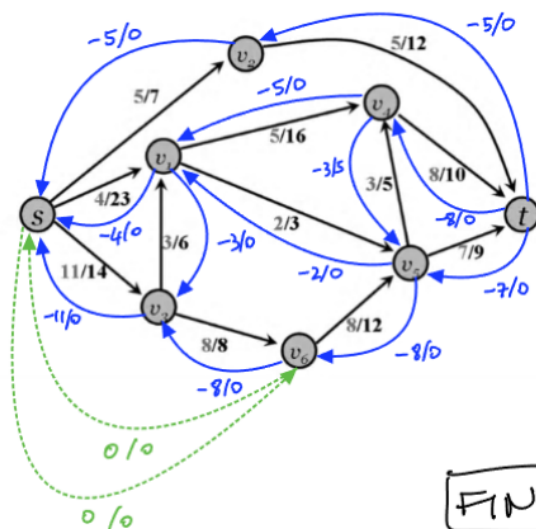
9.1 Problema del Massimo Flusso

Definiamo il **problema del massimo flusso** in una rete di flusso come la determinazione di un flusso φ in G di valore massimo. Distinguiamo adesso due casi:

- **Flusso Massimale:** Significa che, mantenendo fissi i percorsi che abbiamo già scelto, ciò che abbiamo ottenuto è il massimo che si può senza violare le regole.
- **Flusso Massimo:** È l'ottimo globale, cioè il flusso più alto che si può ottenere anche a costo di modificare scelte precedenti (**backtracking**).

Per evitare il problema del backtracking, utilizziamo il concetto di **flusso netto**. Data una rete $G = (V, E)$ con capacità $c : E \rightarrow R_0^+$ e con un **flusso reale** $\varphi : V \times V \rightarrow R_0^+$, poniamo:

$$f_\varphi(u, v) = \varphi(u, v) - \varphi(v, u) \quad \forall (u, v) \in V \times V$$



Nel grafico che vediamo, il flusso netto viene calcolato da un nodo al precedente. Da questo segue il fatto che il valore è negativo. Il flusso netto gode delle seguenti proprietà:

- **Vincolo di capacità:** Il flusso netto da un vertice all'altro non può eccedere la corrispondente capacità:

$$f_\varphi(u, v) \leq c(u, v), \quad \text{per tutti gli } u, v \in V.$$

Dimostrazione:

$$f_\varphi(u, v) = \varphi(u, v) - \varphi(v, u) \leq \varphi(u, v) \leq c(u, v)$$

- **Antisimmetria:** Il flusso netto in una direzione è l'opposto del flusso nella direzione inversa:

$$f_\varphi(u, v) = -f_\varphi(v, u), \quad \text{per tutti gli } u, v \in V.$$

Segue da ciò che il flusso netto da un vertice a sé stesso deve essere 0 poiché per ogni $u \in V$ abbiamo che $f_\varphi(u, u) = -f_\varphi(u, u)$ il che implica che $f_\varphi(u, u) = 0$

- **Conservazione del flusso:** Per ogni nodo intermedio (né sorgente né pozzo), la somma algebrica dei flussi netti uscenti è nulla:

$$\sum_{v \in V} f_\varphi(u, v) = 0, \quad \text{per tutti gli } u \in V - \{s, t\}$$

Cioè, il flusso netto uscente da u è nullo poiché quello che entra è uguale a quello che esce. Allo stesso modo vale che:

$$\sum_{v \in V} f_\varphi(v, u) = 0$$

Cioè, il flusso netto entrante in un vertice di transizione deve essere obbligatoriamente nullo.

Sia $G = (V, E, c, s, t)$ una rete di flusso con funzione capacità c , sorgente s e pozzo t e sia $g : V \times V \rightarrow R$ una funzione che soddisfi:

- Il vincolo di capacità (relativamente a c);
- La proprietà di antisimmetria;
- La legge di conservazione del flusso (relativamente a s e a t).

Allora la funzione $\varphi_g : V \times V \rightarrow R_0^+$ definita da:

$$\varphi_g(u, v) = \begin{cases} g(u, v) & \text{se } g(u, v) > 0 \\ 0 & \text{altrimenti} \end{cases}$$

È un **flusso reale** in $G = (V, E, c, s, t)$ che induce il flusso netto g , cioè:

$$g = f_{\varphi_g}$$

Dimostriamo che è possibile convertire g in un flusso reale φ_g :

- **Vincolo di capacità per φ_g :** Per definizione abbiamo impostato:

$$\varphi_g(u, v) = \begin{cases} g(u, v) & \text{se } g(u, v) > 0 \\ 0 & \text{altrimenti} \end{cases}$$

Verifichiamo che rispetti la capacità $c(u, v)$:

- Se $g(u, v) > 0$, allora $\varphi_g(u, v) = g(u, v) \leq c(u, v)$ (per ipotesi su g).
- Se $g(u, v) \leq 0$, allora $\varphi_g(u, v) = 0 \leq c(u, v)$ (poiché le capacità sono non negative).

E quindi, in ogni caso:

$$\varphi_g(u, v) \leq c(u, v)$$

- **Conservazione del Flusso per φ_g :** Sappiamo che per ogni coppia $u, v \in V$:

$$\varphi_g(u, v) - \varphi_g(v, u) = g(u, v) \quad (*)$$

Infatti, analizziamo i tre casi possibili per il valore di $g(u, v)$:

$$\varphi_g(u, v) - \varphi_g(v, u) = \begin{cases} g(u, v) - 0 = g(u, v) & \text{se } g(u, v) > 0 \\ 0 - 0 = g(u, v) & \text{se } g(u, v) = 0 \\ 0 - g(v, u) = -g(v, u) = g(u, v) & \text{se } g(u, v) < 0 \end{cases}$$

¹ Sia adesso $u \in V - \{s, t\}$, allora sappiamo che:

$$\begin{aligned} \sum_{v \in V} \varphi_g(u, v) - \sum_{v \in V} \varphi_g(v, u) &= \sum_{v \in V} (\varphi_g(u, v) - \varphi_g(v, u)) \\ &= \sum_{v \in V} g(u, v) \quad (\text{Per la proprietà } (*)) \\ &= 0 \quad (\text{Per ipotesi di conservazione su } g) \end{aligned}$$

Quindi il flusso reale si conserva:

$$\sum_{v \in V} \varphi_g(u, v) = \sum_{v \in V} \varphi_g(v, u)$$

Pertanto, g è esattamente uguale al flusso netto indotto da φ_g .

Il **valore** $|f|$ di un flusso f è definito come:

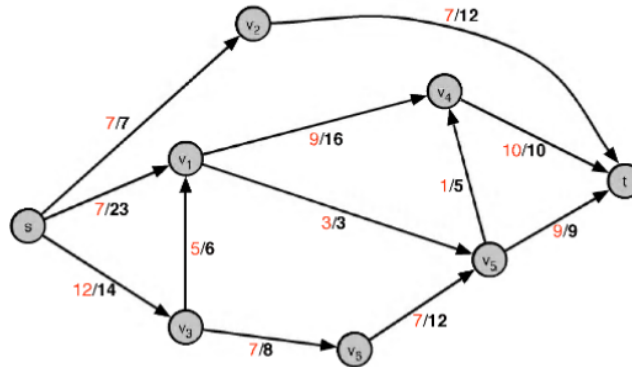
$$|f| = \sum_{v \in V} f(s, v)$$

Se f_ϕ è un flusso netto indotto da un flusso reale φ , allora:

$$\begin{aligned} |f_\phi| &= \sum_{v \in V} f_\phi(s, v) \\ &= \sum_{v \in V} (\varphi(s, v) - \varphi(v, s)) \\ &= \sum_{v \in V} \varphi(s, v) - \sum_{v \in V} \varphi(v, s) \\ &= |\varphi|. \end{aligned}$$

D'ora in poi indicheremo come **Problema del flusso massimo** il problema di determinare un **flusso netto** f in G di valore massimo da s a t .

La figura mostra un esempio di flusso massimo in una rete. Il valore di tale flusso è dato da $|f| = f(s, v_1) + f(s, v_2) + f(s, v_3) = 7 + 7 + 12 = 26$.

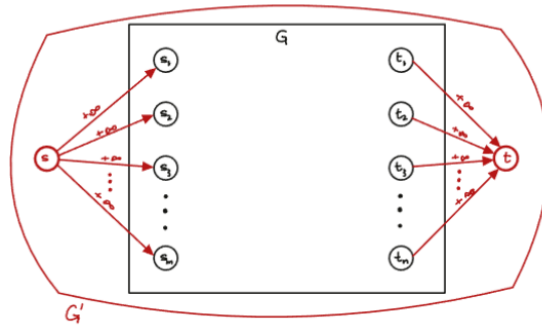


¹Ricordiamo che $g(u, v) = -g(v, u)$ dunque, se uno è positivo, l'altro è per forza negativo e, dunque, scatta lo zero.

Il problema di flusso massimo su una rete G con più sorgenti $\{s_1, s_2, \dots, s_m\}$ e più pozzi $\{t_1, t_2, \dots, t_n\}$ può essere ricondotto a un problema di flusso massimo ordinario. Questo viene fatto trasformando la rete G in una rete di flusso ordinaria G' con singola sorgente e singolo pozzo nel modo seguente:

- si aggiunge una **super-sorgente** s e un arco orientato (s, s_i) , per ogni $i = 1, \dots, m$, con capacità $c(s, s_i) = \infty$
- si aggiunge un **super-pozzo** t e un arco orientato (t_j, t) , per ogni $j = 1, \dots, n$, con capacità $c(t_j, t) = \infty$

È facile verificare che ogni flusso nella rete G' corrisponde a un flusso nella rete G .



Lemma 1

Siano X e Y due insiemi di vertici allora:

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$$

Sia $G = (V, E)$ una rete di flusso, e sia f un flusso in G . Allora:

- Per $X, Y \subseteq V$ si ha $f(X, Y) = -f(Y, X)$;
- Per $X \subseteq V$ si hanno $f(X, X) = 0$ e $f(\emptyset, X) = 0$;
- Per $X, Y, Z \subseteq V$, si hanno

$$\begin{aligned} f(X \cup Y, Z) &= f(X, Z) + f(Y, Z) - f(X \cap Y, Z) \quad \text{e} \\ f(Z, X \cup Y) &= f(Z, X) + f(Z, Y) - f(Z, X \cap Y). \end{aligned}$$

Quindi, nel caso in cui $X \cap Y = \emptyset$, si hanno

$$\begin{aligned} f(X \cup Y, Z) &= f(X, Z) + f(Y, Z) \quad \text{e} \\ f(Z, X \cup Y) &= f(Z, X) + f(Z, Y). \end{aligned}$$

Segue che il valore di un flusso è uguale al flusso netto totale entrante nel pozzo, cioè

$$|f| = f(V, t)$$

Dimostrazione:

$$\begin{aligned}
 |f| &= f(s, V) && \text{(per definizione)} \\
 &= f(V, V) - f(V - s, V) && \text{(per il Lemma 1)} \\
 &= f(V, V - s) && \text{(per il Lemma 1)} \\
 &= f(V, t) + f(V, V - s - t) && \text{(per il Lemma 1)} \\
 &= f(V, t) && \text{(per la conservazione del flusso)}
 \end{aligned}$$

9.2 Metodo Ford-Fulkerson

Il metodo **Ford-Fulkerson** è iterativo. Partiamo con $f(u, v) = 0$ per ogni nodi, ottenendo un flusso iniziale nullo. A ogni iterazione, il flusso viene incrementato lungo un **cammino aumentante**, ovvero un cammino dalla sorgente al pozzo lungo il quale si può immettere ulteriore flusso. Iteriamo questo processo finché non ci sono cammini aumentanti e, dunque otteniamo il flusso massimo.

```

Ford-Fulkerson( $G, s, t$ )
1  si inizializzi il flusso  $f$  a 0
2  while esiste un cammino aumentante  $p$ 
3      do si incrementi il flusso  $f$  lungo  $p$ 
4  return  $f$ 

```

Definiamo come **capacità residua** la quantità di flusso netto addizionale che possiamo inviare su di un certo arco prima di saturarlo. Questa si calcola nel seguente modo:

$$c_f(u, v) = c(u, v) - f(u, v)$$

Quando il flusso netto $f(u, v)$ è negativo, allora vuol dire che la capacità residua $c_f(u, v)$ è maggiore rispetto alla capacità $c(u, v)$. Questo ci viene utile nel momento in cui l'algoritmo deve capire quanto può eliminare in termini di flusso da quell'arco. Chiamiamo **rete residua** di G indotta da f la rete $G_f = (V, E_f)$ dove:

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}$$

Ogni arco nella rete è, quindi, chiamato **arco residuo** e questo può ammettere flusso netto strettamente positivo. Questi archi possono provenire da due fonti:

- E (Archi originali): I tubi veri che hanno ancora spazio vuoto;
- E^T (Archi trasposti/inversi): I tubi dai quali è possibile togliere il flusso.

Vale che $|E_f| \leq 2E$. Data una rete di flusso, un **cammino aumentante** p è un cammino semplice dalla sorgente al pozzo nella rete residua G_f . Per definizione di rete residua, ogni arco (u, v) su un cammino aumentante ammette un flusso netto positivo addizionale senza violare il vincolo di capacità dell'arco.

Lemma 2

Sia (G, c) una rete di flusso e sia f un flusso di G e sia p un cammino aumentante in G_f . Si definisca una funzione $f_p : V \times V \rightarrow R$ come:

$$f_p(u, v) = \begin{cases} c_f(p) & \text{se } (u, v) \text{ è su } p \\ -c_f(p) & \text{se } (v, u) \text{ è su } p \\ 0 & \text{altrimenti} \end{cases}$$

Dimostriamo che f_p è un flusso in G_f :

- **Vincolo di Capacità:** Consideriamo i due casi:

- Se $f_p(u, v) \leq 0$, allora $f_p \leq c_f(u, v)$ per definizione dato che c_f non può essere negativo e, inoltre, vuol dire che quell'arco non appartiene al cammino aumentante.
- Se $f_p(u, v) > 0$, allora (u, v) fa parte del cammino aumentante e, quindi, il vincolo è banalmente rispettato poiché:

$$f_p(u, v) = c_f(p) = \min\{c_f(u', v') \mid (u', v') \in p\} \leq c_f(u, v)$$

- **Antisimmetria:** Dobbiamo verificare che $f_p(u, v) = -f_p(v, u)$ per ogni $(u, v) \in V \times V$. Dunque:

- Se $f_p(u, v) = 0$: Significa che $(u, v) \notin p$ e $(v, u) \notin p$. Di conseguenza anche $f_p(v, u) = 0$.

$$0 = -0 \quad \checkmark$$

- Se $f_p(u, v) \neq 0$: Allora uno dei due archi appartiene al cammino p .

* Caso $(u, v) \in p$:

$$f_p(u, v) = c_f(p) \quad \text{e} \quad f_p(v, u) = -c_f(p)$$

$$\Rightarrow f_p(u, v) = -f_p(v, u)$$

* Caso $(v, u) \in p$:

$$f_p(v, u) = c_f(p) \quad \text{e} \quad f_p(u, v) = -c_f(p)$$

$$\Rightarrow f_p(u, v) = -f_p(v, u)$$

- **Conservazione:** Dobbiamo verificare che per ogni nodo intermedio $u \in V \setminus \{s, t\}$, il flusso netto uscente è nullo: $\sum_{v \in V} f_p(u, v) = 0$.

Analizziamo due casi per il nodo u :

- **Caso $u \notin p$ (il nodo non è sul cammino):** Per ogni $v \in V$, né (u, v) né (v, u) sono su p . Quindi $f_p(u, v) = 0$ per ogni v .

$$\sum_{v \in V} f_p(u, v) = 0$$

- **Caso $u \in p$ (il nodo è sul cammino):** Poiché u è un nodo interno del cammino semplice p , esiste un unico vertice v' che precede u e un unico vertice v'' che segue u nel cammino ($v' \rightarrow u \rightarrow v''$).

I valori del flusso sono:

$$\begin{aligned} v = v'' &\implies f_p(u, v'') = c_f(p) \quad (\text{flusso in avanti}) \\ v = v' &\implies f_p(u, v') = -c_f(p) \quad (\text{flusso all'indietro verso il predecessore}) \\ v \notin \{v', v''\} &\implies f_p(u, v) = 0 \end{aligned}$$

Facendo la somma totale:

$$\sum_{v \in V} f_p(u, v) = c_f(p) - c_f(p) + 0 = 0$$

Lemma 3

Sia (G, c) una rete di flusso e sia f un flusso di G . Sia G_f la rete residua di G indotta da f e sia f' un flusso in G_f , allora la somma $f + f'$ è un flusso in G con valore $|f + f'| = |f| + |f'|$. Cioè, Se hai un flusso esistente (f) e trovi un flusso aggiuntivo (f') nella rete residua, puoi sommarli matematicamente e il risultato sarà ancora un flusso valido per la rete originale, ma più grande. Dimostriamo che è un flusso:

- **Antisimmetrica:** Sappiamo che, per definizione, $(f + f')(u, v) = f(u, v) + f'(u, v)$. Dunque vediamo che:

$$f(u, v) + f'(u, v) = -f(v, u) - f'(v, u) = -(f(v, u) + f'(v, u)) = -(f + f')(v, u)$$

Cioè la proprietà è rispettata.

- **Vincolo di capacità:** Dobbiamo essere sicuri che sommando flussi non scoppiamo i tubi (non superiamo $c(u, v)$). Vediamo che:

$$\begin{aligned} (f + f')(u, v) &= f(u, v) + f'(u, v) \leq f(u, v) + c_f(u, v) \\ &\leq f(u, v) + (c(u, v) - f(u, v)) = c(u, v) \end{aligned}$$

- **Conservazione del flusso:** Sappiamo che, per definizione:

$$\begin{aligned} \sum_{v \in V} (f + f')(u, v) &= \sum_{v \in V} (f(u, v) + f'(u, v)) \\ &= \sum_{v \in V} f(u, v) + \sum_{v \in V} f'(u, v) = 0 + 0 = 0 \end{aligned}$$

Dunque, sia (G, c) una rete di flusso e sia f un flusso di G , sia p un cammino aumentante in G_f . Sia f_p il flusso in G_f definito come nel lemma 3 e sia $f' : V \times V \rightarrow R$ la funzione:

$$f' = f + f_p$$

Allora f' è un flusso in G con valore $|f'| = |f| + |f_p| > |f|$. Scriveremo dunque la procedura Ford Fulkerson come segue:

```

Ford-Fulkerson( $G, c, s, t$ )
  - sia  $G = (V, E)$ 
1. for  $(u, v) \in E$  do
2.    $f(u, v) = 0$ 
3.    $f(v, u) = 0$ 
4. while esiste un cammino aumentante  $p$  da  $s$  a  $t$  nella rete residua  $G_f$  do
5.    $c_f(p) = \min\{c_f(u, v) : (u, v) \text{ è in } p\}$ 
6.   for  $(u, v)$  in  $p$  do
7.      $f(u, v) = f(u, v) + c_f(p)$ 
8.      $f(v, u) = -f(u, v)$ 
9. return  $f$ 

```

Analizziamo il tempo di esecuzione della procedura:

- Per il ciclo all'inizio, abbiamo un tempo $O(E)$.
- Per il ciclo while da 4 a 8, questo può essere eseguito un massimo di $|f^*|$ poiché il valore del flusso aumenta almeno di una unità ogni iterazione. Un cammino aumentante a ogni iterazione può essere calcolato in tempo $O(E)$

La procedura ha un tempo di esecuzione pari a $O(E|f^*|)$ dove f^* è il flusso massimo trovato dall'algoritmo. È importante notare che l'algoritmo potrebbe non terminare in caso di numeri non razionali.² Si noti che è possibile, scegliendo il cammino aumentante p con una visita in ampiezza, migliorare la complessità temporale a $O(VE^2)$. Questa variante si chiama **Algoritmo di Edmonds-Karp** e non ha problemi in caso di esecuzione con numeri irrazionali.

9.3 Tagli in Reti di flusso

Definiamo un **Taglio** (S, T) in una rete di flusso (G, c) con sorgente s e pozzo t una partizione di V in S e $T = V - S$ tale che $s \in S$ e $t \in T$. Se f è un flusso, allora il **flusso netto attraverso il taglio** (S, T) è definito come:

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v)$$

La **capacità del taglio** è:

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

Lemma 4

Sia f un flusso su una rete con sorgente s e pozzo t , e sia (S, T) un taglio. Allora il flusso netto attraverso (S, T) è $f(S, T) = |f|$. Si dimostra banalmente poiché:

$$\begin{aligned}
 f(S, T) &= f(S, V) - f(S, S) && \text{Consideriamo che } f(S, S) = 0 \\
 &= f(S, V) = f(s, V) + f(S - s, V) && f(S - s, V) = 0 \text{ poiché sono nodi intermedi} \\
 &= |f|
 \end{aligned}$$

²C'è un pdf che mostra un esempio di questa casistica che non ho inserito.

Ne segue che il valore di un qualunque flusso f in una rete di flusso è limitato superiormente dalla capacità di un qualunque taglio di G . Si può dimostrare come segue:³

$$\begin{aligned} |f| &= f(S, T) \\ &= \sum_{u \in S, v \in T} f(u, v) \\ &\leq \sum_{u \in S, v \in T} c(u, v) = c(S, T) \end{aligned}$$

Ne segue che $\sup(|f|) \leq \min c(S, T)$. Da qui ne possiamo derivare il teorema che segue.

9.4 Teorema del Massimo Flusso/Minimo Taglio

Sia f^* un flusso in una rete di flusso con sorgente s e pozzo t , allora abbiamo che le seguenti proprietà sono equivalenti:⁴

- $|f^*|$ è un flusso massimo in G .
- La rete residua G_{f^*} non contiene alcun cammino aumentante.
- $|f^*| = c(S^*, T^*)$ per qualche taglio di G .

Ne segue che il massimo flusso $|f|$ è uguale al minimo $c(S, T)$. Che vuol dire che la massima quantità di acqua che puoi spingere da A a B è esattamente uguale alla capacità del collo di bottiglia' più stretto della rete.

1. (1 $\implies$ 2) (Per assurdo)

Supponiamo per assurdo che f^* sia massimo ma che esista un cammino aumentante p in G_{f^*} .

- Possiamo aumentare il flusso lungo p di una quantità $f_p > 0$.
- Il nuovo flusso sarà $f' = f^* + f_p$.
- Il valore del nuovo flusso è $|f'| = |f^*| + |f_p| > |f^*|$.

Questo contraddice l'ipotesi che f^* fosse massimo.

³Ciao Angelo, io non so se stai leggendo questa cosa, tuttavia se lo stai facendo sappi che sto imprecando mentalmente. P.S.: Palle

⁴Cioè, se ne vale una valgono tutte.

2. (2 $\implies$ 3) (Costruttiva)

Supponiamo che non esistano cammini aumentanti da s a t in G_{f^*} . Costruiamo l'insieme S^* dei nodi raggiungibili da s nella rete residua:

$$S^* = \{v \in V \mid \text{esiste un cammino } s \rightarrow v \text{ in } G_{f^*}\}$$

Poniamo $T^* = V \setminus S^*$.

- Poiché t non è raggiungibile, $t \in T^*$, quindi (S^*, T^*) è un taglio valido.
- Per ogni arco (u, v) con $u \in S^*$ e $v \in T^*$, deve valere $f(u, v) = c(u, v)$ (l'arco è saturo), altrimenti (u, v) esisterebbe nella rete residua e v sarebbe in S^* .
- Di conseguenza, il flusso netto attraverso il taglio eguaglia la capacità:

$$|f^*| = f(S^*, T^*) = \sum_{u \in S^*, v \in T^*} c(u, v) = c(S^*, T^*)$$

3. (3 $\implies$ 1) (Deduzione diretta)

Supponiamo che $|f^*| = c(S^*, T^*)$. Per il corollario della dualità debole, sappiamo che per *qualsiasi* flusso f e *qualsiasi* taglio (S, T) vale sempre:

$$|f| \leq c(S, T)$$

In particolare:

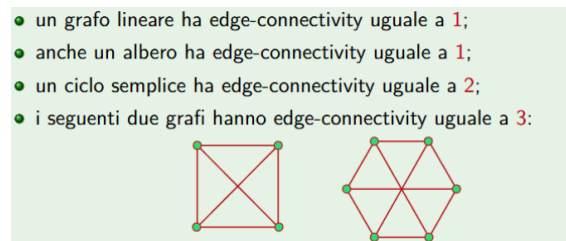
$$|f| \leq c(S^*, T^*) = |f^*|$$

Poiché nessun flusso può superare questo valore, f^* deve essere necessariamente un flusso massimo.

Capitolo 10

Edge Connectivity

Sia $G = (V, E)$ un grafo non orientato connesso. L'**edge-connectivity** di G è il **minimo** numero di archi che occorre rimuovere da G affinché il grafo sia disconnesso.



Un **insieme di disconnessione** per G è un insieme minimale $D \subseteq E$ di archi di G tale che $(V, E - D)$ sia disconnesso. Indichiamo con $Disconn(G)$ la collezione di tutti gli insiemi di disconnessione per G . Avremo che $edge - connectivity(G) = \min |D|$.

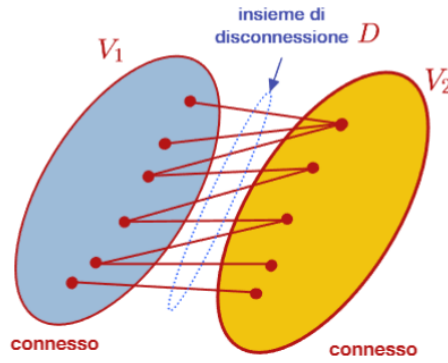
Definiamo un **Taglio** in $G = (V, E)$ è un partizione non banale $|\langle V_1, V_2 \rangle|$ di V tale che nessuno degli insiemi è vuoto. La **dimensione** del taglio è data dal numero di archi che lo attraversano cioè:

$$|\langle V_1, V_2 \rangle| = |\{(u, v) \in V_1 \times V_2 : (u, v) \in G\}|$$

Lemma 1

Sia $D \subseteq E$ un insieme di disconnessione per un grafo connesso $G = (V, E)$, allora:

1. Il sottografo $(V, E - D)$ ha esattamente due componenti connesse.
2. Il taglio in G introdotto da D è attraversato da tutti e soli gli archi in D .



Si consideri un arco qualunque $e \in D$, poiché:

- $(V, E - D)$ non è connesso.
- $(V, (E - D) \cup \{e\})$ è connesso per via della minimalità di D . (e il grafo rimanesse rotto anche dopo aver rimosso e , vorrebbe dire che e non serviva a nulla per rompere il grafo)

Ne consegue che $(V, E - D)$ è formato da 2 componenti connesse. Siano (V_1, E_1) e (V_2, E_2) le due componenti connesse, il taglio $|\langle V_1, V_2 \rangle|$ indotto da D non è attraversato alcun arco in $E - D$. Dunque il taglio è attraversato tutti e soli dagli archi di D .

Sappiamo che ogni insieme di disconnessione minimale (D) crea un taglio perfetto, attraversato esattamente dagli archi di D . Di conseguenza, se ogni disconnessione è un taglio, allora la dimensione del taglio più piccolo possibile ($\min |\langle V_1, V_2 \rangle|$) non può essere più grande della disconnessione più piccola ($\min |D|$). Dunque:

$$\min \text{Taglio} \leq \min \text{Disconnessione}$$

Viceversa, se prendiamo un taglio qualsiasi. Gli archi che attraversano questo taglio separano il grafo. Magari ne hai presi troppi (non è minimale), ma sicuramente dentro quel mucchio di archi c'è un sottoinsieme che basta a disconnettere il grafo. Di conseguenza, a disconnessione più piccola possibile ($\min |D|$) deve essere per forza più piccola o uguale a quel mucchio di archi del taglio. Dunque:

$$\min \text{Disconnessione} \leq \min \text{Taglio}$$

Pertanto $\min \text{Taglio} = \min \text{Disconnessione}$ e, dunque, l'edge connectivity del grafo è uguale al minimo taglio possibile.

Dato un grafo non orientato G , indichiamo $\bar{G}$ il grafo orientato ottenuto da G introducendo due archi orientati opposti al posto dell'arco non orientato. Una volta fatto ciò, possiamo definire la **rete unitaria** inserendo $c(u, v) = 1$ per ogni arco nel grafo $\bar{G}$.

Lemma 2

Sia $G = (V, E)$ un grafo non orientato connesso e sia $|\langle V_1, V_2 \rangle|$ un taglio in G . Siano $s, t \in V$ tali che $s \in V_1$ e $t \notin V_2$ e sia $(\bar{G}, s, t, c)$ la rete unitaria indotta da G, s e t ,

allora:

$$|\langle V_1, V_2 \rangle| = c(V_1, V_2) = c(V_2, V_1)$$

In pratica, il numero di archi fisici che tagli nel grafo originale ($|\langle V_1, V_2 \rangle|$) è esattamente uguale alla capacità matematica del taglio nella nuova rete di flusso ($c(V_1, V_2)$). Per dimostrarlo, siano:¹

$$e_1 = (v_{11}, v_{12}), \dots, e_k = (v_{k1}, v_{k2})$$

gli archi di G che attraversano il taglio. Allora:

$$\begin{aligned} |\langle V_1, V_2 \rangle| &= k \\ c(V_1, V_2) &= \sum_{i=1}^k c(v_{i1}, v_{i2}) = k \\ c(V_2, V_1) &= \sum_{i=1}^k c(v_{i2}, v_{i1}) = k \end{aligned}$$

Da cui la tesi.

Lemma 3

Sia $G = (V, E)$ un grafo non orientato e sia $s \in V$. Esiste $t \in V - \{s\}$ per cui la rete unitaria indotta da G ha un taglio minimo la cui capacità eguaglia l'edge connectivity di G . *Dimostrazione*²

Sia $\langle V_1, V_2 \rangle$ un taglio di dimensione minima in G di dimensione uguale alla edge connectivity di G . Supponiamo che s sia in V_1 e $t \in V_2$. Allora (V_1, V_2) è un taglio della rete unitaria la cui capacità è uguale alla dimensione del taglio $\langle V_1, V_2 \rangle$ in G , pertanto (V_1, V_2) è un taglio minimo.

```

CONSTRUCT-MINIMUM-CUT( $V, E$ )
- sia  $s \in V$ 
 $\mathcal{T} := \langle \{s\}, V \setminus \{s\} \rangle$     --  $\mathcal{T}$ , inizializzato ad un taglio qualunque;
                                alla fine conterrà un taglio minimo
for  $t \in V \setminus \{s\}$  do
  - sia  $(V, \vec{E}, s, t, c)$  la rete di flusso a capacità unitaria indotta
    da  $G = (V, E)$  e  $s, t \in V$ 
  - si determini un taglio minimo  $\langle V_1, V_2 \rangle$  nella rete  $(V, \vec{E}, s, t, c)$ 
    mediante l'algoritmo di Edmonds-Karp
  if  $|\langle V_1, V_2 \rangle| < |\mathcal{T}|$  then
     $\mathcal{T} := \langle V_1, V_2 \rangle$ 
return  $\mathcal{T}$ 

```

Dunque, per ciascuna esecuzione dell'algoritmo **Edmonds-Karp** nel ciclo for, richiede al più $|V|$ incrementi di flusso, dunque ha complessità $O(VE)$. Poiché vi sono complessivamente $|V| - 1$ esecuzioni di Edmonds-Karp, Construct Minimum Cut ha complessità $O(V^2E)$.

¹Giuro che è l'ultima dimostrazione

²Mentivo